Q1. **FAIRSHARE AI - STOCK ALLOCATION AND SHORTAGE DECISION PLATFORM**

**PROBLEM STATEMENT**

Wrenfield Hygiene makes hand wash, hand sanitiser, surface disinfectant, laundry liquid, washing-up liquid and toilet cleaner, and ships every case from one distribution centre to **180** business customers: supermarkets, convenience groups, discounters, wholesalers, online retailers, pharmacy chains, healthcare suppliers serving hospitals and care homes, and contract cleaning companies. When a product runs short, its orders add up to far more than the cases there are, and the product is **on allocation**: every order for it waits until the supply desk decides how many cases each customer gets.

Orders arrive by three routes, and each writes them its own way. One customer's account is **C0021** on an **EDI** order, **CUS-0021** on the **Portal** and **Acct 21** in an **Email**, and one thousand cases of a twelve-bottle product are **12000 units** by EDI, **1000 cs** by the Portal and **20 plt** of fifty-case pallets by Email. The same purchase order sometimes arrives twice, by two routes.

How many cases a customer should get depends on three things held in three places: the product category's own allocation policy, one of five written from one template that differ in four rules — the weeks of shipments behind a customer's usual demand, the cap on an order, the customers protected and served first, and the reserve that may be held back; the shipment history, a row for each week a customer received the product; and the open orders, written in three notations, some of them twice. The shortage queue is larger than the desk can work in the order the products went short.

Wrenfield requires a system that ranks the queue by how short each product will be next week, divides each product's cases among its customers exactly as the category's own policy says, assembles an allocation review with a recommended decision, and stops there: no customer is told anything and no case is confirmed until a **Supply Planning Manager** who did not open the review decides.

**OBJECTIVE**

Build **FairShare AI**, a working stock allocation and shortage decision platform. The finished system must:

- Clean five raw extracts that arrive with data-quality defects, and report what cleaning did to each.
- Train two machine learning models on tabular data, one predicting whether a customer will cut an order placed during a shortage once supply recovers and one forecasting the cases a customer will need next week, and one NLP model naming the request a customer's email makes.
- Build a searchable knowledge base from sixteen company documents, every statement traceable to its document.
- Read each product category's four rules — the weeks of history, the cap, the protected customers and the reserve — from that category's own allocation policy. No rule is written into the solution or the configuration.
- Read every account code and quantity through the notation of the route that wrote it, keep an order sent twice once, work out each customer's usual demand and eligible cases exactly, and divide the cases among the customers with every case accounted for.
- Run a multi-agent workflow in which each agent is one language model call that interprets established facts and decides no case.
- Stop the workflow before any customer is told, wait for a decision by a **Supply Planning Manager** other than the person who opened the review, and survive a restart while waiting.
- Record every run, review, decision, agent's language model call and chat question for audit.
- Serve everything through a REST service and a dashboard; the dashboard holds no business rule and reads no file.
- Require sign-in, offer each of the three roles only its own pages, and refuse every other request in the service itself.
- Answer chat questions from the documents the chat may read, in one language model call each, refusing when the documents do not cover the question, and telling a **Customer Service Adviser** about an allocation only for an account whose code and delivery postcode the question gives and the customer master holds.

**PROJECT STRUCTURE**

```
Project/
|-- .env                              <- pre-loaded, your key goes in it
|-- .streamlit/                       <- pre-loaded
|   '-- config.toml
|-- access/
|   |-- password_security.py
|   '-- permissions.py
|-- agentic/
|   |-- agents.py
|   |-- graph.py
|   |-- nodes.py
|   '-- state.py
|-- api/
|   |-- routes.py
|   '-- schemas.py
|-- chat/
|   |-- allocation_assistant.py
|   |-- allocation_status_answer.py
|   '-- conversation_memory.py
|-- config/
|   '-- config.py                     <- pre-loaded, complete, do not edit
|-- dashboard/
|   |-- allocation_chat.py
|   |-- allocation_detail.py
|   |-- api_client.py
|   |-- audit_trail.py
|   |-- decision_queue.py
|   '-- shortage_queue.py
|-- data/                             <- pre-loaded
|   |-- docs/                         <- the sixteen PDF documents
|   |-- input/
|   |   |-- customer_master.csv
|   |   |-- open_customer_emails.csv
|   |   |-- open_orders.csv
|   |   |-- shipment_history.csv
|   |   '-- shortage_products.csv
|   |-- customer_emails.csv
|   |-- order_history_eval.csv
|   |-- order_history_train.csv
|   |-- weekly_demand_eval.csv
|   '-- weekly_demand_train.csv
|-- data_pipeline/
|   |-- common_cleaning.py
|   |-- dataset_loader.py
|   |-- demand_cleaning.py
|   |-- email_cleaning.py
|   |-- feature_builder.py
|   |-- order_cleaning.py
|   '-- quality_profiler.py
|-- database/                         <- empty
|   |-- checkpoint.db                 <- created by run_api_service.py
|   |-- chroma_store/                 <- created by run_knowledge_indexing.py
|   '-- fairshare.db                  <- created by run_database_setup.py
|-- knowledge/
|   |-- chunk_splitting.py
|   |-- pdf_extraction.py
|   |-- policy_reading.py
|   '-- vector_index.py
|-- modeling/
|   |-- demand_model.py
|   |-- inflated_order_model.py
|   |-- model_evaluation.py
|   |-- model_registry.py
|   '-- request_model.py
|-- output/                           <- empty
|   |-- models/                       <- empty; filled by run_model_training.py
|   |-- data_quality_report.json      <- created by run_data_pipeline.py
|   '-- model_metrics.json            <- created by run_model_training.py
|-- persistence/
|   |-- database.py
|   |-- models.py
|   '-- repository.py
|-- processed/                        <- empty; filled by run_data_pipeline.py
|-- tools/
|   |-- allocation_tools.py
|   |-- prediction_tools.py
|   |-- retrieval_tools.py
|   '-- shortage_data_tools.py
|-- requirements.txt                  <- pre-loaded
|-- run_api_service.py
|-- run_dashboard.py
|-- run_data_pipeline.py
|-- run_database_setup.py
|-- run_knowledge_indexing.py
'-- run_model_training.py
```

## DATASET DESCRIPTION

### order_history_train.csv —

| Column | Type once cleaned | What it holds |
|---|---|---|
| order_line_id | **str** | The order log's own identifier for the line. **config.ORDER_LINE_ID_COLUMN** names it |
| order_reference | **str** | The purchase order, in the form **PO-NNNNNN**. One order is one purchase order and one product. **config.ORDER_REFERENCE_COLUMN** names it |
| order_route | **str** | How the order arrived: **EDI**, **Portal** or **Email**. **config.ORDER_ROUTE_COLUMN** names it |
| account_code | **str** | The customer, written in the notation of the route; once cleaned, its canonical code **CUS-NNNN**. **config.ACCOUNT_CODE_COLUMN** names it |
| customer_name | **str** | The customer's name, sometimes padded or ending in **Ltd** or **Limited**. **config.CUSTOMER_NAME_COLUMN** names it |
| received_sequence | **int** | The line's place in the order log; a second arrival of an order always comes later. **config.RECEIVED_SEQUENCE_COLUMN** names it |
| product_code | **str** | The product, two capital letters, a hyphen and its pack size. **config.PRODUCT_CODE_COLUMN** names it |
| product_category | **str** | The product's category, and so its allocation policy: one of the five categories. **config.PRODUCT_CATEGORY_COLUMN** names it |
| pack_description | **str** | The units in a case and the size of a unit, written four ways; once cleaned, in the form **12x500ml**. **config.PACK_DESCRIPTION_COLUMN** names it |
| cases_per_pallet | **int** | Cases on one of the product's pallets. **config.CASES_PER_PALLET_COLUMN** names it |
| ordered_quantity | **int** | What was ordered, written in the notation of the route; once cleaned, whole cases. **config.ORDERED_QUANTITY_COLUMN** names it |
| ordered_vs_usual_pct | **float** | The order as a percentage of the customer's average week over the last four weeks, as the order system shows it. Written with a sign on some rows |
| orders_this_week | **int** | Orders the customer placed for the product that week, this one included. Written with a unit on some rows |
| same_order_other_route_flag | **str** | The same purchase order also arrived by another route |
| lead_time_days | **int** | Days between the order and the delivery it asks for. Written with a unit on some rows |
| channel | **str** | The customer's channel: one of the eight channels. **config.CHANNEL_COLUMN** names it |
| region | **str** | The customer's region |
| sites_supplied | **int** | Stores, hospitals and homes, client sites, depots or fulfilment centres the customer supplies. Written with a unit on some rows |
| customer_stock_cover_weeks | **float** | Weeks of the product on the customer's shelves, from its own stock report. Written with a unit on some rows |
| account_age_weeks | **int** | Weeks since the account was opened. Written with a unit on some rows |
| past_shortage_orders | **int** | Orders the customer placed during shortages in the last **52** weeks |
| past_orders_cut | **int** | How many of those it went on to cut or cancel, once their outcome was known; never more than **past_shortage_orders** |
| key_account_flag | **str** | The customer is a key account |
| shortage_publicised_flag | **str** | The shortage was in the news when the order arrived |
| weeks_on_allocation | **int** | The week of the shortage the order arrived in, **1** the first |
| promotion_flag | **str** | The customer had a promotion of the product planned |
| entered_by | **str** | The order desk clerk who checked the line. Carries no signal |
| weekday_entered | **str** | The weekday the line was entered. **config.ORDER_WEEKDAY_COLUMN** names it. Carries no signal |
| form_version | **str** | The order form template. Carries no signal |
| cut_after_shortage | **str** | **The answer.** The customer cut the order by at least a third, or cancelled it, within four weeks of supply recovering. **config.ORDER_TARGET_COLUMN** names it |

### order_history_train.csv: SAMPLE RECORDS

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| order_line_id | OL-0035613 | OL-0001955 | OL-0017028 |
| order_reference | PO-526631 | PO-409721 | PO-332925 |
| order_route | EDI | Portal | Email |
| account_code | C0021 | CUS-0002 | Acct 40 |
| customer_name | Greenacre Foodstores | Bramwell Care Homes | Walsden Wholesale |
| received_sequence | 13157 | 12788 | 24109 |
| product_code | LL-1000 | SM-0750 | LL-1500 |
| product_category | Laundry | Surface disinfection | Laundry |
| pack_description | 6 x 1000 ml | 6/750ML | 1500ml x4 |
| cases_per_pallet | 70 | 100 | 80 |
| ordered_quantity | 1344 units | 76 cs | 3 plt |
| ordered_vs_usual_pct | 169.6% | 125.6 % | 175.9% |
| orders_this_week | 1 | 1 order | 1 |
| same_order_other_route_flag | 0 | Yes | No |
| lead_time_days | 8 days | 5 days | 8 |
| channel | Grocery multiple | healthcare | Wholesaler |
| region | N. East | East Midlands | East of England |
| sites_supplied | 330 | 28 | 14 |
| customer_stock_cover_weeks | 2.2 | 6.8 | 1.8 |
| account_age_weeks | 463 | 905 | 486 |
| past_shortage_orders | 60 | 20 | 71 |
| past_orders_cut | 42 | 5 | 7 |
| key_account_flag | No | No | 1 |
| shortage_publicised_flag | no | No | No |
| weeks_on_allocation | 1 | 2 | 2 |
| promotion_flag | false | No | FALSE |
| entered_by | OD-104 | OD-104 | OD-109 |
| weekday_entered | Tuesday | Monday | MONDAY |
| form_version | OF-4.2 | OF-4.1 | OF-4.2 |
| cut_after_shortage | Yes | Yes | Yes |

**order_history_eval.csv —**
The same columns from the shortages after, held out for scoring the inflated-order model.

**weekly_demand_train.csv —**
One row per customer, product and week in which every order was filled, over the earlier weeks. Every week in which a promotion or a gain in distribution was planned is kept, and one in ten of the ordinary weeks. The demand model learns from it.

| Column | Type once cleaned | What it holds |
|---|---|---|
| demand_row_id | **str** | The planning extract's own identifier for the row |
| account_code | **str** | The customer, on its canonical code |
| product_code | **str** | The product |
| channel | **str** | The customer's channel |
| region | **str** | The customer's region |
| product_category | **str** | The product's category |
| pack_description | **str** | The units in a case and the size of a unit, written four ways; once cleaned, in the form **12x500ml** |
| sites_supplied | **int** | Sites the customer supplies. Written with a unit on some rows |
| last_week_cases | **int** | Cases delivered last week. Written with a unit on some rows |
| avg_4_week_cases | **float** | Average cases a week over the last four weeks |
| avg_26_week_cases | **float** | Average cases a week over the last **26** weeks |
| promotion_next_week_flag | **str** | A promotion runs next week |
| distribution_change_flag | **str** | The product gains distribution at the customer next week |
| seasonal_index | **int** | The planning team's index for next week, **100** for an average week |
| weeks_since_launch | **int** | Weeks since the product was launched. Written with a unit on some rows |
| key_account_flag | **str** | The customer is a key account |
| planner_code | **str** | The planner who ran the extract. Carries no signal |
| extract_weekday | **str** | The weekday the extract ran. **config.DEMAND_WEEKDAY_COLUMN** names it. Carries no signal |
| extract_batch | **str** | The extract batch. Carries no signal |
| next_week_cases | **int** | **The answer.** The cases the customer took the following week, when nothing was short. **config.DEMAND_TARGET_COLUMN** names it |

**weekly_demand_train.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| demand_row_id | DR-0016319 | DR-0012767 | DR-0000121 |
| account_code | CUS-0012 | CUS-0093 | CUS-0157 |
| product_code | TG-0750 | SB-0750 | DG-1000 |
| channel | Grocery multiple | Chemist | Online |
| region | South East | Ldn | NE |
| product_category | Toilet care | Surface disinfection | Dishwash |
| pack_description | 750ml x12 | 6 x 750 ml | 6 x 1000 ml |
| sites_supplied | 260 sites | 398 | 1 site |
| last_week_cases | 347 cs | 190 | 14 cs |
| avg_4_week_cases | 236.00 | 167.75 | 16.25 |
| avg_26_week_cases | 237.7 | 167.2 | 24.3 |
| promotion_next_week_flag | No | Y | 1 |
| distribution_change_flag | no | false | No |
| seasonal_index | 104 | 98 | 101 |
| weeks_since_launch | 438 | 468 | 290 weeks |
| key_account_flag | FALSE | yes | No |
| planner_code | PL-09 | PL-06 | PL-08 |
| extract_weekday | saturday | thursday | THU |
| extract_batch | EB-04 | EB-04 | EB-03 |
| next_week_cases | 283 | 272 | 22 |

**weekly_demand_eval.csv —**
The same columns from the weeks after, held out for scoring the demand model.

**customer_emails.csv —**
One row per email a customer sent to the supply desk. The request model learns from it.

| Column | Type once cleaned | What it holds |
|---|---|---|
| email_id | **str** | The inbox's own identifier for the email. **config.EMAIL_ID_COLUMN** names it |
| account_code | **str** | The customer, on its canonical code |
| customer_name | **str** | The customer's name |
| channel | **str** | The customer's channel |
| product_code | **str** | The product the email is about; empty when it names none |
| sender_role | **str** | Who wrote it: Buyer, Store manager, Clinical lead, Procurement officer or Operations manager. **config.SENDER_ROLE_COLUMN** names it |
| received_at | **str** | The weekday and time it arrived, the weekday written in several notations. **config.EMAIL_RECEIVED_AT_COLUMN** names it |
| email_sequence | **int** | Its place among the customer's emails, oldest first. **config.EMAIL_SEQUENCE_COLUMN** names it |
| email_subject | **str** | The subject line. **config.EMAIL_SUBJECT_COLUMN** names it |
| email_text | **str** | The email, greeting to signature. **config.EMAIL_TEXT_COLUMN** names it |
| email_length_chars | **int** | Characters in the email. **config.EMAIL_LENGTH_COLUMN** names it, and on some rows it was measured on the text as it arrived, before any repair |
| request_type | **str** | **The answer.** The one request the email makes, one of the eight keys of **config.REQUEST_GUIDE_DOCUMENTS**. **config.EMAIL_TARGET_COLUMN** names it |

**customer_emails.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| email_id | EM-029307 | EM-001669 | EM-026388 |
| account_code | CUS-0162 | CUS-0022 | CUS-0154 |
| customer_name | Ravensdale Village Stores | Emberton Online Grocer | Harbury Wholesale Supplies |
| channel | Convenience | Online | Wholesale |
| product_code | DW-0900 | SM-0750 | TC-5000 |
| sender_role | Store manager | Buyer | Procurement officer |
| received_at | FRIDAY 10:17 | Wednesday 14:35 | Thursday 13:46 |
| email_sequence | 237 | 2 | 98 |
| email_subject | Our order | Withdraw order | Cancel PO-547316 |
| email_text | <p>Good afternoon, Several cases of Wrenfield Dishwash Liquid 90... | HI TEAM, PLEASE CHANGE OUR ORDER. SORRY FOR THE SHORT NOTICE. WE... | Hello, We will not need the 75 cases of the 5 litre toilet... |
| email_length_chars | 334 | 220 | 226 |
| request_type | Delivery complaint | Order cancellation | Order cancellation |

**shortage_products.csv —**
The products on allocation this week, named by **config.SHORTAGE_PRODUCTS_FILE**, one row each. It arrives clean and is read as text.

| Column | Type as read | What it holds |
|---|---|---|
| product_code | **str** | The product. **config.PRODUCT_CODE_COLUMN** names it |
| product_name | **str** | The product's full name. **config.PRODUCT_NAME_COLUMN** names it |
| product_category | **str** | The product's category, which names its allocation policy. **config.PRODUCT_CATEGORY_COLUMN** names it |
| pack_description | **str** | The units in a case and the size of a unit, in the form **12x500ml** |
| units_per_case | **str** | Units in a case, a whole number. **config.UNITS_PER_CASE_COLUMN** names it |
| cases_per_pallet | **str** | Cases on a pallet, a whole number. **config.CASES_PER_PALLET_COLUMN** names it |
| cases_available_this_week | **str** | The cases there are to divide this week. **config.CASES_AVAILABLE_THIS_WEEK_COLUMN** names it |
| cases_available_next_week | **str** | The cases expected next week. **config.CASES_AVAILABLE_NEXT_WEEK_COLUMN** names it |
| shortage_reason | **str** | Why the product is short. **config.SHORTAGE_REASON_COLUMN** names it |
| weeks_on_allocation | **str** | The weeks the product has been on allocation, this one included. **config.WEEKS_ON_ALLOCATION_COLUMN** names it |
| seasonal_index | **str** | The planning team's index for next week, **100** for an average week |

**config.SHORTAGE_PRODUCT_WHOLE_NUMBER_PATTERNS** reads its six whole numbers.

**open_orders.csv —**
Every open order line for the products on allocation, named by **config.OPEN_ORDERS_FILE**, exactly as each route wrote it, with **no answer column**. None of these orders appears in the history.
Its first **29** columns are the order history's columns other than **cut_after_shortage**, in the same order, carrying the order history's defects; the **seven** below follow them, in this order. They are the demand model's inputs the order history does not carry, and the first three are worked out from the shipment history: **last_week_cases** is week **1**, **avg_4_week_cases** weeks **1** to **4** over four and **avg_26_week_cases** all **26** weeks over **26**. Of the first **29**, **ordered_vs_usual_pct** is empty for a customer with nothing shipped in the last four weeks.

| Column | Type as read | What it holds |
|---|---|---|
| last_week_cases | **str** | Cases of the product shipped to the customer last week. Written with a unit on some rows |
| avg_4_week_cases | **str** | Average cases a week over the last four weeks |
| avg_26_week_cases | **str** | Average cases a week over the last **26** weeks |
| promotion_next_week_flag | **str** | A promotion runs next week |
| distribution_change_flag | **str** | The product gains distribution at the customer next week |
| seasonal_index | **str** | The product's seasonal index for next week |
| weeks_since_launch | **str** | Weeks since the product was launched. Written with a unit on some rows |

**config.OPEN_ORDER_VALUE_MAPPINGS**, **config.OPEN_ORDER_WHOLE_NUMBER_PATTERNS**, **config.OPEN_ORDER_DECIMAL_NUMBER_PATTERNS** and **config.OPEN_ORDER_MINIMUM_VALUES** extend the order history's reading rules to these seven columns, with the rules the weekly demand extract shares for them, **config.DEMAND_INPUT_VALUE_MAPPINGS**, **config.DEMAND_INPUT_WHOLE_NUMBER_PATTERNS** and **config.DEMAND_INPUT_MINIMUM_VALUES**.

**customer_master.csv —**
Every customer, named by **config.CUSTOMER_MASTER_FILE**, on its canonical code. It arrives clean and is read as text.

| Column | Type as read | What it holds |
|---|---|---|
| account_code | **str** | The customer, on its canonical code **CUS-NNNN** |
| customer_name | **str** | The customer's name |
| channel | **str** | The customer's channel, one of the eight; the channel decides whether a policy protects the customer. **config.CHANNEL_COLUMN** names it |
| region | **str** | The customer's region |
| sites_supplied | **str** | Sites the customer supplies |
| key_account_flag | **str** | The customer is a key account |
| delivery_postcode | **str** | The postcode deliveries go to, the proof a caller gives of the account. **config.DELIVERY_POSTCODE_COLUMN** names it |
| account_status | **str** | **Active**, **config.ACCOUNT_STATUS_ACTIVE**, or **Closed**. **config.ACCOUNT_STATUS_COLUMN** names it |

**shipment_history.csv —**
Every week's shipment of each live product to each customer over the last **26** weeks, named by **config.SHIPMENT_HISTORY_FILE**. It arrives clean and is read as text. It holds a row only for a week in which something was shipped: a week with no row is a week of nought.

| Column | Type as read | What it holds |
|---|---|---|
| shipment_reference | **str** | The shipment, in the form **SHP-NNNNNNN**. **config.SHIPMENT_REFERENCE_COLUMN** names it |
| account_code | **str** | The customer, on its canonical code |
| product_code | **str** | The product |
| weeks_ago | **str** | How many weeks ago it was shipped, **1** for last week. **config.WEEKS_AGO_COLUMN** names it |
| cases_shipped | **str** | Cases shipped that week. **config.CASES_SHIPPED_COLUMN** names it |

**config.SHIPMENT_WHOLE_NUMBER_PATTERNS** reads its two whole numbers.

**open_customer_emails.csv —**
The open emails of the customers with open orders, named by **config.OPEN_CUSTOMER_EMAILS_FILE**, read as text. Its columns are the email extract's first eleven, in the same order, without **request_type**, and they carry the email extract's defects. Each customer's emails are numbered in the order they arrived this week, and an email with no **product_code** bears on every order its customer has.

**shortage_products.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| product_code | HS-0500 | DW-0450 | LN-1500 |
| product_name | Wrenfield Hand Sanitiser 500 ml | Wrenfield Dishwash Liquid 450 ml | Wrenfield Non-Bio Laundry Liquid 1.5 litre |
| product_category | Hand hygiene | Dishwash | Laundry |
| pack_description | 12x500ml | 12x450ml | 4x1500ml |
| units_per_case | 12 | 12 | 4 |
| cases_per_pallet | 50 | 60 | 80 |
| cases_available_this_week | 1200 | 1000 | 330 |
| cases_available_next_week | 1000 | 900 | 450 |
| shortage_reason | Filling line breakdown | Bottle supplier missed a delivery | Fragrance could not be sourced |
| weeks_on_allocation | 1 | 1 | 3 |
| seasonal_index | 96 | 103 | 94 |

**open_orders.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 | Record 4 |
|---|---|---|---|---|
| order_line_id | OL-8001338 | OL-8001057 | OL-8000104 | OL-8000716 |
| order_reference | PO-771201 | PO-771202 | PO-771205 | PO-771209 |
| order_route | EDI | Email | Portal | Email |
| account_code | C0021 | Acct 62 | CUS-0105 | Acct 46 |
| customer_name | Greenacre Foodstores | Kestrel Wholesale | Northgate Health Supplies | Saverly Discount |
| received_sequence | 485 | 735 | 917 | 1280 |
| product_code | HS-0500 | HS-0500 | HS-0500 | HS-0500 |
| product_category | Hand hygiene | Hand hygiene | Hand hygiene | Hand hygiene |
| pack_description | 12x500ml | 12x500ml | 12x500ml | 12x500ml |
| cases_per_pallet | 50 | 50 | 50 | 50 |
| ordered_quantity | 12000 units | 10 plt | 300 cs | 20 cs |
| ordered_vs_usual_pct | 250.0 | 416.7 | 120.0 | 66.7 |
| orders_this_week | 1 | 1 | 1 | 1 |
| same_order_other_route_flag | No | No | No | Yes |
| lead_time_days | 9 | 13 | 4 | 5 |
| channel | Supermarket | Wholesale | Healthcare | Discounter |
| region | North East | South East | West Midlands | East of England |
| sites_supplied | 330 | 10 | 95 | 45 |
| customer_stock_cover_weeks | 3.5 | 2.3 | 3.1 | 1.0 |
| account_age_weeks | 689 | 1459 | 143 | 1139 |
| past_shortage_orders | 64 | 88 | 79 | 49 |
| past_orders_cut | 40 | 23 | 6 | 19 |
| key_account_flag | No | Yes | Yes | No |
| shortage_publicised_flag | Yes | Yes | Yes | Yes |
| weeks_on_allocation | 1 | 1 | 1 | 1 |
| promotion_flag | No | No | No | No |
| entered_by | OD-106 | OD-102 | OD-110 | OD-106 |
| weekday_entered | Monday | Thursday | Tuesday | Thursday |
| form_version | OF-4.2 | OF-4.2 | OF-4.1 | OF-4.2 |
| last_week_cases | 450 | 120 | 250 | 60 |
| avg_4_week_cases | 400.00 | 120.00 | 250.00 | 30.00 |
| avg_26_week_cases | 330.8 | 106.2 | 215.4 | 30.0 |
| promotion_next_week_flag | No | No | No | No |
| distribution_change_flag | No | No | No | No |
| seasonal_index | 96 | 96 | 96 | 96 |
| weeks_since_launch | 700 | 700 | 700 | 700 |

**customer_master.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 |
|---|---|---|---|
| account_code | CUS-0021 | CUS-0105 | CUS-0046 |
| customer_name | Greenacre Foodstores | Northgate Health Supplies | Saverly Discount |
| channel | Supermarket | Healthcare | Discounter |
| region | North East | West Midlands | East of England |
| sites_supplied | 330 | 95 | 45 |
| key_account_flag | No | Yes | No |
| delivery_postcode | SR20 8WJ | ST12 8ED | CO5 3YZ |
| account_status | Active | Active | Active |

**shipment_history.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 | Record 4 |
|---|---|---|---|---|
| shipment_reference | SHP-0034007 | SHP-0031333 | SHP-0028646 | SHP-0025952 |
| account_code | CUS-0046 | CUS-0046 | CUS-0046 | CUS-0046 |
| product_code | HS-0500 | HS-0500 | HS-0500 | HS-0500 |
| weeks_ago | 1 | 3 | 5 | 7 |
| cases_shipped | 60 | 60 | 60 | 60 |

**open_customer_emails.csv: SAMPLE RECORDS**

| Column | Record 1 | Record 2 | Record 3 | Record 4 |
|---|---|---|---|---|
| email_id | EM-900007 | EM-900015 | EM-900034 | EM-900174 |
| account_code | CUS-0118 | CUS-0021 | CUS-0077 | CUS-0131 |
| customer_name | Oakhaven Care Homes | Greenacre Foodstores | Elm Pharmacy Group | Brightline Cleaning |
| channel | Healthcare | Supermarket | Pharmacy | Facilities |
| product_code | HS-0500 | HS-0500 | HS-0500 | HS-0500 |
| sender_role | Clinical lead | Buyer | Buyer | Operations manager |
| received_at | Monday 08:12 | Monday 09:40 | Monday 11:05 | Tuesday 14:22 |
| email_sequence | 1 | 1 | 1 | 1 |
| email_subject | Outbreak at two homes | Amend PO-771201 | Earlier delivery please | Our allocation |
| email_text | Good morning, We have an outbreak at two of our homes and our us... | Hello, Our stores are selling out, so please add another 200 cas... | Hi team, Could you bring our delivery of the 500 ml hand sanitis... | Hello, How are allocations being decided at the moment? We would... |
| email_length_chars | 277 | 196 | 213 | 208 |

**THE DEFECTS IN THE RAW EXTRACTS**

Every defect below is present in the raw extracts, in the open orders or in the open emails. The last column names the constants that repair it; a dash means none is needed.

| Defect | How it looks | Where the repair values live |
|---|---|---|
| Padding and case | **"SUPERMARKET"**, **"YORKSHIRE"**, **"CLINICAL LEAD"**, **"Store Manager"** | **config.ORDER_VALUE_MAPPINGS**, **config.DEMAND_VALUE_MAPPINGS**, **config.EMAIL_VALUE_MAPPINGS**, **config.OPEN_ORDER_VALUE_MAPPINGS**, **config.OPEN_EMAIL_VALUE_MAPPINGS** |
| Spelling variants | A channel as **"Supermkt"**, **"Grocery multiple"**, **"Chemist"** or **"Contract cleaning"**; a region as **"NE"**, **"N. East"** or **"Ldn"**; a sender as **"Lead nurse"** or **"Ops manager"**; a request as **"Clinical urgency"** or **"Qty increase"** | **config.PRODUCT_CATEGORY_MAPPING**, **config.CHANNEL_MAPPING**, **config.REGION_MAPPING**, **config.SENDER_ROLE_MAPPING**, **config.REQUEST_TYPE_MAPPING** |
| Weekdays written several ways | **"Thurs"**, **"MONDAY"**, **"Fr."**, **"WED"** | **config.WEEKDAY_MAPPING** |
| Yes and no written many ways | **"Y"**, **"TRUE"**, **"1"**, **"N"**, **"false"**, **"0"** | **config.BOOLEAN_MAPPING**, **config.BOOLEAN_YES**, **config.BOOLEAN_NO** |
| Missing written many ways | **"-"**, **"?"**, **"N/A"**, **"NULL"**, **"Not recorded"**, blank | **config.NULL_TOKENS** |
| Duplicated rows | The same record twice, identical in every column | — |
| Account codes in three notations | **"C0021"** by EDI, **"CUS-0021"** by the Portal, **"Acct 21"** by Email | **config.ACCOUNT_CODE_NOTATIONS**, **config.ACCOUNT_NUMBER_PART**, **config.CANONICAL_ACCOUNT_CODE_FORMAT**, **config.CANONICAL_ACCOUNT_CODE_PATTERN** |
| Quantities in three notations | **"12000 units"** by EDI, **"300 cs"** by the Portal, **"20 cs"** or **"10 plt"** by Email | **config.QUANTITY_NOTATIONS**, **config.QUANTITY_AMOUNT_PART**, **config.QUANTITY_UNIT_PART**, **config.CONSUMER_UNITS_WORD**, **config.CASES_WORD**, **config.PALLETS_WORD** |
| Pack descriptions written four ways | **"12x500ml"**, **"12 x 500 ml"**, **"500ml x12"**, **"12/500ML"** | **config.PACK_DESCRIPTION_NOTATIONS**, **config.UNITS_PER_CASE_PART**, **config.UNIT_MILLILITRES_PART**, **config.CANONICAL_PACK_DESCRIPTION_FORMAT**, **config.CANONICAL_PACK_DESCRIPTION_PATTERN** |
| The same order sent twice | One purchase order for one product arriving by two routes, the second later in the order log and written in its own route's notation | **config.ORDER_KEY_COLUMNS**, **config.RECEIVED_SEQUENCE_COLUMN** |
| Customer names padded or with a legal suffix | **"Stokesley Cleaning Limited"**, **"Ollerton Food Halls Ltd"**, a name with a run of spaces in it | **config.CUSTOMER_NAME_LEGAL_SUFFIX_PATTERN**, **config.WHITESPACE_RUN_PATTERN** |
| Numbers written with a unit or a sign | **"169.6%"**, **"159.8 %"**, **"3 orders"**, **"8 days"**, **"27 sites"**, **"768 wks"**, **"1.4 weeks"**, **"347 cs"**, **"290 weeks"** | **config.ORDER_WHOLE_NUMBER_PATTERNS**, **config.ORDER_DECIMAL_NUMBER_PATTERNS**, **config.DEMAND_WHOLE_NUMBER_PATTERNS**, **config.DEMAND_DECIMAL_NUMBER_PATTERNS**, **config.EMAIL_WHOLE_NUMBER_PATTERNS**, built from **config.WHOLE_NUMBER_PATTERN**, **config.COUNT_PATTERN**, **config.POSITIVE_WHOLE_NUMBER_PATTERN**, **config.DECIMAL_NUMBER_PATTERN**, **config.NON_NEGATIVE_DECIMAL_PATTERN**, **config.DECIMAL_PERCENTAGE_PATTERN**, **config.ORDERS_COUNT_PATTERN**, **config.SITES_COUNT_PATTERN**, **config.DAYS_COUNT_PATTERN**, **config.WHOLE_WEEKS_PATTERN**, **config.DECIMAL_WEEKS_PATTERN** and **config.CASES_COUNT_PATTERN** |
| Impossible readings | More orders cut than shortage orders; a stock cover, a lead time or an order percentage below nought; a week's cases or a seasonal index below nought; the demand answer below nought | **config.ORDER_MINIMUM_VALUES**, **config.ORDER_COUNT_TOTALS**, **config.DEMAND_MINIMUM_VALUES** |
| Missing answer column | A row with no value in the column the model must learn | — |
| Email times in several notations | **"FRIDAY 10:17"**, **"Mo. 11:18"**, **"Thurs 10:14"** | **config.RECEIVED_AT_PATTERN**, **config.RECEIVED_WEEKDAY_PART**, **config.RECEIVED_TIME_PART**, **config.WEEKDAY_MAPPING** |
| Markup in free text | **<p>**, **<b>**, **** and **<br/>** around and inside an email | **config.MARKUP_PATTERN** |
| Escaped characters in free text | ** **, **'** and no-break spaces where plain characters belong | **config.ENCODING_ARTIFACT_PATTERN**, **config.WHITESPACE_RUN_PATTERN** |
| Emails written in capitals | A whole email shouted, which can be restored only in lower case | — |
| Emails too short to learn from | **"Any update?"** | **config.MINIMUM_EMAIL_WORDS** |
| A stale derived column | **email_length_chars** measured on the raw text | **config.EMAIL_LENGTH_COLUMN** |

Some gaps are facts, not defects: an email that names no product, and an order percentage for a customer with nothing shipped in four weeks. They stay empty in the cleaned tables.

**THE SIXTEEN DOCUMENTS**

Sixteen PDFs under **data/docs/**, the only source of any rule, check or advice the system states.

| Document | What it settles |
|---|---|
| **Allocation-Policy-Hand-Hygiene.pdf** | The hand hygiene policy: the weeks of shipments behind usual demand, the cap on an order, the protected customers or that none is, the reserve or that none is held, the advice to customers and how the sales team is briefed |
| **Allocation-Policy-Surface-Disinfection.pdf** | The same for surface disinfection |
| **Allocation-Policy-Laundry.pdf** | The same for laundry |
| **Allocation-Policy-Dishwash.pdf** | The same for dishwash |
| **Allocation-Policy-Toilet-Care.pdf** | The same for toilet care |
| **Supply-Allocation-Policy.pdf** | The roles at the desk, the separation of reviewing and deciding, the three decisions and when each is open, what the allocation notice and the sales briefing must show, the competition-law rule, and what the customer service line may say |
| **Allocation-Rules.pdf** | Account codes and the three notations, units, cases and pallets, an order sent twice, usual demand and the silent week, the cap, the division, whole cases and the largest remainders, the reserve and stock carried into next week |
| **Past-Shortages-Casebook.pdf** | Eight past shortages, one per request type, and how each was decided. It tells its cases in sentences of its own, and it is the one document the chat may not read |
| **Guide-Urgent-Clinical-Need.pdf** | How the desk handles an urgent clinical need: how it shows up, what the planner checks, the evidence to ask for, when it changes nothing, what to tell the customer and when to escalate |
| **Guide-Order-Cancellation.pdf** | The same, for an order cancellation |
| **Guide-Quantity-Increase.pdf** | The same, for a quantity increase |
| **Guide-Expedite-Request.pdf** | The same, for an expedite request |
| **Guide-Substitution-Accepted.pdf** | The same, for a substitution accepted |
| **Guide-Delivery-Complaint.pdf** | The same, for a delivery complaint |
| **Guide-Forecast-Update.pdf** | The same, for a forecast update |
| **Guide-Allocation-Query.pdf** | The same, for an allocation query |

Every document opens with **WRENFIELD HYGIENE**, its title and one header line matched by **config.DOCUMENT_HEADER_PATTERN**, and is divided into sections numbered from **1**, matched by **config.SECTION_HEADING_PATTERN**.
The five category policies share one template and most of its sentences; they differ in their four rules, the products they cover, their advice to customers and their briefing for the sales team. **config.CATEGORY_POLICY_DOCUMENTS** names each category's policy, **config.REQUEST_GUIDE_DOCUMENTS** the guide for each request type, and **config.CHAT_EXCLUDED_DOCUMENTS** the casebook.
Each rule is one sentence of a fixed form, read by **config.HISTORY_SENTENCE_PATTERN**, **config.CAP_SENTENCE_PATTERN**, **config.PROTECTED_SENTENCE_PATTERN**, **config.NO_PROTECTION_SENTENCE_PATTERN**, **config.RESERVE_SENTENCE_PATTERN** and **config.NO_RESERVE_SENTENCE_PATTERN**. The configuration holds no policy's weeks, cap, protected channels or reserve; each is read from its own policy at run time.

**DOMAIN KNOWLEDGE CONTEXT**
Every value named here is in **config/config.py**, except the rules of the five allocation policies, which exist only in the policies.
**WHAT ALLOCATION IS**
A product is **on allocation** when the cases there are this week fall short of the orders for it. Allocation decides how many of those cases each customer gets: every customer's **usual demand** is measured from its shipments, every order is held to what the policy lets the customer have, the policy's **protected customers** are served first, and the cases left are shared out in proportion to usual demand. Nothing is confirmed to a customer until a Supply Planning Manager decides.
**ACCOUNT CODES AND QUANTITIES IN THREE NOTATIONS**
Every customer has one **canonical account code**: **CUS**, a hyphen and four digits, matched by **config.CANONICAL_ACCOUNT_CODE_PATTERN** and written by **config.CANONICAL_ACCOUNT_CODE_FORMAT**. Each order route writes the account and the quantity its own way:

| Route | Account | Quantity |
|---|---|---|
| **EDI** | **C0021** | Consumer units: **12000 units** |
| **Portal** | **CUS-0021** | Cases: **1000 cs** |
| **Email** | **Acct 21**, with no leading noughts | Cases or pallets: **20 cs**, **10 plt** |

**config.ACCOUNT_CODE_NOTATIONS** and **config.QUANTITY_NOTATIONS** hold each route's notation. Every quantity is read into **whole cases** with the sizes on its own order line: units divided by the units in a case, which the line's **pack_description** gives, and pallets multiplied by its **cases_per_pallet**. **12000 units** of a twelve-bottle case are **1,000** cases, and **10 plt** of a fifty-case pallet are

**500**. An account or a quantity in a notation its route does not write is refused, never guessed: **1000 cs** by EDI, **12000 units** by Email, **CUS-0021** by EDI, and units that are not a whole number of cases.
A pack description is read in any of the four notations of **config.PACK_DESCRIPTION_NOTATIONS** into the units in a case and the millilitres in a unit, and written back as **config.CANONICAL_PACK_DESCRIPTION_FORMAT**: **12 x 500 ml**, **500ml x12** and **12/500ML** are all **12x500ml**.
An order is identified by its **purchase order and product**, **config.ORDER_KEY_COLUMNS**. The same purchase order for the same product arriving by two routes is **one order**, kept as it first arrived in the order log, **config.RECEIVED_SEQUENCE_COLUMN**; the second arrival changes nothing, even when it states another quantity. A purchase order is one customer's: the same purchase order and product arriving from two accounts is refused.

**FIVE CATEGORIES, FIVE POLICIES**

| Product category | Allocation policy |
|---|---|
| Hand hygiene | **Allocation-Policy-Hand-Hygiene.pdf** |
| Surface disinfection | **Allocation-Policy-Surface-Disinfection.pdf** |
| Laundry | **Allocation-Policy-Laundry.pdf** |
| Dishwash | **Allocation-Policy-Dishwash.pdf** |
| Toilet care | **Allocation-Policy-Toilet-Care.pdf** |

**config.CATEGORY_POLICY_DOCUMENTS** maps each category to its policy. Each policy states four rules of its own: the number of weeks of shipments behind usual demand, the **cap** on an order as a multiple of usual demand, the channels whose customers are protected or that no customer is, and the percentage that may be held back as a **reserve** or that none is held.

**THE RULES ARE NOT IN THE SOLUTION**
A policy's weeks of history are a whole number, its cap a decimal, its protected channels Wrenfield's channel names in the order its sentence writes them, and its reserve a whole percentage. They are read at run time from the product category's own policy; no policy's weeks, cap, protected channels or reserve are written into the solution or the configuration, and no category's rule is used for another or assumed from its name. The five policies share most of their sentences word for word, so a search across all the documents reads another category's rules: a policy is read whole, from its own document.
Each rule must be stated once: the same sentence met in two passages is one statement. A policy is refused, naming the document, when it states two different numbers of weeks or none, two different caps or none, protected channels and that no customer is protected, neither of them, two different lists of protected channels, a channel that is not one of the eight, a reserve and that none is held, neither of them, or two different reserves; a category with no configured policy is refused with **config.NO_POLICY_CONFIGURED_MESSAGE**.

**USUAL DEMAND AND THE SILENT WEEK**
A customer's **usual demand** of a product is the cases shipped to it over the policy's last N weeks, a shipment exactly N weeks ago included, divided by N. The shipment history holds a row only for a week in which something was shipped, so **a week with no shipment counts as nought**: a customer that takes **60** cases every other week has a usual demand of **30**, not **60**. A customer with no shipment in the policy's weeks has a usual demand of nought.
Usual demand is a fraction and is kept **exact**: **55** cases over seven weeks is exactly **55/7**. It is shown, rounded half up, to **config.USUAL_DEMAND_DECIMALS** places, **7.8571**.

**WHAT A CUSTOMER MAY HAVE**
A customer's cases ordered are every order it sent for the product, each counted once. Its **eligible** cases, what it may have, are the lower of its cases ordered and the policy's cap times its usual demand, rounded down to whole cases. The product is worked out exactly: **1.4** times **55** cases over seven weeks is exactly **11** cases, where floating point gives **10.999999999999998** and rounds it down to **10**. A customer with no usual demand may have nothing while the product is on allocation.
**THE DIVISION**
The cases available are divided under each release the policy allows — **RELEASE** always, and **RELEASE WITH RESERVE** under a policy that holds a reserve — in this order:

- **Protected customers first.** A customer is protected when its channel in the customer master is one the policy protects. If the protected customers' eligible cases add up to no more than the cases available, each gets its eligible cases in full. If not, the cases are divided among the protected customers alone, by the division below, and no other customer gets anything.
- **The reserve, on RELEASE WITH RESERVE only.** The policy's percentage of the cases left after the protected customers, rounded down to whole cases, is held at the distribution centre for the protected customers' urgent orders.
- **Everyone else.** If their eligible cases fit in the cases left, each gets its eligible cases. If not, the cases left are divided:

```
share of a customer  =  cases to divide  x  its usual demand
                        / usual demand of every customer still sharing
```

A customer whose share is **at least** its eligible cases gets its eligible cases and leaves the division; the cases it took come off, and the shares of those still sharing are worked out again, until no share reaches its customer's eligible cases. Shares are kept exact throughout, and a customer with no usual demand gets nothing.

- **Whole cases.** Each share is rounded down. The cases left over, never more than one for each customer still sharing, go one each to the customers with the **largest remainders**: the largest fraction first, a tie to the **larger usual demand**, then to the **lower account code**.

A customer is never given more than its eligible cases: cases beyond what every customer may have **stay in stock** for next week. Every case is placed, held in reserve or left in stock, and none twice.

**THE WORKED EXAMPLE**

**HS-0500**, **Wrenfield Hand Sanitiser 500 ml**: twelve bottles to a case, fifty cases to a pallet. A filling line broke down; this week **1,200** cases are available and next week **1,000**. Category **Hand hygiene**. The hand hygiene policy states that usual demand is the average weekly cases shipped over the last **8** weeks, that an order counts for no more than **1.0** times the customer's usual demand, that customers in the **Healthcare** channel are protected and are served first, and that up to **10** percent of the cases left after protected customers may be held back as a reserve.

Nine customers ordered, on ten order lines:

| Customer | Route | Account as written | Quantity as written | Cases |
|---|---|---|---|---|
| Greenacre Foodstores | EDI | **C0021** | **12000 units** | **1,000** |
| Kestrel Wholesale | Email | **Acct 62** | **10 plt** | **500** |
| BasketDrop Online | EDI | **C0089** | **480 units** | **40** |
| Dunmore Market | EDI | **C0034** | **7200 units** | **600** |
| Northgate Health Supplies | Portal | **CUS-0105** | **300 cs** | **300** |
| Elm Pharmacy Group | EDI | **C0077** | **1800 units** | **150** |
| Brightline Cleaning | Email | **Acct 131** | **200 cs** | **200** |
| Oakhaven Care Homes | Portal | **CUS-0118** | **150 cs** | **150** |
| Saverly Discount | EDI | **C0046** | **240 units** | **20** |
| Saverly Discount | Email | **Acct 46** | **20 cs** | the same purchase order, **PO-771209**, arriving again: kept once |

**2,960** cases are ordered against **1,200**. Saverly Discount took **60** cases every other week, **240** in eight weeks, so its usual demand is **30**, not **60**; Brightline Cleaning took **120** cases in six of the eight weeks, so **90**, not **120**.

| Customer | Account | Channel | Ordered | Usual demand | Eligible | RELEASE | RELEASE WITH RESERVE |
|---|---|---|---|---|---|---|---|
| Greenacre Foodstores | CUS-0021 | Supermarket | **1,000** | **400** | **400** | **293** | **262** |
| Dunmore Market | CUS-0034 | Supermarket | **600** | **300** | **300** | **220** | **196** |
| Saverly Discount | CUS-0046 | Discounter | **20** | **30** | **20** | **20** | **20** |
| Kestrel Wholesale | CUS-0062 | Wholesale | **500** | **120** | **120** | **88** | **78** |
| Elm Pharmacy Group | CUS-0077 | Pharmacy | **150** | **100** | **100** | **73** | **65** |
| BasketDrop Online | CUS-0089 | Online | **40** | **80** | **40** | **40** | **40** |
| Northgate Health Supplies | CUS-0105 | Healthcare, protected | **300** | **250** | **250** | **250** | **250** |
| Oakhaven Care Homes | CUS-0118 | Healthcare, protected | **150** | **150** | **150** | **150** | **150** |
| Brightline Cleaning | CUS-0131 | Facilities | **200** | **90** | **90** | **66** | **59** |
| **Total** | | | **2,960** | | **1,470** | **1,200** | **1,120**, and **80** held |

**RELEASE.** The two protected customers may have **400** cases and get them in full; **800** are left for the other seven, whose usual demand adds up to **1,120**. In proportion to it, BasketDrop's share is **57.1429** and Saverly's **21.4286**, at least the **40** and **20** they may have, so each takes that and leaves the division. The **740** left go to the other five, whose usual demand adds up to **1,010**: Greenacre **293.0693**, Dunmore **219.8020**, Kestrel **87.9208**, Elm **73.2673** and Brightline **65.9406**. Rounded down they come to **737**; the three cases left go one each to the largest remainders, Brightline's **.9406**, Kestrel's **.9208** and Dunmore's **.8020**.

**RELEASE WITH RESERVE.** Ten percent of the **800** cases, **80**, is held, and **720** are divided. BasketDrop's share, **51.4286**, still reaches its **40**; Saverly's, **19.2857**, is now below its **20**, so Saverly stays in. The **680** left go to six customers: Greenacre **261.5385**, Dunmore **196.1538**, Kestrel **78.4615**, Elm **65.3846**, Saverly **19.6154** and Brightline **58.8462**. Rounded down they come to **677**, and the three left go to Brightline, Saverly and Greenacre.

The cases cover **1,200** of the **1,470** the customers may have, **81.63** percent, and both protected customers are served in full, so the decisions open are

**RELEASE**, **RELEASE WITH RESERVE**, because the hand hygiene policy holds a reserve, and **ESCALATE**. On **RELEASE WITH RESERVE**, Greenacre's line of the allocation notice reads **Greenacre Foodstores: 262 of the 1,000 cases ordered of Wrenfield Hand Sanitiser 500 ml**, and the sales briefing gives **1,120** cases released to nine customers and **80** held, and no channel's figure, because no channel here has three customers.

### THREE DECISIONS, AND THE DECISIONS OPEN

The Supply Planning Manager chooses one of the three decisions in **config.DECISIONS** that the allocation leaves open; **ESCALATE** is always open.

| Decision | Open when | What happens | Status afterwards | Further model calls |
|---|---|---|---|---|
| **config.DECISION_RELEASE** | Every protected customer gets its eligible cases under **RELEASE**, and the cover is at least **config.MINIMUM_RELEASE_COVER_PERCENT** percent | Each customer's **RELEASE** cases are confirmed; cases no customer may have stay in stock | **config.REVIEW_STATUS_RELEASED** | Two: the allocation notice, then the sales briefing |
| **config.DECISION_RELEASE_WITH_RESERVE** | **RELEASE** is open and the policy holds a reserve | Each customer's **RELEASE WITH RESERVE** cases are confirmed, and the reserve is held | **config.REVIEW_STATUS_RELEASED_WITH_RESERVE** | Two: the allocation notice, then the sales briefing |
| **config.DECISION_ESCALATE** | Always | No case is confirmed; every order stays open for the supply review, the cases stay at the distribution centre and no customer is told anything | **config.REVIEW_STATUS_ESCALATED** | One: the sales briefing |

The **cover** is the cases available as a percentage of the cases every customer together may have, protected or not, over **config.PERCENT_SCALE**. It is decided exactly and shown, rounded half up, to **config.COVER_DECIMALS** places. Where no customer may have any case there is no cover to work out, and only **ESCALATE** is open.
A closed decision carries every reason that closes it, joined by **config.REASON_SEPARATOR**: **config.PROTECTED_CUSTOMER_SHORT_REASON**, **config.NOTHING_ELIGIBLE_REASON**, **config.COVER_BELOW_MINIMUM_REASON** and, for **RELEASE WITH RESERVE**, **config.NO_RESERVE_HELD_REASON**. A decision not open is refused with **config.DECISION_NOT_OPEN_MESSAGE**, naming its reasons. While it waits, a review stands at **config.REVIEW_STATUS_PENDING**; **config.REVIEW_STATUS_BY_DECISION** maps each decision to its status. **config.LLM_CALLS_BEFORE_DECISION** is the number of language model calls before the gate, and **config.LLM_CALLS_BY_DECISION** the total for each decision. A product is reviewed once.

### THE FORECAST SHORTFALL, AND THE ORDER OF THE QUEUE

The demand model forecasts, for every customer with an open order of a product, the cases it will need next week, from the demand inputs on that customer's first order line of the product in the order log. The **forecast shortfall** adds the forecasts up, rounds the sum once to whole cases, and takes away the cases available next week; it stays below nought where next week's supply is the larger.
The queue puts the largest forecast shortfall first, a tie to the lower product code. Positions count from **1**. The forecast shortfall orders the queue and decides nothing; each trainee's own model gives its own figures.

### THE DEMAND FORECAST

The demand model forecasts the cases a customer will take next week from what the planning team knows at the end of a week: the sites supplied, last week's cases, the four-week and **26**-week averages, the seasonal index, the weeks since launch, the channel, region, category and pack, a promotion next week, a gain in distribution next week and the key account flag, the columns of **config.DEMAND_NUMERIC_FEATURES** and **config.DEMAND_CATEGORICAL_FEATURES**. No forecast is below nought.

### THE INFLATED-ORDER SCORE AND ITS BAND

The inflated-order model gives the probability that the customer will cut an order by at least a third, or cancel it, once supply recovers. Every open order is scored; the score is rounded to **config.INFLATED_ORDER_SCORE_DECIMALS** places and banded on the rounded figure: **config.INFLATED_ORDER_BAND_HIGH** at or above **config.INFLATED_ORDER_BAND_HIGH_THRESHOLD**, **config.INFLATED_ORDER_BAND_MEDIUM** at or above **config.INFLATED_ORDER_BAND_MEDIUM_THRESHOLD**, otherwise **config.INFLATED_ORDER_BAND_LOW**. The score is read as a probability: over the held-out orders its mean lies within **0.05** of the share of orders cut. It is shown and explained, and it moves no case: the cap has already held every order to what its customer may have.

### READING A CUSTOMER EMAIL

The request model names which of the eight request types, the keys of **config.REQUEST_GUIDE_DOCUMENTS**, an email makes. Every open email of a product's customers that names the product, or names no product, is read. Each customer's **most urgent request** is the one of its emails' requests that comes first in **config.REQUEST_URGENCY_ORDER**; a request the configuration does not rank is refused.

### THE THREE ROLES

| Role | Reaches | Does not reach |
|---|---|---|
| **config.ROLE_MANAGER** | Everything, and is the only role that may record a decision, never on a review they opened themselves | — |
| **config.ROLE_PLANNER** | **config.AREA_INTAKE**, **config.AREA_REVIEW**, **config.AREA_AUDIT** and **config.AREA_CHAT** | **config.AREA_DECISION** |
| **config.ROLE_ADVISER** | **config.AREA_CHAT** only | Everything else |

The three roles are **Supply Planning Manager**, **Supply Planner** and **Customer Service Adviser**. The areas are named by **config.ROLE_AREA_ACCESS** and the pages each role is offered by **config.ROLE_PAGE_ACCESS**; the area a request belongs to is

decided by the route it matched, its method and its path template, through **config.AREA_BY_ROUTE**, never by the text of the address.
The person who opened a review may not decide it; another Supply Planning Manager does, and the refusal is worded **config.OWN_REVIEW_REFUSED_MESSAGE**.

**THE IDENTITY LINE**
A Customer Service Adviser takes customers' calls, and hears an allocation only for an account the caller has proved: the question gives the account's code, in any of the three routes' notations, and the delivery postcode on that account's record in the customer master. A postcode matches whatever its spaces and its case.

- The account proved: **config.CHAT_ACCOUNT_HEADING_TEMPLATE**, then one line for each reviewed product the account ordered, giving all its cases ordered of that product, newest review first, narrowed to the product when the question names one by its code — **config.CHAT_ORDER_UNDER_REVIEW_TEMPLATE** while the review waits, **config.CHAT_ORDER_RELEASED_TEMPLATE** with the cases confirmed once released, **config.CHAT_ORDER_ESCALATED_TEMPLATE** once escalated — and last **config.CHAT_OTHER_ORDERS_TEXT**.
- No account named: **config.CHAT_ACCOUNT_NOT_NAMED_MESSAGE**.
- Any other question about an account — no postcode, two different postcodes, a postcode that is not the account's, another customer's account, an account that does not exist: **config.CHAT_NO_ALLOCATION_TEMPLATE**, identical in every case, naming the account by its canonical code where the code can be read in one of the three notations, and as asked otherwise.

A Supply Planner or a Supply Planning Manager hears every review of the product or the account the question names, newest first: **config.CHAT_REVIEWS_HEADING_TEMPLATE**, then for each review **config.CHAT_REVIEW_TEMPLATE**, followed by **config.CHAT_AWAITING_DECISION_TEMPLATE** while it waits or **config.CHAT_DECIDED_TEMPLATE** once decided, and a **config.CHAT_REVIEW_LINE_TEMPLATE** for each customer, or for the named account alone, a review without it being left out: the cases under every release still open, or **config.CHAT_NO_RELEASE_OPEN_TEXT**, while it waits, and **config.CHAT_CONFIRMED_CASES_TEMPLATE** once decided, nought on an escalation. A product or account no review covers is answered with **config.CHAT_NO_REVIEW_TEMPLATE**, and a question that names neither with **config.CHAT_SUBJECT_NOT_NAMED_MESSAGE**.
A Customer Service Adviser never hears another customer's name, code or cases, an inflated-order score, an email, a planner's or a manager's name, or anything else from the desk's records; every document the chat may read is answered for anyone.

**A SHORT GLOSSARY**

| Term | Meaning |
|---|---|
| On allocation | A product whose cases this week fall short of its orders, so every order waits for a decision |
| Canonical account code | A customer's one code, **CUS**, a hyphen and four digits |
| Order route | How an order arrives: EDI, the Portal or Email, each writing the account and the quantity its own way |
| Order sent twice | One purchase order for one product arriving by two routes, kept once as it first arrived |
| Usual demand | The cases shipped to a customer over the policy's weeks, divided by that number of weeks |
| Silent week | A week with no shipment, which counts as nought |
| Cap | The multiple of usual demand above which an order no longer counts |
| Eligible cases | What a customer may have: the lower of its order and the cap times its usual demand, in whole cases |
| Protected customer | A customer in a channel the policy serves first |
| Reserve | Cases held at the distribution centre for the protected customers' urgent orders, on **RELEASE WITH RESERVE** only |
| Cover | The cases available as a percentage of what every customer together may have |
| Division | Sharing the cases in proportion to usual demand, every share at or above its customer's limit passed on |
| Largest remainder | Handing out the cases left after rounding every share down, the largest fraction first |
| Release | Confirming each customer's cases as divided, with or without the reserve held |
| Escalate | Confirming nothing and sending the shortage to the supply review |
| Allocation notice | The notice to every customer that ordered, each customer's own line added by the code |
| Sales briefing | What the sales team is told, with totals only and a channel's figure only for three customers or more |
| Inflated order | An order the customer will cut or cancel once supply recovers |
| Forecast shortfall | The customers' forecast demand next week less next week's supply |
| Delivery postcode | The postcode on an account's record, the caller's proof of the account |
| Allocation review | The write-up and recommendation the workflow produces for the Supply Planning Manager |

**TASKS**

For each module the responsibility is stated, and then every function within it is specified by its signature, its parameters, its purpose, the requirements its result must satisfy, and what it returns.

Names matter. Every function name, parameter name, module path and dictionary key given below is read by other parts of the system and by the marking suite, and must be reproduced exactly as written, including case.
Every path, spelling correction, validation limit, threshold, vocabulary, pattern and business rule already exists in **config/config.py**, which is supplied complete.

## ========== DATA PREPARATION ==========

### MODULE: data_pipeline/dataset_loader.py

The only data preparation module that reads or writes CSV files. Raw files are read without inferring any column's type; cleaned files are read with each column's type inferred.

#### FUNCTION SIGNATURE

```
load_raw_dataset(file_path)
```

#### PARAMETERS

**file_path** — **str**. The path of one raw extract under **data/**, or of a live-queue file under **data/input/**, passed by a cleaning function, by **run_data_pipeline.py** or by the workflow's nodes module from config, for example **config.RAW_ORDER_HISTORY_TRAIN_FILE** or **config.OPEN_ORDERS_FILE**.

#### PURPOSE

Read one raw file as text, guessing nothing.

#### REQUIREMENTS

- Every column holds text, and no value carries leading or trailing whitespace.
- A value that matches one of **config.NULL_TOKENS**, ignoring case, is a missing value (NaN); a value that merely contains such a token is kept.
- The row count and the column count match the file on disk.

#### RETURNS

**pandas.DataFrame**. Every row of the named file, every column held as text and every missing value as NaN.

#### FUNCTION SIGNATURE

```
load_processed_dataset(file_path)
```

#### PARAMETERS

**file_path** — **str**. The path of a cleaned table under **processed/**, passed by **run_model_training.py** from config, for example **config.CLEAN_ORDER_HISTORY_TRAIN_FILE**.

#### PURPOSE

Read a table the pipeline has already cleaned.

#### REQUIREMENTS

- Column types follow the file's contents, and only an empty cell is read as missing.
- Every row and every column of the file is returned, unfiltered and unsorted.

#### RETURNS

**pandas.DataFrame**. The file as it stands on disk.

#### FUNCTION SIGNATURE

```
save_processed_dataset(dataframe, file_path)
```

#### PARAMETERS

**dataframe** — **pandas.DataFrame**. The finished table, passed by the cleaning function that produced it.
**file_path** — **str**. The destination under **processed/**, passed on by that cleaning function from its **output_file_path**, for example **config.CLEAN_WEEKLY_DEMAND_TRAIN_FILE**.

#### PURPOSE

Save a cleaned table.

#### REQUIREMENTS

- The folder the file belongs in exists afterwards, whether or not it existed before.
- The file holds the table's own columns and no added index column.

#### RETURNS

**None**.

### MODULE: data_pipeline/common_cleaning.py

Repairs shared by the cleaning runs, the profiler and the shortage data tools. Every function returns a repaired copy and leaves its input untouched; none reads or writes a file. A value in an unrecognised notation is refused with a **ValueError** naming the column and the values.

#### FUNCTION SIGNATURE

```
remove_duplicate_records(dataframe)
```

#### PARAMETERS

**dataframe** — **pandas.DataFrame**. The table a cleaning run has reached, passed by **clean_order_records**, **clean_demand_dataset** or **clean_email_dataset**.

#### PURPOSE

Drop records the extract wrote twice.

#### REQUIREMENTS

- Rows identical in every column appear once, the first of them kept; rows that differ anywhere are all kept.
- The result is numbered from nought upwards with no gaps.

**RETURNS**
**pandas.DataFrame**. The table without exact duplicates.

**FUNCTION SIGNATURE**
`keep_first_arrival_of_each_order(dataframe)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. Order lines whose numbers are already read, passed by **clean_order_records** or **profile_order_details**.
**PURPOSE**
Keep an order sent by two routes once, as it first arrived.
**REQUIREMENTS**

- There is one row for each purchase order and product, **config.ORDER_KEY_COLUMNS**: the one earliest in **config.RECEIVED_SEQUENCE_COLUMN**, whatever its route and whatever a later arrival says, and wherever it stands in the table.
- The rows are in the order they arrived, numbered from nought upwards with no gaps.
- A line that leaves its purchase order, product or place in the order log empty, and two lines that share a place in the order log, are refused, naming the order lines.

**RETURNS**
**pandas.DataFrame**. One row per order.

**FUNCTION SIGNATURE**
`normalize_category_values(dataframe, value_mappings)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table to fold, passed by a cleaning function or by **fetch_open_emails**.
**value_mappings** — **dict[str, dict]**. Each column against its lower-case mapping, passed from config, for example **config.ORDER_VALUE_MAPPINGS** or **config.OPEN_EMAIL_VALUE_MAPPINGS**.
**PURPOSE**
Fold every spelling and case of a category value onto its one canonical form.
**REQUIREMENTS**

- Every value in a mapped column is its mapping's canonical value, whatever case it arrived in; a missing value stays missing.
- A value the mapping does not recognise is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with every mapped column folded.

**FUNCTION SIGNATURE**
`normalize_customer_names(customer_names)`

**PARAMETERS**
**customer_names** — **pandas.Series**. Customer names as read, passed by **clean_order_records**.
**PURPOSE**
Give each customer one spelling of its name.
**REQUIREMENTS**

- Every run matched by **config.WHITESPACE_RUN_PATTERN** is a single space, and a legal suffix matched by **config.CUSTOMER_NAME_LEGAL_SUFFIX_PATTERN** is removed: **Stokesley Cleaning Limited** is **Stokesley Cleaning**. A gap stays a gap.

**RETURNS**
**pandas.Series**. The names, on the same index.

**FUNCTION SIGNATURE**
`remove_rows_missing_target(dataframe, target_column)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table reached so far, passed by a cleaning function.
**target_column** — **str**. The answer column, passed from config, for example **config.ORDER_TARGET_COLUMN**.
**PURPOSE**
Remove the rows no model can learn from.
**REQUIREMENTS**

- Exactly the rows with no answer are removed, and the result is numbered from nought upwards with no gaps.

**RETURNS**
**pandas.DataFrame**. The rows that carry an answer.

**FUNCTION SIGNATURE**
`read_pack_description(pack_descriptions)`

**PARAMETERS**
**pack_descriptions** — **pandas.Series**. Pack descriptions as written, named after their column, passed by **clean_order_records** or **convert_pack_descriptions**.

**PURPOSE**
Read the units in a case and the millilitres in a unit out of a pack description, in any of its notations.
**REQUIREMENTS**
- The result has the two columns **config.UNITS_PER_CASE_PART** and **config.UNIT_MILLILITRES_PART**, whole numbers read through **config.PACK_DESCRIPTION_NOTATIONS**: **12 x 500 ml**, **500ml x12** and **12/500ML** each give **12** and **500**. A gap gives gaps.
- A description in none of the notations is refused, naming the column and the descriptions.

**RETURNS**
**pandas.DataFrame**. The two parts, on the same index.

**FUNCTION SIGNATURE**
`convert_pack_descriptions(dataframe)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A table with a **pack_description** column, passed by **clean_order_records** or **clean_demand_dataset**.
**PURPOSE**
Write every pack description one way.
**REQUIREMENTS**
- **config.PACK_DESCRIPTION_COLUMN** holds each description's two parts written by **config.CANONICAL_PACK_DESCRIPTION_FORMAT**, **12x500ml**. A gap stays a gap, and a description no notation reads is refused.

**RETURNS**
**pandas.DataFrame**. The table with canonical pack descriptions.

**FUNCTION SIGNATURE**
`convert_account_code(account_codes, order_routes)`

**PARAMETERS**
**account_codes** — **pandas.Series**. Account codes as written, named after their column, passed by **clean_order_records**.
**order_routes** — **pandas.Series**. On the same index, the route that wrote each code, passed by the same caller from the lines' own **order_route**.
**PURPOSE**
Read every account code through the notation of the route that wrote it.
**REQUIREMENTS**
- Each code is the canonical code written by **config.CANONICAL_ACCOUNT_CODE_FORMAT** from the part **config.ACCOUNT_NUMBER_PART** of its own route's notation in **config.ACCOUNT_CODE_NOTATIONS**: **C0021** by **EDI**, **CUS-0021** by the **Portal** and **Acct 21** by **Email** are each **CUS-0021**. A gap stays a gap.
- A code its own route does not write, or a code whose route has no notation, is refused, naming the column and the codes or routes it could not read.

**RETURNS**
**pandas.Series**. The canonical codes, on the same index.

**FUNCTION SIGNATURE**
`convert_quantity_to_cases(quantities, order_routes, units_per_case, cases_per_pallet)`

**PARAMETERS**
**quantities** — **pandas.Series**. Quantities as written, named after their column, passed by **clean_order_records**.
**order_routes** — **pandas.Series**. On the same index, the route that wrote each quantity, passed by the same caller.
**units_per_case** — **pandas.Series**. On the same index, the units in a case of each line's product, which **read_pack_description** reads from its pack description, passed by the same caller.
**cases_per_pallet** — **pandas.Series**. On the same index, the cases on a pallet of each line's product, as whole numbers, passed by the same caller.
**PURPOSE**
Read every quantity, in whatever unit its route writes, as whole cases of the line's own product.
**REQUIREMENTS**
- Each quantity is read as the parts **config.QUANTITY_AMOUNT_PART** and **config.QUANTITY_UNIT_PART** of its own route's notation in **config.QUANTITY_NOTATIONS**: an amount in **config.CASES_WORD** is its cases, one in **config.CONSUMER_UNITS_WORD** is divided by the line's units per case, and one in **config.PALLETS_WORD** is multiplied by its cases per pallet. **12000 units** at twelve to a case, **1000 cs** and **20 plt** at fifty to a pallet are each **1000**. A gap stays a gap.
- A quantity its own route does not write, a quantity whose route has no notation, units or pallets with no size to convert them, and units that are not a whole number of cases are refused, naming the column and the quantities, or the routes it has no notation for.

**RETURNS**
**pandas.Series**. The whole cases, on the same index.

**FUNCTION SIGNATURE**
`convert_whole_numbers(dataframe, number_patterns)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table to convert, passed by a cleaning function, a profiler or a shortage data tool.

**number_patterns** — **dict[str, str]**. Each column against the pattern that captures its number, passed from config, for example **config.ORDER_WHOLE_NUMBER_PATTERNS** or **config.SHIPMENT_WHOLE_NUMBER_PATTERNS**.
**PURPOSE**
Read counts, weeks, days and cases written with a unit as whole numbers.
**REQUIREMENTS**

- Each named column holds whole numbers, read through its pattern, with gaps kept as gaps.
- A value its pattern does not read, a fraction included, is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with each named column as whole numbers.

**FUNCTION SIGNATURE**
`convert_decimal_numbers(dataframe, number_patterns)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. The table to convert, passed by a cleaning function or a profiler.
**number_patterns** — **dict[str, str]**. Each column against the pattern that captures its number, passed from config, for example **config.ORDER_DECIMAL_NUMBER_PATTERNS**.
**PURPOSE**
Read percentages, weeks of stock and averages written with a unit or a sign as decimal numbers.
**REQUIREMENTS**

- Each named column holds decimal numbers, read through its pattern, with gaps kept as gaps.
- A value its pattern does not read is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with each named column as decimals.

**FUNCTION SIGNATURE**
`treat_values_below_minimum_as_missing(dataframe, minimum_values)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A table whose named columns are already numbers, passed by **clean_order_records** or **clean_demand_dataset**.
**minimum_values** — **dict[str, int]**. Each column against its lowest possible value, passed from config, for example **config.ORDER_MINIMUM_VALUES** or **config.DEMAND_MINIMUM_VALUES**.
**PURPOSE**
Blank readings that cannot be true.
**REQUIREMENTS**

- A value below its column's minimum becomes missing; every other value, and every row, is kept.

**RETURNS**
**pandas.DataFrame**. The table with impossible readings blanked.

**FUNCTION SIGNATURE**
`treat_counts_above_total_as_missing(dataframe)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. Order lines whose counts are already numbers, passed by **clean_order_records**.
**PURPOSE**
Blank a count that is larger than the whole it counts.
**REQUIREMENTS**

- For each count of **config.ORDER_COUNT_TOTALS**, a count greater than its total becomes missing; a count equal to its total, a count beside a missing total, and every row are kept.

**RETURNS**
**pandas.DataFrame**. The table with contradictory counts blanked.

**MODULE: data_pipeline/order_cleaning.py**
The cleaning every set of order lines goes through, the history's and the live queue's alike, and the cleaning run for the order history extracts.

**FUNCTION SIGNATURE**
`clean_order_records(frame, value_mappings, whole_number_patterns, decimal_number_patterns, mini`

**PARAMETERS**
**frame** — **pandas.DataFrame**. Order lines as **load_raw_dataset** read them, passed by **clean_order_dataset** or **fetch_open_orders**.
**value_mappings** — **dict[str, dict]**. The category columns and their mappings, passed from config: **config.ORDER_VALUE_MAPPINGS** or **config.OPEN_ORDER_VALUE_MAPPINGS**.
**whole_number_patterns** — **dict[str, str]**. The whole-number columns and their patterns, passed from config: **config.ORDER_WHOLE_NUMBER_PATTERNS** or **config.OPEN_ORDER_WHOLE_NUMBER_PATTERNS**.
**decimal_number_patterns** — **dict[str, str]**. The decimal columns and their patterns, passed from config: **config.ORDER_DECIMAL_NUMBER_PATTERNS** or **config.OPEN_ORDER_DECIMAL_NUMBER_PATTERNS**.
**minimum_values** — **dict[str, int]**. The columns and their lowest possible values, passed from config: **config.ORDER_MINIMUM_VALUES** or **config.OPEN_ORDER_MINIMUM_VALUES**.
**PURPOSE**

Clean a set of order lines, whatever route wrote each one.

**REQUIREMENTS**

- Duplicates are removed, every mapped column holds its canonical vocabulary, customer names are normalised, and every number is read under the patterns given.
- **account_code** is canonical and **ordered_quantity** is whole cases, each read through the notation of the line's own route, the units in a case taken from the line's own pack description and the cases on a pallet from its own **cases_per_pallet**; **pack_description** is canonical.
- A reading below its minimum, and a count above its total in **config.ORDER_COUNT_TOTALS**, becomes missing.
- An order sent by two routes is kept once, as it first arrived; a purchase order and product arriving from two accounts is refused, naming the orders. A value no notation reads is refused even on a row cleaning removes.

**RETURNS**

**pandas.DataFrame**. One cleaned line per order, in the order they arrived, numbered from nought upwards.

**FUNCTION SIGNATURE**

```
clean_order_dataset(input_file_path, output_file_path)
```

**PARAMETERS**

**input_file_path** — **str**. A raw order history extract, passed by **run_data_pipeline.py** from **config.RAW_ORDER_HISTORY_TRAIN_FILE** or **config.RAW_ORDER_HISTORY_EVAL_FILE**.
**output_file_path** — **str**. Where the cleaned table is written, passed by the same caller from **config.CLEAN_ORDER_HISTORY_TRAIN_FILE** or **config.CLEAN_ORDER_HISTORY_EVAL_FILE**.

**PURPOSE**

Produce an order table the inflated-order model can be fitted on.

**REQUIREMENTS**

- **cut_after_shortage** is folded by **config.ORDER_TARGET_VALUE_MAPPINGS**, and the lines are cleaned by **clean_order_records** under the order history's reading rules: **config.ORDER_VALUE_MAPPINGS**, **config.ORDER_WHOLE_NUMBER_PATTERNS**, **config.ORDER_DECIMAL_NUMBER_PATTERNS** and **config.ORDER_MINIMUM_VALUES**.
- An order whose first arrival carries no answer is removed, whatever a later arrival carries.
- The table is saved to **output_file_path** with the raw file's columns, in the raw file's order.

**RETURNS**

**pandas.DataFrame**. The cleaned order table, as saved.

**MODULE: data_pipeline/demand_cleaning.py**

The cleaning run for the weekly demand extracts.

**FUNCTION SIGNATURE**

```
clean_demand_dataset(input_file_path, output_file_path)
```

**PARAMETERS**

**input_file_path** — **str**. A raw weekly demand extract, passed by **run_data_pipeline.py** from **config.RAW_WEEKLY_DEMAND_TRAIN_FILE** or **config.RAW_WEEKLY_DEMAND_EVAL_FILE**.
**output_file_path** — **str**. Where the cleaned table is written, passed by the same caller from **config.CLEAN_WEEKLY_DEMAND_TRAIN_FILE** or **config.CLEAN_WEEKLY_DEMAND_EVAL_FILE**.

**PURPOSE**

Produce a demand table the demand model can be fitted on.

**REQUIREMENTS**

- Duplicates are removed, every category column holds its canonical vocabulary under **config.DEMAND_VALUE_MAPPINGS**, every number is read under **config.DEMAND_WHOLE_NUMBER_PATTERNS** and **config.DEMAND_DECIMAL_NUMBER_PATTERNS**, and **pack_description** is canonical.
- A reading below its minimum in **config.DEMAND_MINIMUM_VALUES** becomes missing. A week whose **next_week_cases** is missing or below nought has no answer and is removed, but a value no pattern reads is refused even on such a row.
- The table is saved to **output_file_path** with the raw file's columns, in the raw file's order.

**RETURNS**

**pandas.DataFrame**. The cleaned demand table, as saved.

**MODULE: data_pipeline/email_cleaning.py**

The cleaning run for the customer emails, and the repairs the live emails share with it.

**FUNCTION SIGNATURE**

```
convert_received_times(dataframe)
```

**PARAMETERS**

**dataframe** — **pandas.DataFrame**. An emails table, passed by **clean_email_dataset** or **fetch_open_emails**.

**PURPOSE**

Write every email's time one way.

**REQUIREMENTS**

- **config.EMAIL_RECEIVED_AT_COLUMN** holds the weekday in full, a space and the time, read as the parts **config.RECEIVED_WEEKDAY_PART** and **config.RECEIVED_TIME_PART** of **config.RECEIVED_AT_PATTERN**, the weekday

through **config.WEEKDAY_MAPPING**: **FRIDAY 10:17** is **Friday 10:17**. A gap stays a gap.
- A time that cannot be read is refused, naming the column and the values.

**RETURNS**
**pandas.DataFrame**. The table with every time written canonically.

**FUNCTION SIGNATURE**
`convert_markup_to_plain_text(email_texts)`

**PARAMETERS**
**email_texts** — **pandas.Series**. Email texts as written, passed by **clean_email_text** or **profile_email_details**.
**PURPOSE**
Turn email text into plain text.
**REQUIREMENTS**
- No markup matched by **config.MARKUP_PATTERN** is left; escaped characters are the characters they stand for, and a no-break space is an ordinary space; words a tag separated stay separate, every run matched by **config.WHITESPACE_RUN_PATTERN** is a single space, and nothing leads or trails. A gap stays a gap, and a text that is nothing but markup becomes empty.

**RETURNS**
**pandas.Series**. The plain texts, on the same index.

**FUNCTION SIGNATURE**
`clean_email_text(dataframe)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. An emails table, passed by **clean_email_dataset** or **fetch_open_emails**.
**PURPOSE**
Leave every email as plain words.
**REQUIREMENTS**
- **config.EMAIL_TEXT_COLUMN** holds the plain text **convert_markup_to_plain_text** gives, and a plain text that has letters and is wholly in capitals is restored in lower case; any other text is kept as it is.

**RETURNS**
**pandas.DataFrame**. The table with plain email text.

**FUNCTION SIGNATURE**
`clean_email_dataset(input_file_path, output_file_path)`

**PARAMETERS**
**input_file_path** — **str**. The raw emails extract, passed by **run_data_pipeline.py** from **config.RAW_CUSTOMER_EMAILS_FILE**.
**output_file_path** — **str**. Where the cleaned table is written, passed by the same caller from **config.CLEAN_CUSTOMER_EMAILS_FILE**.
**PURPOSE**
Produce an emails table the request model can learn from.
**REQUIREMENTS**
- Duplicates and rows with no answer are removed, every category column holds its canonical vocabulary under **config.EMAIL_VALUE_MAPPINGS**, **email_sequence** is a whole number read under **config.EMAIL_WHOLE_NUMBER_PATTERNS**, and every time is written by **convert_received_times**; a value no mapping or pattern reads is refused even on a row removed for having no answer.
- **email_text** is cleaned by **clean_email_text**, and **email_length_chars** is the length of the text as cleaned. An email of fewer than **config.MINIMUM_EMAIL_WORDS** words, a word being any run of characters between whitespace, is removed.
- The table is saved to **output_file_path** with the raw file's columns, in the raw file's order.

**RETURNS**
**pandas.DataFrame**. The cleaned emails table, as saved.

**MODULE: data_pipeline/feature_builder.py**
Turns a cleaned table into model inputs, and builds the preparation saved with a model and applied alike in fitting and scoring.

**FUNCTION SIGNATURE**
`build_feature_frame(dataframe, numeric_columns, categorical_columns)`

**PARAMETERS**
**dataframe** — **pandas.DataFrame**. A cleaned table or one product's open orders, passed by **run_model_training.py**, **build_inflated_order_facts** or **forecast_customer_demand**.
**numeric_columns** — **list[str]**. The numeric inputs, passed from config, for example **config.INFLATED_ORDER_NUMERIC_FEATURES**.
**categorical_columns** — **list[str]**. The category inputs, passed from config, for example **config.INFLATED_ORDER_CATEGORICAL_FEATURES**.
**PURPOSE**
Select and type a model's inputs.
**REQUIREMENTS**
- The result holds the numeric columns, then the categorical columns, in the order given, and no other column; its rows, their order and their index are the table's own.

- Numeric columns are floating-point, whether they arrived as text or as numbers, and every gap, whether written as **None**, **NA** or **NaN**, is **NaN**; categorical columns hold text, and every gap is a missing value.

**RETURNS**
**pandas.DataFrame**. The model's inputs, one row per row of the table.

**FUNCTION SIGNATURE**
`build_model_pipeline(estimator, numeric_columns, categorical_columns)`

**PARAMETERS**
**estimator** — **sklearn.base.BaseEstimator**. An unfitted estimator, passed by **train_inflated_order_model** or **train_demand_model**.
**numeric_columns** — **list[str]**. The numeric inputs, passed on by the same caller from config.
**categorical_columns** — **list[str]**. The category inputs, passed on by the same caller from config.
**PURPOSE**
Join the input preparation and the estimator, so the preparation is saved with the model.
**REQUIREMENTS**

- Every preparation the inputs need is learned inside the pipeline, from the rows it is fitted on.
- A row with gaps, or with a category value never seen in fitting, is scored without error.

**RETURNS**
**sklearn.pipeline.Pipeline**. The preparation followed by the estimator, unfitted.

**MODULE: data_pipeline/quality_profiler.py**
Measures how dirty each raw extract was and how clean its cleaned table is, and writes that evidence to **output/data_quality_report.json**. Every count taken as received is taken once for each record, over its first copy, and a gap is never counted as a defect. Every row removed is accounted for once: **rows_removed** is the sum of **duplicate_rows**, **rows_missing_target** and, for each extract, **answered_second_arrivals_removed**, **rows_with_impossible_target** or **rows_with_too_few_words**.

**FUNCTION SIGNATURE**
`profile_dataset(raw_frame, clean_frame, target_column)`

**PARAMETERS**
**raw_frame** — **pandas.DataFrame**. The extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py**.
**clean_frame** — **pandas.DataFrame**. The same extract after cleaning, passed by the same caller.
**target_column** — **str**. The extract's answer column, passed from config, for example **config.EMAIL_TARGET_COLUMN**.
**PURPOSE**
Account for every row an extract arrived with.
**REQUIREMENTS**

- **rows_received**, **rows_kept** and **rows_removed** count the raw rows, the cleaned rows and the difference.
- **duplicate_rows** counts every copy of a row after its first, and **rows_missing_target** the first copies that carry no answer.
- **columns** is the cleaned table's column count, and **missing_cells_as_received** and **missing_cells_after_cleaning** give, for every column with at least one gap, its count of gaps, as received over the first copies and after cleaning.

**RETURNS**
**dict**. The eight keys above.

**FUNCTION SIGNATURE**
`profile_order_details(raw_frame, clean_frame)`

**PARAMETERS**
**raw_frame** — **pandas.DataFrame**. An order history extract as read, passed by **run_data_pipeline.py**.
**clean_frame** — **pandas.DataFrame**. The same extract after cleaning, passed by the same caller.
**PURPOSE**
Record the defects peculiar to the orders.
**REQUIREMENTS**

- Every count is taken over the first copies of the rows of **raw_frame**, as received.
- **answered_second_arrivals_removed** counts the arrivals of an order after its first that carry an answer; **account_codes_not_written_canonically** counts the account codes **config.CANONICAL_ACCOUNT_CODE_PATTERN** does not match in full; **quantities_by_unit_as_written** counts, for each unit word, the quantities written in it, read through their own route's notation.
- **pack_descriptions_not_written_canonically** counts the pack descriptions **config.CANONICAL_PACK_DESCRIPTION_PATTERN** does not match in full, and **numbers_written_with_a_unit_or_percent_sign** gives, for every column of **config.ORDER_WHOLE_NUMBER_PATTERNS** and **config.ORDER_DECIMAL_NUMBER_PATTERNS**, the number of values **config.DECIMAL_NUMBER_PATTERN** does not match in full.
- **values_below_minimum_as_received** gives, for every column of **config.ORDER_MINIMUM_VALUES**, the number of readings below its minimum, and **counts_above_their_total_as_received**, for every count of **config.ORDER_COUNT_TOTALS**, the number of rows whose count is greater than its total.
- **weekdays_not_written_canonically** counts the weekdays not written as one of the canonical weekdays of **config.WEEKDAY_MAPPING**, and **target_shares** gives each answer's share of the cleaned rows to **config.TARGET_SHARE_DECIMALS** places.

**RETURNS**

**dict**. The nine keys above.

**FUNCTION SIGNATURE**
`profile_demand_details(raw_frame, clean_frame)`

**PARAMETERS**
**raw_frame** — **pandas.DataFrame**. A weekly demand extract as read, passed by **run_data_pipeline.py**.
**clean_frame** — **pandas.DataFrame**. The same extract after cleaning, passed by the same caller.
**PURPOSE**
Record the defects peculiar to the customer weeks.
**REQUIREMENTS**

- Every count is taken over the first copies of the rows of **raw_frame**, as received.
- **pack_descriptions_not_written_canonically** counts the pack descriptions **config.CANONICAL_PACK_DESCRIPTION_PATTERN** does not match in full, and **numbers_written_with_a_unit_or_percent_sign** gives, for every column of **config.DEMAND_WHOLE_NUMBER_PATTERNS** and **config.DEMAND_DECIMAL_NUMBER_PATTERNS**, the number of values **config.DECIMAL_NUMBER_PATTERN** does not match in full.
- **values_below_minimum_as_received** gives, for every column of **config.DEMAND_MINIMUM_VALUES**, the number of readings below its minimum, and **rows_with_impossible_target** counts the first copies whose answer is below its minimum.
- **weekdays_not_written_canonically** counts the weekdays not written as one of the canonical weekdays of **config.WEEKDAY_MAPPING**, and **median_next_week_cases** is the cleaned answer's median.

**RETURNS**
**dict**. The six keys above.

**FUNCTION SIGNATURE**
`profile_email_details(raw_frame, clean_frame)`

**PARAMETERS**
**raw_frame** — **pandas.DataFrame**. The emails extract as read, passed by **run_data_pipeline.py**.
**clean_frame** — **pandas.DataFrame**. The same extract after cleaning, passed by the same caller.
**PURPOSE**
Record the defects peculiar to free text.
**REQUIREMENTS**

- Every count is taken over the first copies of the rows of **raw_frame**, as received.
- **weekdays_not_written_canonically** counts the times whose weekday, read through **config.RECEIVED_AT_PATTERN**, is not written as one of the canonical weekdays; **emails_with_markup** and **emails_with_encoding_artifacts** count the emails matched by **config.MARKUP_PATTERN** and by **config.ENCODING_ARTIFACT_PATTERN** as received, and **emails_in_capitals** the emails whose plain text is wholly in capitals.
- **email_lengths_disagreeing_with_plain_text_as_received** counts the lengths that differ from the length of their email's plain text, and **rows_with_too_few_words** the first copies with an answer whose plain text is too short to learn from.
- **shortest_email_words** is the fewest words a cleaned email holds, and **target_shares** gives each request type's share of the cleaned rows to **config.TARGET_SHARE_DECIMALS** places.

**RETURNS**
**dict**. The eight keys above.

**FUNCTION SIGNATURE**
`write_quality_report(profiles)`

**PARAMETERS**
**profiles** — **dict[str, dict]**. Each extract's name, the file name without its extension, against its general and detail profiles merged, passed by **run_data_pipeline.py**.
**PURPOSE**
Write the quality report.
**REQUIREMENTS**

- **config.DATA_QUALITY_REPORT_FILE** holds the profiles as indented JSON with sorted keys, its folder created if needed.

**RETURNS**
**None**.

## ========== MODELLING ==========

Three models are trained, scored on held-out data and saved with everything needed to score one row. The estimator and its tuning are the team's choice; the interface, the scores reported and each model's minimum are fixed.

**MODULE: modeling/model_registry.py**
Saves the models, their feature schemas and their scores, and loads the models and schemas back. Nothing else in the project writes to **output/models/**.

**FUNCTION SIGNATURE**
`save_model(model, file_path)`

**PARAMETERS**

**model** — **sklearn.pipeline.Pipeline**. A fitted model, passed by **run_model_training.py**.
**file_path** — **str**. Where it is written, passed from config: **config.INFLATED_ORDER_MODEL_FILE**, **config.DEMAND_MODEL_FILE** or **config.REQUEST_MODEL_FILE**.
**PURPOSE**
Save a fitted model, preparation included.
**REQUIREMENTS**
- The folder exists afterwards, and **load_model** on the same path gives back a model that scores exactly as the one saved.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
```
save_feature_schema(feature_names, file_path)
```
**PARAMETERS**
**feature_names** — **list[str]**. The columns of the inputs the model was fitted on, in order, passed by **run_model_training.py**.
**file_path** — **str**. Where they are written, passed from config: **config.INFLATED_ORDER_SCHEMA_FILE** or **config.DEMAND_SCHEMA_FILE**.
**PURPOSE**
Record the exact inputs a model expects.
**REQUIREMENTS**
- The file is a JSON list of the column names in the order given, its folder created if needed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
```
save_metrics(metrics)
```
**PARAMETERS**
**metrics** — **dict[str, dict]**. Each model's name against its record, as described under **run_model_training.py**, passed by **run_model_training.py**.
**PURPOSE**
Save every model's scores.
**REQUIREMENTS**
- **config.MODEL_METRICS_FILE** holds the metrics as indented JSON with sorted keys, its folder created if needed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
```
load_model(file_path)
```
**PARAMETERS**
**file_path** — **str**. A saved model, passed by the workflow's nodes module from config, for example **config.REQUEST_MODEL_FILE**.
**PURPOSE**
Bring a saved model back into memory.
**REQUIREMENTS**
- The model is returned exactly as saved, ready to score.

**RETURNS**
**sklearn.pipeline.Pipeline**. The fitted model.

**FUNCTION SIGNATURE**
```
load_feature_schema(file_path)
```
**PARAMETERS**
**file_path** — **str**. A saved schema, passed by the workflow's nodes module from config, for example **config.INFLATED_ORDER_SCHEMA_FILE**.
**PURPOSE**
Bring a model's input columns back into memory, in order.
**REQUIREMENTS**
- The names are returned in the order they were saved.

**RETURNS**
**list[str]**. The column names.

**MODULE: modeling/model_evaluation.py**
Scores a fitted model on held-out rows and on its training rows. No function here fits anything, and every score is a **float** rounded to **config.METRIC_DECIMALS** places.

**FUNCTION SIGNATURE**
```
evaluate_classifier(model, train_features, train_target, held_out_features, held_out_target)
```
**PARAMETERS**

**model** — **sklearn.pipeline.Pipeline**. The fitted inflated-order pipeline, whose classes are **0** and **1**, passed by **train_inflated_order_model** once it has fitted it.
**train_features** — **pandas.DataFrame**. The inputs the model was fitted on, passed on by **train_inflated_order_model**.
**train_target** — **pandas.Series**. The answers for those rows as integers, encoded by **config.ORDER_TARGET_ENCODING**, passed on by **train_inflated_order_model**.
**held_out_features** — **pandas.DataFrame**. The held-out inputs the model never saw, passed on by **train_inflated_order_model**.
**held_out_target** — **pandas.Series**. The answers for those rows, encoded the same way, passed on by **train_inflated_order_model**.
**PURPOSE**
Score the inflated-order model.
**REQUIREMENTS**

- **held_out_roc_auc** and **train_roc_auc** are measured from the probability of the positive class, and **generalisation_gap** is the training score less the held-out score.
- **accuracy**, **precision**, **recall** and **f1** describe the held-out predictions, and **mean_predicted_probability** and **positive_share** the held-out rows.

**RETURNS**
**dict**. The nine scores above.

**FUNCTION SIGNATURE**
`evaluate_regressor(model, train_features, train_target, held_out_features, held_out_target)`
**PARAMETERS**
**model** — **sklearn.pipeline.Pipeline**. The fitted demand pipeline, passed by **train_demand_model** once it has fitted it.
**train_features** — **pandas.DataFrame**. The inputs the model was fitted on, passed on by **train_demand_model**.
**train_target** — **pandas.Series**. The next week's cases for the training rows, passed on by **train_demand_model**.
**held_out_features** — **pandas.DataFrame**. The held-out inputs the model never saw, passed on by **train_demand_model**.
**held_out_target** — **pandas.Series**. The next week's cases for those rows, passed on by **train_demand_model**.
**PURPOSE**
Score the demand model and the always-the-average baseline.
**REQUIREMENTS**

- **held_out_r2** and **train_r2** are the R squared on each set of rows, and **generalisation_gap** is the training score less the held-out score.
- **mean_absolute_error** is the model's average miss on the held-out rows, and **average_estimate_mean_absolute_error** the average miss there of always answering the mean of **train_target**; **lowest_prediction** and **highest_prediction** are the model's lowest and highest forecasts there.

**RETURNS**
**dict**. The seven scores above.

**FUNCTION SIGNATURE**
`evaluate_text_classifier(model, train_text, train_target, held_out_text, held_out_target)`
**PARAMETERS**
**model** — **sklearn.pipeline.Pipeline**. The fitted request pipeline, passed by **train_request_model** once it has fitted it.
**train_text** — **pandas.Series**. The email texts it was fitted on, passed on by **train_request_model**.
**train_target** — **pandas.Series**. Their request types as text, passed on by **train_request_model**.
**held_out_text** — **pandas.Series**. Email texts it never saw, passed on by **train_request_model**.
**held_out_target** — **pandas.Series**. Their request types, passed on by **train_request_model**.
**PURPOSE**
Score the request model across every request type.
**REQUIREMENTS**

- **held_out_macro_f1** and **train_macro_f1** are the macro-averaged F1 on each set of emails, and **generalisation_gap** is the training score less the held-out score.
- **accuracy** and **weighted_f1** describe the held-out predictions, and **per_class_f1** is a **dict[str, float]** giving each of the model's classes its own held-out score.

**RETURNS**
**dict**. The six entries above.

**MODULE: modeling/inflated_order_model.py**
The model that predicts whether a customer will cut an order placed during a shortage once supply recovers.

**FUNCTION SIGNATURE**
`train_inflated_order_model(train_features, train_target, held_out_features, held_out_target)`
**PARAMETERS**
**train_features** — **pandas.DataFrame**. The cleaned training orders as **build_feature_frame** built them from **config.INFLATED_ORDER_NUMERIC_FEATURES** and **config.INFLATED_ORDER_CATEGORICAL_FEATURES**, passed by **run_model_training.py**.
**train_target** — **pandas.Series**. **cut_after_shortage** for those orders as integers, encoded by **config.ORDER_TARGET_ENCODING**, passed by **run_model_training.py**.
**held_out_features** — **pandas.DataFrame**. The cleaned held-out orders, built the same way, passed by **run_model_training.py**.
**held_out_target** — **pandas.Series**. **cut_after_shortage** for those orders, encoded the same way, passed by **run_model_training.py**.

**PURPOSE**
Predict the probability that a customer cuts or cancels an order once supply recovers.
**REQUIREMENTS**

- The model is a pipeline from **build_model_pipeline**, fitted on the training rows only; it records the names of the columns it was fitted on, gives probabilities, and its classes are **0** and **1** in that order.
- Its held-out ROC AUC reaches **config.MINIMUM_INFLATED_ORDER_ROC_AUC**; across the held-out orders its probabilities spread with a standard deviation above **0.05**, orders that were cut average a higher score than orders that were kept, and the mean score is within **0.05** of the share of those orders that were cut. Its scores for the live queue fall in all three bands, with a standard deviation above **0.05**.
- On average it scores the same orders higher when each stands **100** percentage points further above the customer's usual week, and when the same order also arrived by another route than when it did not.
- Its scores come from **evaluate_classifier**.

**RETURNS**
**tuple[sklearn.pipeline.Pipeline, dict]**. The fitted model and its scores.

**MODULE: modeling/demand_model.py**
The model that forecasts the cases a customer will need next week.

**FUNCTION SIGNATURE**
`train_demand_model(train_features, train_target, held_out_features, held_out_target)`

**PARAMETERS**
**train_features** — **pandas.DataFrame**. The cleaned training weeks as **build_feature_frame** built them from **config.DEMAND_NUMERIC_FEATURES** and **config.DEMAND_CATEGORICAL_FEATURES**, passed by **run_model_training.py**.
**train_target** — **pandas.Series**. **next_week_cases** for those weeks, passed by **run_model_training.py**.
**held_out_features** — **pandas.DataFrame**. The cleaned held-out weeks, built the same way, passed by **run_model_training.py**.
**held_out_target** — **pandas.Series**. **next_week_cases** for those weeks, passed by **run_model_training.py**.
**PURPOSE**
Forecast how many cases a customer will really need next week.
**REQUIREMENTS**

- The model is a pipeline from **build_model_pipeline**, fitted on the training rows only, and it records the names of the columns it was fitted on.
- Its held-out R squared reaches **config.MINIMUM_DEMAND_R2**, its average miss is smaller than that of always answering the training average, its forecasts spread across the held-out weeks with a standard deviation above **20** cases, and no forecast the saved pipeline itself gives is below nought.
- On average it forecasts more for the same weeks with a four-week average **20** cases higher, and for the same weeks with a promotion next week than without one.
- Its scores come from **evaluate_regressor**.

**RETURNS**
**tuple[sklearn.pipeline.Pipeline, dict]**. The fitted model and its scores.

**MODULE: modeling/request_model.py**
The NLP model that reads a customer's email and names the request it makes.

**FUNCTION SIGNATURE**
`train_request_model(train_text, train_target, held_out_text, held_out_target)`

**PARAMETERS**
**train_text** — **pandas.Series**. The training share of the cleaned emails' **email_text** in the split **run_model_training.py** makes, passed by **run_model_training.py**.
**train_target** — **pandas.Series**. Their **request_type**, from the same split, passed by **run_model_training.py**.
**held_out_text** — **pandas.Series**. The held-out share of the cleaned emails' **email_text**, from the same split, passed by **run_model_training.py**.
**held_out_target** — **pandas.Series**. Their **request_type**, from the same split, passed by **run_model_training.py**.
**PURPOSE**
Name the request an email makes.
**REQUIREMENTS**

- The model is one pipeline that takes raw email texts, as a list or a Series of strings, fitted on the training emails only.
- Its classes are the eight request types of **config.REQUEST_GUIDE_DOCUMENTS**, no more and no fewer, and any text, however strange, is read as one of them.
- Its held-out macro F1 reaches **config.MINIMUM_REQUEST_MACRO_F1**, every request type is predicted for some held-out email, and the emails making each request are most often read as that request.
- Its scores come from **evaluate_text_classifier**.

**RETURNS**
**tuple[sklearn.pipeline.Pipeline, dict]**. The fitted model and its scores.

**========== THE KNOWLEDGE BASE ==========**

The sixteen documents are read once, cut into passages and stored for search by meaning. Every later lookup goes through that store, and a category's rules are read from its own policy alone.

**MODULE: knowledge/pdf_extraction.py**
Gets the text out of the documents.

**FUNCTION SIGNATURE**
`extract_text_from_directory(docs_directory)`

**PARAMETERS**
**docs_directory** — **str**. The folder that holds the documents, passed by **run_knowledge_indexing.py** from **config.DOCS_DIRECTORY**.
**PURPOSE**
Read every PDF in the folder.
**REQUIREMENTS**

- Every PDF in the folder gives one entry, in file-name order, holding **document_name**, the file name, and **document_text**, the text of all its pages joined by line breaks; a file that is not a PDF is skipped.
- A document with a page from which no text can be extracted is refused, naming the file, and so is a folder that holds no PDF.

**RETURNS**
**list[dict]**. One entry per document.

**MODULE: knowledge/chunk_splitting.py**
Cuts the documents into numbered sections, and the sections into passages, never cutting a sentence in two.

**FUNCTION SIGNATURE**
`split_documents_into_sections(document_entries)`

**PARAMETERS**
**document_entries** — **list[dict]**. The documents, each holding **document_name** and **document_text**, as **extract_text_from_directory** returned them, passed by **run_knowledge_indexing.py**.
**PURPOSE**
Divide each document at its section headings and drop its header line.
**REQUIREMENTS**

- Each document carries exactly one header line matched by **config.DOCUMENT_HEADER_PATTERN**, which is removed, and sections matched by **config.SECTION_HEADING_PATTERN** numbered **1**, **2**, **3** and on without a gap; a document that does not is refused, naming it.
- Each section becomes one entry holding **document_name** and **section_text**, the text before the second heading belonging to the first section, with every run matched by **config.WHITESPACE_RUN_PATTERN** written as one space and nothing leading or trailing.

**RETURNS**
**list[dict]**. One entry per section, documents and sections in their own order.

**FUNCTION SIGNATURE**
`build_chunk_records(section_entries)`

**PARAMETERS**
**section_entries** — **list[dict]**. The sections, as **split_documents_into_sections** returned them, passed by **run_knowledge_indexing.py**.
**PURPOSE**
Cut each section into passages of about **config.CHUNK_SIZE_CHARACTERS** characters that overlap their neighbours.
**REQUIREMENTS**

- A section is cut only between sentences, at the boundaries **config.SENTENCE_BOUNDARY_PATTERN** marks, so every sentence lies whole within a passage; a passage never crosses from one section into the next, and a sentence longer than a passage stands alone.
- A passage ends only where its next sentence would take it past **config.CHUNK_SIZE_CHARACTERS**, and the passage after it opens with as many of its last whole sentences as fit within **config.CHUNK_OVERLAP_CHARACTERS** while leaving room for that next sentence.
- Each passage holds **chunk_id**, written as the document name, a hyphen and the passage's number within that document counted from **0**, **chunk_text** and **document_name**.

**RETURNS**
**list[dict]**. Every passage of every section, in order.

**MODULE: knowledge/vector_index.py**
Turns passages into vectors, stores them in a persistent Chroma collection, reads a whole document back, and searches by meaning.

**FUNCTION SIGNATURE**

`build_vector_index(chunk_records, store_path)`

**PARAMETERS**
**chunk_records** — **list[dict]**. The passages, as **build_chunk_records** returned them, passed by **run_knowledge_indexing.py**.
**store_path** — **str**. The folder the store lives in, passed by **run_knowledge_indexing.py** from **config.VECTOR_STORE_DIRECTORY**.
**PURPOSE**
Build the searchable knowledge base from scratch.
**REQUIREMENTS**

- Any store already at **store_path** is replaced, so building twice never holds a passage twice.
- The collection is named **config.VECTOR_COLLECTION_NAME**; passages are embedded with **config.EMBEDDING_MODEL_NAME** on **config.EMBEDDING_DEVICE**, and closeness is measured in the distance **config.VECTOR_DISTANCE_METRIC** names.
- Each passage is stored under its **chunk_id**, with its text, and with its **document_name** as metadata.

**RETURNS**
**int**. The number of passages the store holds.

**FUNCTION SIGNATURE**
`open_vector_index(store_path)`

**PARAMETERS**
**store_path** — **str**. The folder the store lives in, passed by the workflow's nodes module and by **run_knowledge_indexing.py** from **config.VECTOR_STORE_DIRECTORY**.
**PURPOSE**
Open the store that indexing built, without building anything.
**REQUIREMENTS**

- The collection **config.VECTOR_COLLECTION_NAME** is opened with the same embedding model it was built with.
- A folder holding no store file **config.VECTOR_STORE_FILE_NAME**, or no folder at all, is refused, naming it.

**RETURNS**
**chromadb.api.models.Collection.Collection**. The opened collection.

**FUNCTION SIGNATURE**
`read_document_chunks(collection, document_name)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by **read_allocation_policy** or **answer_from_documents**.
**document_name** — **str**. One document's file name, passed by the same caller from **config.CATEGORY_POLICY_DOCUMENTS**, for example **Allocation-Policy-Laundry.pdf**.
**PURPOSE**
Read one document whole.
**REQUIREMENTS**

- Every passage stored for **document_name** is returned, and no passage of any other document.
- A document with no passages in the store is refused with **config.INDEX_HOLDS_NO_CHUNKS_MESSAGE**.

**RETURNS**
**list[str]**. The document's passage texts.

**FUNCTION SIGNATURE**
`find_nearest_passage(collection, question, document_name)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by the retrieval tools.
**question** — **str**. What the passage should answer, passed by the retrieval tools from config, for example **config.CUSTOMER_ADVICE_QUESTION**.
**document_name** — **str**. The one document to search, passed by the retrieval tools from **config.CATEGORY_POLICY_DOCUMENTS** or **config.REQUEST_GUIDE_DOCUMENTS**.
**PURPOSE**
Find the one passage of one document that best answers one question.
**REQUIREMENTS**

- The passage returned is the one of **document_name** closest in meaning to **question**, and never another document's.
- A document with no passages in the store is refused with **config.INDEX_HOLDS_NO_CHUNKS_MESSAGE**.

**RETURNS**
**str**. The passage text.

**FUNCTION SIGNATURE**
`search_documents(collection, query_texts)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by **answer_from_documents**.
**query_texts** — **list[str]**. One or more questions searched together, passed by **answer_from_documents** as the question and the conversation's **search_text**.

**PURPOSE**
Search by meaning, across every document the chat may read.
**REQUIREMENTS**
- There is one result list per question, in the order the questions were given, each holding the **config.CHAT_RETRIEVAL_RESULT_COUNT** passages of the documents the chat may read that are closest to it, closest first.
- Each passage carries **chunk_id**, **chunk_text**, **document_name** and **distance**, the distance as a **float**.
- No passage comes from a document in **config.CHAT_EXCLUDED_DOCUMENTS**.

**RETURNS**
**list[list[dict]]**. The passages found for each question.

**MODULE: knowledge/policy_reading.py**
Reads one product category's four rules out of its own allocation policy.

**FUNCTION SIGNATURE**
```
read_allocation_policy(collection, product_category)
```
**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by **run_knowledge_indexing.py**, by the service at start-up or by **compute_allocation_node**.
**product_category** — **str**. One of the five categories, from a product's **product_category** or from the keys of **config.CATEGORY_POLICY_DOCUMENTS**, passed by the same caller.
**PURPOSE**
Read the rules a product's cases are divided under.
**REQUIREMENTS**
- The rules are read from every passage of the category's own policy, named by **config.CATEGORY_POLICY_DOCUMENTS**, and from no other document; a category with no configured policy is refused with **config.NO_POLICY_CONFIGURED_MESSAGE**.
- **history_weeks** is the **int config.HISTORY_SENTENCE_PATTERN** reads; **cap** is the **fractions.Fraction config.CAP_SENTENCE_PATTERN** reads, so **1.4** would be exactly seven fifths; **protected_channels** lists the channels **config.PROTECTED_SENTENCE_PATTERN** reads, split at **config.PROTECTED_CHANNEL_SEPARATOR**, in the order written, or is empty where **config.NO_PROTECTION_SENTENCE_PATTERN** is stated; **reserve_percent** is the **int config.RESERVE_SENTENCE_PATTERN** reads, or **None** where **config.NO_RESERVE_SENTENCE_PATTERN** is stated.
- **rule_sentences** maps **history**, **cap**, **protected** and **reserve** to the policy's own sentence for each rule.
- A policy that fails in any way set out in **THE RULES ARE NOT IN THE SOLUTION** is refused, naming the document.

**RETURNS**
**dict**. **document_name**, the policy's file name, **history_weeks**, **cap**, **protected_channels**, **reserve_percent** and **rule_sentences**.

**========== AGENT TOOLS ==========**
Every fact an agent is given is assembled here; no agent works one out for itself. Every function works on data its caller has loaded: no tool reads a file or changes the data it is given.

**MODULE: tools/shortage_data_tools.py**
Serves the live queue: one product's profile, its open orders read through each route's notation, its shipment history, its customers and their open emails.

**FUNCTION SIGNATURE**
```
fetch_shortage_profile(shortage_products, product_code)
```
**PARAMETERS**
**shortage_products** — **pandas.DataFrame**. The products on allocation, read as text once at start-up by **load_raw_dataset** from **config.SHORTAGE_PRODUCTS_FILE**, passed by **load_shortage_context_node** or by the service at start-up.
**product_code** — **str**. The product being asked about, for example **HS-0500**, taken by the same caller from the workflow state or from the queue.
**PURPOSE**
Return one product's own facts.
**REQUIREMENTS**
- The record is the product's one row, with every column the queue holds and the six columns of **config.SHORTAGE_PRODUCT_WHOLE_NUMBER_PATTERNS** read as whole numbers.
- A product the queue does not hold is refused with **config.PRODUCT_NOT_IN_QUEUE_MESSAGE** filled with the code; a product the queue holds more than once, and a product with any detail left blank, are refused.

**RETURNS**
**dict**. The product's row, keyed by column name, its six counts as integers and every other value the **str** the queue holds.

**FUNCTION SIGNATURE**
```
fetch_open_orders(open_orders, product_code)
```
**PARAMETERS**

**open_orders** — **pandas.DataFrame**. Every open order line, read as text once at start-up by **load_raw_dataset** from **config.OPEN_ORDERS_FILE**, each line exactly as its route wrote it, passed by **load_shortage_context_node** or by the service at start-up.
**product_code** — **str**. The product whose orders are wanted, passed by the same caller.
**PURPOSE**
Read one product's open orders through the notation of the route that wrote each one.
**REQUIREMENTS**

- Only the product's lines are read, through **clean_order_records** under the live orders' reading rules: **config.OPEN_ORDER_VALUE_MAPPINGS**, **config.OPEN_ORDER_WHOLE_NUMBER_PATTERNS**, **config.OPEN_ORDER_DECIMAL_NUMBER_PATTERNS** and **config.OPEN_ORDER_MINIMUM_VALUES**; an order sent by two routes is kept once, as it first arrived.
- A product with no open orders, lines that do not each carry an order line id of their own, and an order that leaves its account or its quantity blank, are refused, naming them.

**RETURNS**
**pandas.DataFrame**. One cleaned line per order, every account canonical and every quantity in whole cases.

**FUNCTION SIGNATURE**
`fetch_shipment_history(shipment_history, product_code)`
**PARAMETERS**
**shipment_history** — **pandas.DataFrame**. Every weekly shipment, read as text once at start-up by **load_raw_dataset** from **config.SHIPMENT_HISTORY_FILE**, passed by **compute_allocation_node** or by the service at start-up.
**product_code** — **str**. The product whose shipments are wanted, passed by the same caller.
**PURPOSE**
Read one product's shipments in whole weeks and cases.
**REQUIREMENTS**

- Only the product's shipments are returned, numbered from nought upwards, with **weeks_ago** and **cases_shipped** read through **config.SHIPMENT_WHOLE_NUMBER_PATTERNS**.
- An account code that is not canonical, a shipment with no reference, a reference recorded more than once, and a shipment that leaves its week or its cases blank, are refused, naming them.

**RETURNS**
**pandas.DataFrame**. The product's shipments, a row for each week a customer received it.

**FUNCTION SIGNATURE**
`fetch_customer_accounts(customer_master, account_codes)`
**PARAMETERS**
**customer_master** — **pandas.DataFrame**. Every customer, read as text once at start-up by **load_raw_dataset** from **config.CUSTOMER_MASTER_FILE**, passed by **load_shortage_context_node** or by the service at start-up.
**account_codes** — **list[str]**. The canonical codes of the customers with an open order of the product, passed by the same caller.
**PURPOSE**
Give each customer of a product its name and channel, the channel deciding whether it is protected.
**REQUIREMENTS**

- Every code given has one entry, keyed by its code in code order, holding **customer_name** and **channel** as the master holds them.
- An account the master does not hold, holds more than once, leaves without a name or does not hold as **config.ACCOUNT_STATUS_ACTIVE**, and a channel that is not written as one of the eight channels of **config.CHANNEL_MAPPING**, are refused, naming the accounts.

**RETURNS**
**dict[str, dict]**. Each account code against its **customer_name** and **channel**.

**FUNCTION SIGNATURE**
`fetch_open_emails(open_customer_emails, product_code, account_codes)`
**PARAMETERS**
**open_customer_emails** — **pandas.DataFrame**. The open emails, read as text once at start-up by **load_raw_dataset** from **config.OPEN_CUSTOMER_EMAILS_FILE**, passed by **load_shortage_context_node** or **get_shortage_detail**.
**product_code** — **str**. The product under review, passed by the same caller.
**account_codes** — **list[str]**. The canonical codes of the product's customers, passed by the same caller.
**PURPOSE**
Hand the request model the open emails that bear on one product's orders.
**REQUIREMENTS**

- The emails returned are those of the given customers that name the product or name no product, each customer's in **email_sequence** order read as a number, customers in code order, whatever order the table is in.
- Each email is repaired as the training emails are: the sender through **config.OPEN_EMAIL_VALUE_MAPPINGS**, the time by **convert_received_times** and the text by **clean_email_text**; it holds exactly the fields of **config.OPEN_EMAIL_FIELDS**, with **email_sequence** as an integer.
- An email naming the product or no product from an account code that is not canonical is refused, whether or not it is a given customer's, and so are emails that do not each carry their own place among their customer's emails, an email with no text once cleaned, and an email that leaves its sender, time or subject blank, each refusal naming them.

**RETURNS**

**list[dict]**. One entry per email, in that order.

**MODULE: tools/prediction_tools.py**
Turns a model's raw output into the facts the business acts on.
**FUNCTION SIGNATURE**
`build_inflated_order_facts(inflated_order_model, inflated_order_feature_names, product_orders)`
**PARAMETERS**
**inflated_order_model** — **sklearn.pipeline.Pipeline**. The saved inflated-order model, loaded once at start-up by the workflow's nodes module, passed by **load_shortage_context_node** or **get_shortage_detail**.
**inflated_order_feature_names** — **list[str]**. Its saved feature schema, loaded with it, passed by the same caller.
**product_orders** — **pandas.DataFrame**. One product's open orders, as **fetch_open_orders** returned them, passed by the same caller.
**PURPOSE**
Score and band every order's chance of being cut once supply recovers.
**REQUIREMENTS**
- Every order is scored on the inputs **build_feature_frame** builds from its line, in the saved schema's order; **inflated_order_score** and **inflated_order_band** are as **THE INFLATED-ORDER SCORE AND ITS BAND** sets out.
- Facts follow the lines by position, whatever labels the table's index carries.

**RETURNS**
**list[dict]**. One entry per order, in the table's order, holding **order_line_id**, **account_code**, **inflated_order_score** and **inflated_order_band**.

**FUNCTION SIGNATURE**
`forecast_customer_demand(demand_model, demand_feature_names, product_orders)`
**PARAMETERS**
**demand_model** — **sklearn.pipeline.Pipeline**. The saved demand model, loaded once at start-up by the workflow's nodes module, passed by **load_shortage_context_node** or by the service at start-up.
**demand_feature_names** — **list[str]**. Its saved feature schema, loaded with it, passed by the same caller.
**product_orders** — **pandas.DataFrame**. One product's open orders, as **fetch_open_orders** returned them, passed by the same caller.
**PURPOSE**
Forecast the cases each customer of a product will need next week.
**REQUIREMENTS**
- Every customer with an open order is forecast once, on the inputs **build_feature_frame** builds from its first order line of the product in the order log, in the saved schema's order.

**RETURNS**
**dict[str, float]**. Each account code, in code order, against its forecast cases.

**FUNCTION SIGNATURE**
`build_request_facts(request_model, open_emails)`
**PARAMETERS**
**request_model** — **sklearn.pipeline.Pipeline**. The saved request model, loaded once at start-up by the workflow's nodes module, passed by **load_shortage_context_node** or **get_shortage_detail**.
**open_emails** — **list[dict]**. One product's open emails, as **fetch_open_emails** returned them, passed by the same caller.
**PURPOSE**
Name the request every email makes, and each customer's most urgent.
**REQUIREMENTS**
- **email_requests** is every email, in the order given, carried whole with **request_type** added, the request the model reads in its text.
- **customer_requests** gives each customer that wrote its most urgent request, as **READING A CUSTOMER EMAIL** sets out; a request the configuration does not rank is refused. No emails give two empty results.

**RETURNS**
**dict**. **email_requests** and **customer_requests**.

**FUNCTION SIGNATURE**
`compute_forecast_shortfall(customer_forecasts, cases_available_next_week)`
**PARAMETERS**
**customer_forecasts** — **dict[str, float]**. A product's customer forecasts, as **forecast_customer_demand** returned them, passed by **compose_allocation_recommendation_node** or by the service at start-up.
**cases_available_next_week** — **int**. The product's **cases_available_next_week**, from its profile, passed by the same caller.
**PURPOSE**
Work out how short a product will be next week.
**REQUIREMENTS**
- The shortfall is the sum of the forecasts, rounded once to the nearest whole case, an exact half to the even number, less next week's cases, as **THE FORECAST SHORTFALL, AND THE ORDER OF THE QUEUE** sets out; a shortfall below nought is kept as it is.

**RETURNS**

**int**. The forecast shortfall, in cases.

**MODULE: tools/allocation_tools.py**
Works out usual demand, divides the cases under each release, summarises the division and the decisions open, works out what a decision confirms, and ranks the queue.

**FUNCTION SIGNATURE**
`compute_usual_demand(product_shipments, account_codes, history_weeks)`
**PARAMETERS**
**product_shipments** — **pandas.DataFrame**. One product's shipments, as **fetch_shipment_history** returned them, passed by **compute_allocation_node** or by the service at start-up.
**account_codes** — **list[str]**. The product's customers, the keys of **fetch_customer_accounts**, passed by the same caller.
**history_weeks** — **int**. The policy's weeks of history, the **history_weeks** of **read_allocation_policy**, passed by the same caller.
**PURPOSE**
Work out each customer's usual demand, exactly.
**REQUIREMENTS**
- The result holds exactly the customers given, each with a usual demand, worked out as **USUAL DEMAND AND THE SILENT WEEK** sets out: the shipments at most **history_weeks** weeks ago, divided by **history_weeks**, a silent week counting as nought and a customer with no shipment in the window having nought.
- Each is an exact fraction, never rounded.

**RETURNS**
**dict[str, fractions.Fraction]**. Each account code against its usual demand.

**FUNCTION SIGNATURE**
`allocate_cases(cases_available, product_orders, customer_accounts, usual_demand, policy_rules)`
**PARAMETERS**
**cases_available** — **int**. The cases to divide this week, the product's **cases_available_this_week**, passed by **compute_allocation_node** or by the service at start-up.
**product_orders** — **pandas.DataFrame**. The product's open orders, as **fetch_open_orders** returned them, passed by the same caller.
**customer_accounts** — **dict[str, dict]**. The product's customers, as **fetch_customer_accounts** returned them, passed by the same caller.
**usual_demand** — **dict[str, fractions.Fraction]**. Each customer's usual demand, as **compute_usual_demand** returned it, passed by the same caller.
**policy_rules** — **dict**. The category's rules, as **read_allocation_policy** returned them, the cap an exact **fractions.Fraction**, passed by the same caller.
**PURPOSE**
Divide the cases among the customers under each release the policy allows.
**REQUIREMENTS**
- Each customer's cases ordered and eligible cases are worked out as **WHAT A CUSTOMER MAY HAVE** sets out, and the cases are divided as **THE DIVISION** sets out, under **config.DECISION_RELEASE** and, where **reserve_percent** is not **None**, under **config.DECISION_RELEASE_WITH_RESERVE**.
- **allocation_lines** holds one line per customer, in account-code order: **account_code**, **customer_name**, **channel**, **protected** as a **bool**, **cases_ordered**, **usual_demand** as a **float** rounded half up to **config.USUAL_DEMAND_DECIMALS** places, **eligible_cases**, and **cases_by_release**, each release's name against the customer's cases under it.
- **reserves_by_release** gives each release's name against the cases it holds in reserve, nought under **RELEASE**.

**RETURNS**
**dict**. **cases_available**, **allocation_lines** and **reserves_by_release**.

**FUNCTION SIGNATURE**
`summarise_allocation(allocation)`
**PARAMETERS**
**allocation** — **dict**. The division, as **allocate_cases** returned it, passed by **compute_allocation_node** or by the service at start-up.
**PURPOSE**
Summarise the division and find the decisions open.
**REQUIREMENTS**
- **cases_available**, **cases_ordered** and **cases_eligible** are the totals; **cover_percent** is the cover as **THREE DECISIONS, AND THE DECISIONS OPEN** sets out, a **float**, or **None** where no customer may have any case; **protected_served_in_full** says whether every protected customer gets its eligible cases under **RELEASE**.
- **releases** gives each release of the division its **cases_placed**, **reserve_cases**, **cases_in_stock** and **customers_short**, the codes, in code order, of the customers given fewer cases than they may have.
- **decisions_open** lists the open decisions in the order of **config.DECISIONS**, and **decisions_not_open** gives each closed decision its reasons, joined by **config.REASON_SEPARATOR**, as **THREE DECISIONS, AND THE DECISIONS OPEN** sets out.

**RETURNS**
**dict**. **cases_available**, **cases_ordered**, **cases_eligible**, **cover_percent**, **protected_served_in_full**, **releases**, **decisions_open** and **decisions_not_open**.

**FUNCTION SIGNATURE**

```
compute_allocation_outcome(decision, allocation, allocation_summary, product_name)
```

**PARAMETERS**

**decision** — **str**. The Supply Planning Manager's decision, from the decision the workflow resumed with, passed by **request_allocation_decision_node**.

**allocation** — **dict**. The division, as **allocate_cases** returned it, from the workflow state, passed by the same caller.

**allocation_summary** — **dict**. Its summary, as **summarise_allocation** returned it, from the workflow state, passed by the same caller.

**product_name** — **str**. The product's **product_name**, from its profile, passed by the same caller.

**PURPOSE**

Work out what a decision confirms, holds and leaves in stock.

**REQUIREMENTS**

- **config.DECISION_ESCALATE** confirms nothing: **cases_confirmed**, **notice_lines** and **channel_totals** are empty, **cases_released**, **customers_receiving_cases** and **reserve_cases** are nought, and **cases_in_stock** is every case available.
- A release confirms each customer its cases under that release in **cases_confirmed**, and **notice_lines** gives each customer, both keyed by account code and holding every customer of the division, **config.NOTICE_LINE_FORMAT** filled with its name, its cases confirmed, its cases ordered and **product_name**; **cases_released**, **reserve_cases** and **cases_in_stock** are that release's, and **customers_receiving_cases** counts the customers confirmed at least one case.
- **channel_totals** gives, in alphabetical order of channel name, each channel with at least **config.SALES_BRIEFING_MINIMUM_CHANNEL_CUSTOMERS** customers its **customers** and **cases_released**, every customer of the channel counted whether or not it receives a case.
- A value that is not one of **config.DECISIONS** is refused, and a decision the allocation does not leave open is refused with **config.DECISION_NOT_OPEN_MESSAGE**, naming its reasons.

**RETURNS**

**dict**. **decision**, **cases_confirmed**, **notice_lines**, **cases_released**, **customers_receiving_cases**, **reserve_cases**, **cases_in_stock** and **channel_totals**.

**FUNCTION SIGNATURE**

```
rank_shortage_queue(queue_entries)
```

**PARAMETERS**

**queue_entries** — **list[dict]**. One entry per product on allocation, each holding at least **product_code** and **forecast_shortfall**, built by the service at start-up, which passes them.

**PURPOSE**

Order the shortage queue.

**REQUIREMENTS**

- The entries are ordered as **THE FORECAST SHORTFALL, AND THE ORDER OF THE QUEUE** sets out, each returned unchanged.
- A product code that appears more than once is refused.

**RETURNS**

**list[dict]**. The same entries, in queue order.

**MODULE: tools/retrieval_tools.py**

Reads the passage an agent quotes from the one document that states it.

**FUNCTION SIGNATURE**

```
read_planner_checks(collection, request_type)
```

**PARAMETERS**

**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index**, passed by **review_customer_requests_node**.

**request_type** — **str**. A request the emails make, from the product's request facts, passed by the same caller.

**PURPOSE**

Read what the planner checks for a request, from its own guide.

**REQUIREMENTS**

- The passage is the one of the request type's guide, named by **config.REQUEST_GUIDE_DOCUMENTS**, nearest to **config.PLANNER_CHECKS_QUESTION**; a request type with no configured guide is refused.

**RETURNS**

**str**. The passage text.

**FUNCTION SIGNATURE**

```
read_customer_advice(collection, product_category)
```

**PARAMETERS**

**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, passed by **draft_allocation_notice_node**.

**product_category** — **str**. The product's category, from its profile, passed by the same caller.

**PURPOSE**

Read the policy's advice to customers.

**REQUIREMENTS**

- The passage is the one of the category's policy, named by **config.CATEGORY_POLICY_DOCUMENTS**, nearest to **config.CUSTOMER_ADVICE_QUESTION**; a category with no configured policy is refused with **config.NO_POLICY_CONFIGURED_MESSAGE**.

**RETURNS**
**str**. The passage text.

**FUNCTION SIGNATURE**
`read_sales_briefing_guidance(collection, product_category)`

**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, passed by **draft_sales_briefing_node**.
**product_category** — **str**. The product's category, from its profile, passed by the same caller.
**PURPOSE**
Read how the policy says the sales team is briefed.
**REQUIREMENTS**

- The passage is the one of the category's policy nearest to **config.SALES_BRIEFING_QUESTION**; a category with no configured policy is refused with **config.NO_POLICY_CONFIGURED_MESSAGE**.

**RETURNS**
**str**. The passage text.

## ========== PERSISTENCE ==========

Six tables hold the accounts, runs, reviews, decisions, the agents' language model calls and chat questions. Apart from the workflow's checkpoint, nothing else is written.

**MODULE: persistence/database.py**
The connection and the session factory: two module-level objects built on first import and shared by the whole project. It defines no function.
**MODULE-LEVEL OBJECTS**

- **database_engine** — **sqlalchemy.engine.Engine**. The connection to the database at **config.APPLICATION_DATABASE_URL**, whose connections the web framework may use from any thread.
- **session_factory** — **sqlalchemy.orm.sessionmaker**. Bound to that engine, and the only way the rest of the project opens a session.

**REQUIREMENTS**

- Both are created when the module is imported, and no other module creates an engine.
- Importing this module neither creates tables nor writes anything; creating the tables is the database setup entry point's work.

**MODULE: persistence/models.py**
Declares the six tables. Reproduce table and column names exactly, in lower case.

**CLASS**
`Base`

The declarative base every table below inherits from, and the object the setup entry point uses to create them all.

**TABLE**
**user_accounts**
**CLASS**
`UserAccount`
**COLUMNS**

| Column | Type | Notes |
|---|---|---|
| username | **str** | The key |
| display_name | **str** | The name a page shows |
| role | **str** | One of the three roles |
| password_hash | **str** | Never the password itself |
| password_salt | **str** | This account's own salt |
| session_token | **str \| None** | The current session, or nothing when signed out |
| created_at | **datetime.datetime** | Set by the database when the row is written |

**TABLE**
**agent_runs**
**CLASS**
`AgentRun`
**COLUMNS**

| Column | Type | Notes |
|---|---|---|
| run_id | **str** | The key, **config.RUN_ID_PREFIX** followed by **config.IDENTIFIER_HEX_LENGTH** upper-case hexadecimal characters |
| thread_id | **str** | The identifier the paused workflow is stored under |
| product_code | **str** | The product under review |
| planner_id | **str** | The username of whoever opened the review |
| planner_request | **str** | What they asked to be looked at |
| status | **str** | One of **config.AGENT_RUN_STATUS_RUNNING**, **config.AGENT_RUN_STATUS_AWAITING_DECISION**, **config.AGENT_RUN_STATUS_COMPLETE** and **config.AGENT_RUN_STATUS_FAILED** |
| llm_call_count | **int** | The calls recorded against the run, written when it reaches the gate, completes or fails, and nought before then |
| started_at | **datetime.datetime** | Set by the database when the row is written |
| finished_at | **datetime.datetime \| None** | Set when the run completes or fails, and nothing while it runs or waits |

**TABLE**
**allocation_reviews**

**CLASS**
`AllocationReview`

**COLUMNS**

| Column | Type | Notes |
|---|---|---|
| review_id | **str** | The key, **config.REVIEW_ID_PREFIX** followed by **config.IDENTIFIER_HEX_LENGTH** upper-case hexadecimal characters |
| run_id | **str** | The run that produced it |
| product_code, product_name, product_category | **str** | The product's own facts, from its profile |
| policy_document | **str** | The policy its cases were divided under |
| planner_request | **str** | What the planner asked |
| cases_available, cases_ordered, cases_eligible | **int** | The totals, as **summarise_allocation** gave them |
| cover_percent | **float \| None** | The cover; nothing where no customer may have any case |
| protected_served_in_full | **bool** | Whether every protected customer gets its eligible cases under **RELEASE** |
| allocation_lines | **list** | Every customer's line, as **allocate_cases** gave it, stored as JSON |
| releases | **dict** | Each release's summary, as **summarise_allocation** gave it, stored as JSON |
| decisions_open | **list** | The decisions the allocation leaves open, stored as JSON |
| decisions_not_open | **dict** | Each closed decision's reasons, stored as JSON |
| inflated_order_scores | **list** | Every order's score and band, as **build_inflated_order_facts** gave them, stored as JSON |
| email_requests | **list** | Every email read, as **build_request_facts** gave them, stored as JSON |
| customer_requests | **dict** | Each customer's most urgent request, stored as JSON |
| forecast_shortfall | **int** | The product's forecast shortfall |
| review_plan_text | **str** | The planner's fields, each written as its label, a colon, a space and its text, joined by blank lines |
| recommendation_text | **str** | The writer's recommendation |
| recommended_decision | **str** | The writer's recommended decision, one of the decisions open |
| status | **str** | **config.REVIEW_STATUS_PENDING** until decided, then the status the decision leaves |
| cases_confirmed | **dict \| None** | Each customer's cases confirmed, keyed by account code, as **compute_allocation_outcome** gave them, stored as JSON; empty on an escalation, and nothing until decided |
| reserve_cases, cases_in_stock | **int \| None** | The cases the decision holds in reserve and leaves in stock; nothing until decided |
| notice_text | **str \| None** | The notice's fields written the same way as the plan, or nothing when no notice was drafted |
| notice_lines | **dict \| None** | Each customer's notice line, keyed by account code, stored as JSON; empty on an escalation, and nothing until decided |
| briefing_text | **str \| None** | The briefing's fields written the same way as the plan; nothing until decided |
| created_at | **datetime.datetime** | Set by the database when the row is written |
| status_changed_at | **datetime.datetime** | Set when the row is written, and set again when the decision is recorded |

**TABLE**
**allocation_decisions**

CLASS
`AllocationDecision`

COLUMNS

| Column | Type | Notes |
|---|---|---|
| decision_id | **int** | The key, numbered by the database |
| review_id | **str** | The review decided |
| run_id | **str** | Its run |
| decision | **str** | One of **config.DECISIONS** |
| decided_by | **str** | The display name of the Supply Planning Manager who took it |
| decided_by_role | **str** | Their role |
| decision_reason | **str** | Their reason, in their own words |
| decided_at | **datetime.datetime** | Set by the database when the row is written |

TABLE
**llm_calls**
CLASS
`LlmCall`

COLUMNS

| Column | Type | Notes |
|---|---|---|
| call_id | **int** | The key, numbered by the database |
| run_id | **str** | The run the call belongs to |
| agent_name | **str** | The agent that made it, one of the names in config |
| model_name | **str** | The model that answered |
| prompt_characters | **int** | The prompt's length |
| response_characters | **int** | The reply's length |
| latency_milliseconds | **int** | How long the call took |
| called_at | **datetime.datetime** | Set by the database when the row is written |

TABLE
**chat_turns**
CLASS
`ChatTurn`

COLUMNS

| Column | Type | Notes |
|---|---|---|
| turn_id | **int** | The key, numbered by the database |
| username | **str** | Who asked |
| user_role | **str** | Their role |
| question_text | **str** | What they asked, every delivery postcode replaced by **config.CHAT_POSTCODE_WITHHELD_TEXT** |
| question_type | **str** | One of **config.CHAT_TYPE_OPTIONS** |
| account_code | **str \| None** | The account the question named, canonical where it could be read, or nothing |
| product_code | **str \| None** | The product code the question named, or nothing |
| answer_text | **str** | The answer given |
| source_documents | **str** | The documents the answer was given from, joined by a comma and a space; empty when none |
| closest_distance | **float** | The distance of the nearest passage found |
| was_refused | **bool** | Whether the question was refused |
| asked_at | **datetime.datetime** | Set by the database when the row is written |

**MODULE: persistence/repository.py**
Every read and write against the six tables; nothing else builds a query, and every row is returned as a plain record, not a database object. Every function takes an open **session** from **session_factory** and writes and flushes but never commits; the caller

commits.

**FUNCTION SIGNATURE**
`insert_user_account(session, account_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **run_database_setup.py** from **session_factory**.
**account_record** — **dict**. One account's columns other than **session_token** and **created_at**, built by **run_database_setup.py** from **config.SEED_USER_ACCOUNTS**, with the password already in its stored form.
**PURPOSE**
Create one sign-in account.
**REQUIREMENTS**
- The row is written and flushed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`fetch_user_account(session, condition)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**condition** — **sqlalchemy.sql.elements.ColumnElement**. A condition on the **user_accounts** columns, written by the caller: on **username** by **sign_in_user** and **run_database_setup.py**, on **session_token** by **authorize_request** and **sign_out_user**.
**PURPOSE**
Find one account.
**REQUIREMENTS**
- The record carries every column of the account, and nothing is returned when no account matches.

**RETURNS**
**dict | None**. The account's row, keyed by column name, or **None**.

**FUNCTION SIGNATURE**
`update_session_token(session, username, session_token)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**username** — **str**. The account whose session changes, passed by **sign_in_user** or **sign_out_user**.
**session_token** — **str | None**. The new token, or **None** to end the session, passed by the same caller.
**PURPOSE**
Start or end an account's session.
**REQUIREMENTS**
- The account holds the token given, replacing any earlier one, so signing in again ends the session before.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`insert_agent_run(session, run_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **load_shortage_context_node** from **session_factory**.
**run_record** — **dict**. **run_id**, **thread_id**, **product_code**, **planner_id**, **planner_request** and **status**, built by **load_shortage_context_node**.
**PURPOSE**
Record that a run has started.
**REQUIREMENTS**
- The row is written and flushed, with a call count of nought.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`update_agent_run(session, run_id, status_value, llm_call_count)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**run_id** — **str**. The run, from the workflow state or the paused workflow's stored state, passed by **compose_allocation_recommendation_node**, **record_allocation_outcome_node** or **report_failed_request**.
**status_value** — **str**. The run's new status, one of the run statuses in config, passed by the same caller.
**llm_call_count** — **int**. The calls the run has recorded, passed by the same caller.
**PURPOSE**
Move a run on to its next status.
**REQUIREMENTS**
- The run holds the status and the call count given, and **finished_at** is set when the status is one of **config.AGENT_RUN_FINISHED_STATUSES**.
- A run that does not exist is refused.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`insert_llm_call(session, run_id, call_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the node that made the call from **session_factory**.
**run_id** — **str**. The run the call belongs to, from the workflow state, passed by that node.
**call_record** — **dict**. **agent_name**, **model_name**, **prompt_characters**, **response_characters** and **latency_milliseconds**, as **call_language_model** returned them, passed by the same node.
**PURPOSE**
Log what one language model call cost.
**REQUIREMENTS**
- One row is written and flushed for the call.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`fetch_llm_calls(session, run_id)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **get_call_budget** or **report_failed_request** from **session_factory**.
**run_id** — **str**. The run whose calls are wanted, from the review's record or the paused workflow's stored state, passed by the same caller.
**PURPOSE**
Read back every call a run made.
**REQUIREMENTS**
- Every call of the run is returned, in the order the calls were made, each with its **agent_name**, **model_name**, **prompt_characters**, **response_characters** and **latency_milliseconds**.

**RETURNS**
**list[dict]**. The run's calls.

**FUNCTION SIGNATURE**
`insert_allocation_review(session, review_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **compose_allocation_recommendation_node** from **session_factory**.
**review_record** — **dict**. Every column of **allocation_reviews** that is known at the gate, built by **compose_allocation_recommendation_node**.
**PURPOSE**
Record the finished review, waiting for its decision.
**REQUIREMENTS**
- The row is written and flushed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`fetch_review_records(session, condition)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**condition** — **sqlalchemy.sql.elements.ColumnElement**. A condition on the **allocation_reviews** columns, written by the caller: on **review_id**, **product_code** or **status** by the service's endpoints, or one that every row meets by **answer_allocation_status**.
**PURPOSE**
Read reviews back with the run and the decision behind each one.
**REQUIREMENTS**
- Each record holds every column of the review, the run's **thread_id**, **planner_id** and **llm_call_count**, and the decision's **decision**, **decided_by**, **decided_by_role** and **decision_reason**, those four being **None** until it is decided.
- Records are returned newest first: by **created_at**, then by **review_id**, both descending.

**RETURNS**
**list[dict]**. The matching records.

**FUNCTION SIGNATURE**
`insert_allocation_decision(session, decision_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **record_allocation_outcome_node** from **session_factory**.

**decision_record** — **dict**. **review_id**, **run_id**, **decision**, **decided_by**, **decided_by_role** and **decision_reason**, built by **record_allocation_outcome_node**.
**PURPOSE**
Record a Supply Planning Manager's decision, who took it and why.
**REQUIREMENTS**
- The row is written and flushed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`record_allocation_outcome(session, review_id, outcome_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **record_allocation_outcome_node** from **session_factory**.
**review_id** — **str**. The review decided, from the workflow state, passed by **record_allocation_outcome_node**.
**outcome_record** — **dict**. **status**, **cases_confirmed**, **reserve_cases**, **cases_in_stock**, **notice_text**, **notice_lines** and **briefing_text**, built by **record_allocation_outcome_node**.
**PURPOSE**
Close a review with what its decision confirms, holds and leaves in stock, and what customers and the sales team were told.
**REQUIREMENTS**
- Exactly the one review holds the values given, and its **status_changed_at** is set again.
- A review that does not exist is refused.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`insert_chat_turn(session, turn_record)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **ask_allocation_assistant** from **session_factory**.
**turn_record** — **dict**. Every column of **chat_turns** but **turn_id** and **asked_at**, built by **ask_allocation_assistant**.
**PURPOSE**
Record one chat question and its answer.
**REQUIREMENTS**
- The row is written and flushed.

**RETURNS**
**None**.

**FUNCTION SIGNATURE**
`fetch_chat_turns(session, condition, turn_limit)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by the caller from **session_factory**.
**condition** — **sqlalchemy.sql.elements.ColumnElement**. A condition on the **chat_turns** columns, written by the caller: on **username** by **recall_conversation** and **list_own_chat_turns**, or one that every row meets by **list_recent_chat_turns**.
**turn_limit** — **int**. The most turns to return, passed by the same caller.
**PURPOSE**
Read back the latest chat turns.
**REQUIREMENTS**
- At most **turn_limit** matching turns are returned, newest first, each with every column of **chat_turns**.

**RETURNS**
**list[dict]**. The turns.

## ========== AGENTIC WORKFLOW ==========

Five agents review a product on allocation, and one or two more write what the decision calls for. Each agent is one language model call and decides no case; every agent that writes facts is told in its prompt to use only the facts it is given.
The workflow stops before any customer is told or any case confirmed, and the pause is checkpointed to disk.

### MODULE: agentic/state.py

Declares what the workflow carries from one node to the next. Every key a node reads must be present before that node runs, and no key may exist that nothing reads.

**CLASS**
`AllocationReviewState`

A **typing.TypedDict** holding exactly the keys below; every node and the router read it as a plain dictionary.
**STATE KEYS**

| Key | Type | Written by | What it carries |
|---|---|---|---|
| product_code | **str** | the caller | The product under review |
| planner_request | **str** | the caller | What the person who opened the review asked to be looked at |
| planner_id | **str** | the caller | The username of whoever opened the review |
| thread_id | **str** | the caller | The identifier the pause is stored under |
| run_id | **str** | **load_shortage_context_node** | The run, as recorded in **agent_runs** |
| review_id | **str** | **load_shortage_context_node** | The review this run will write |
| shortage_profile | **dict** | **load_shortage_context_node** | The product's own facts |
| open_orders | **list** | **load_shortage_context_node** | Its open orders, one record per order, as read |
| customer_accounts | **dict** | **load_shortage_context_node** | Its customers' names and channels |
| demand_facts | **dict** | **load_shortage_context_node** | **inflated_order_facts** and **customer_forecasts**: what the inflated-order and demand models established |
| request_facts | **dict** | **load_shortage_context_node** | What the request model established |
| allocation_facts | **dict** | **compute_allocation_node** | **policy_rules**, **allocation** and **summary**: the policy read, its cap as a **float**, the division and its summary |
| review_plan | **dict** | **plan_allocation_review_node** | The planning agent's reply |
| demand_analysis | **dict** | **analyse_demand_node** | The demand agent's reply |
| request_review | **dict** | **review_customer_requests_node** | The request agent's reply |
| allocation_advice | **dict** | **compute_allocation_node** | The advisor's reply |
| allocation_decision | **dict** | the gate | The decision, who took it and why: **decision**, one of **config.DECISIONS**; **decided_by**, the person's display name; **decided_by_role**, their role; and **decision_reason**, their own words |
| allocation_outcome | **dict** | the gate | What the decision confirms, holds and leaves in stock |
| allocation_notice | **dict** | the gate and **draft_allocation_notice_node** | The notice, empty until one is written |
| sales_briefing | **dict** | **draft_sales_briefing_node** | The briefing |
| llm_call_count | **int** | every agent node | The calls the run has made, added up across nodes rather than replaced |

**REQUIREMENTS**

- Every key above is declared, and nothing else is.
- **llm_call_count** accumulates across nodes, each agent node adding the one call it made. Every other key is replaced by whichever node writes it.

**MODULE: agentic/agents.py**

The language model client, the reply parser and the seven prompts. Each prompt gives an agent established facts and asks for a reply in labelled fields. Every prompt speaks for **config.ORGANISATION_NAME**, and an evidence field an order leaves blank is written as **config.NOT_RECORDED_TEXT**.

The key and the model name are read from the environment through **config.GEMINI_API_KEY_VARIABLE** and **config.GEMINI_MODEL_VARIABLE**, loaded from **config.ENV_FILE** on import; neither is written into the code.

**MODULE-LEVEL OBJECTS**

- **generation_settings** — **google.genai.types.GenerateContentConfig**. The one generation configuration every call uses, holding **config.GEMINI_MAX_OUTPUT_TOKENS** and **config.GEMINI_TEMPERATURE**, with the client's own function calling turned off.

**FUNCTION SIGNATURE**

```
call_language_model(prompt_text, agent_name)
```

**PARAMETERS**

**prompt_text** — **str**. The whole prompt, assembled by the agent function calling this, or by **answer_from_documents**.
**agent_name** — **str**. Which agent is calling, passed by that function from its name constant in config, for example **config.AGENT_NAME_PLANNER**.

**PURPOSE**
Make one language model call and return the reply with what the call cost.
**REQUIREMENTS**

- The key and the model name are read at the moment of the call; when either is missing the call is refused, naming both settings. The reply is generated under **generation_settings**.
- An empty reply, or a reply that did not finish normally, is refused, naming the agent and the model.
- **call_record** carries **agent_name**, **model_name**, **prompt_characters**, **response_characters** and **latency_milliseconds**, the last measured around the model call itself.

**RETURNS**
**dict**. **response_text**, the reply with surrounding space removed, and **call_record**.

**FUNCTION SIGNATURE**
```
parse_labelled_response(response_text, field_labels, agent_name)
```
**PARAMETERS**
**response_text** — **str**. What the model replied, taken by the calling agent function, or by **answer_from_documents**, from the **response_text** that **call_language_model** returned.
**field_labels** — **list[str]**. The labels the prompt asked for, in order, written by that same caller.
**agent_name** — **str**. Which agent replied, passed by that same caller from its name constant, and used in the refusal when a label is missing.
**PURPOSE**
Read a labelled reply back into its fields.
**REQUIREMENTS**

- A line that starts with one of **field_labels** and a colon starts that field, whatever case the model used, with a hyphen read as a space, and whatever decoration matched by **config.RESPONSE_DECORATION_EDGE_PATTERN**, bold markers, headings and quote marks included, it put around the label.
- The lines after a label, up to the next label, belong to that field, apart from a line left empty once whitespace and the decoration matched by **config.RESPONSE_DECORATION_EDGE_PATTERN** are set aside; text before the first label is ignored; a label written again adds its text to the field rather than replacing it, and the lines a field gathers are joined by line breaks. Bold markers are removed wherever they appear, text on a label's own line carries none of **config.RESPONSE_DECORATION_CHARACTERS** at either end, and no field has surrounding whitespace; a figure keeps its minus sign.
- A label that is missing, or answered with nothing, is refused, naming the agent and the labels; the result holds exactly the labels asked for, in the order asked.

**RETURNS**
**dict**. One entry per label.

**FUNCTION SIGNATURE**
```
plan_allocation_review(shortage_profile, open_orders, customer_accounts, planner_request)
```
**PARAMETERS**
**shortage_profile** — **dict**. The product's own facts, as **fetch_shortage_profile** returned them, passed by **plan_allocation_review_node** from the workflow state.
**open_orders** — **list[dict]**. The product's open orders, one record per order as **fetch_open_orders** read it, passed by the same node.
**customer_accounts** — **dict[str, dict]**. Its customers, as **fetch_customer_accounts** returned them, passed by the same node.
**planner_request** — **str**. What the person who opened the review asked, in their own words, passed by the same node.
**PURPOSE**
Plan what the review must establish.
**REQUIREMENTS**

- The prompt states the product's own facts — its code, name, category, pack, units in a case, cases on a pallet, the cases available this week and next, why it is short and its weeks on allocation — every open order with its customer, channel, cases and route, and the request, and asks for **PRIORITY**, **CHECKS** and **RATIONALE**.
- The agent is told that the policy is read later in the review, and to name no rule, week, cap, protected channel, reserve or figure it was not given. The call is made as **config.AGENT_NAME_PLANNER**.

**RETURNS**
**dict**. **review_plan**, holding the three fields, and **call_record**.

**FUNCTION SIGNATURE**
```
analyse_demand(demand_facts, open_orders, customer_accounts, shortage_profile, review_plan)
```
**PARAMETERS**
**demand_facts** — **dict**. **inflated_order_facts** from **build_inflated_order_facts** and **customer_forecasts** from **forecast_customer_demand**, passed by **analyse_demand_node** from the workflow state.
**open_orders** — **list[dict]**. The product's open orders, passed by the same node.
**customer_accounts** — **dict[str, dict]**. Its customers, passed by the same node.
**shortage_profile** — **dict**. The product's own facts, passed by the same node.
**review_plan** — **dict**. The planning agent's three fields, carried in the workflow state, passed by the same node.
**PURPOSE**
Say which orders look inflated and how far the orders stand above what customers will need.
**REQUIREMENTS**

- The prompt hands the agent every order with its customer, cases, route, score and band and the evidence fields of **config.ORDER_EVIDENCE_FIELDS**, each order matched to its own score by its order line, the band thresholds, every customer's cases ordered against its forecast for next week, and the plan's **CHECKS**.
- The agent is told that the scores and the forecasts are predictions that decide nothing, and not to recalculate or invent any figure.
- The reply carries **INFLATED ORDERS**, **ORDERS AGAINST FORECASTS** and **DEMAND SUMMARY**, and the call is made as **config.AGENT_NAME_DEMAND**.

**RETURNS**
**dict**. **demand_analysis** and **call_record**.

**FUNCTION SIGNATURE**
```
review_customer_requests(request_facts, planner_checks, shortage_profile, customer_accounts, re
```

**PARAMETERS**
**request_facts** — **dict**. What **build_request_facts** established for this product, passed by **review_customer_requests_node** from the workflow state.
**planner_checks** — **dict[str, str]**. Each request type the emails make, most urgent first, against the passage **read_planner_checks** returned for it, gathered by the same node.
**shortage_profile** — **dict**. The product's own facts, passed by the same node.
**customer_accounts** — **dict[str, dict]**. Its customers, passed by the same node.
**review_plan** — **dict**. The planning agent's three fields, passed by the same node.
**PURPOSE**
Weigh what customers are asking for against the request guides.
**REQUIREMENTS**

- The prompt hands the agent every open email with its customer, sender, time, subject, text and the request it was read as, each customer's most urgent request, the order of urgency, each guide's checks with the guide it came from, and the plan's **CHECKS**; a product with no open email is said to have none.
- The agent is told that a request is weighed, never granted, the cases being divided by the policy's rules alone, and to use only the emails, their readings and the guides' passages it is given.
- The reply carries **CUSTOMER REQUESTS**, **CHECKS TO MAKE** and **REQUEST SUMMARY**, and the call is made as **config.AGENT_NAME_REQUESTS**.

**RETURNS**
**dict**. **request_review** and **call_record**.

**FUNCTION SIGNATURE**
```
advise_on_allocation(allocation_facts, shortage_profile, review_plan)
```

**PARAMETERS**
**allocation_facts** — **dict**. **policy_rules**, **allocation** and **summary**, established by **compute_allocation_node** just before it calls this.
**shortage_profile** — **dict**. The product's own facts, passed by the same node.
**review_plan** — **dict**. The planning agent's three fields, passed by the same node.
**PURPOSE**
Say who gets what and why: which customers were protected, which reached their limit, and which got less than they may have.
**REQUIREMENTS**

- The prompt states the policy's own four rule sentences and the document they came from, every customer in account order with its channel, whether it is protected, its cases ordered, its usual demand to four places, its eligible cases and its cases under each release, the decisions open, each closed decision with its reasons, and the plan's **CHECKS**.
- For each release the prompt gives the cases placed, held and left in stock, what the protected customers and the others get against what they may have, written by **config.RELEASE_SPLIT_FORMAT**, or **config.RELEASE_WITHOUT_PROTECTION_FORMAT** where no customer is protected, and the customers short, a release not open marked so; the cover is written by **config.COVER_TEXT_FORMAT**, or **config.NO_COVER_TEXT** where there is none.
- The agent is told to quote each customer's figures exactly as given, never to work out a share and never to move a case from one customer to another.
- The reply carries **PROTECTED CUSTOMERS**, **LIMITS REACHED**, **CUSTOMERS SHORT** and **ALLOCATION SUMMARY**, and the call is made as **config.AGENT_NAME_ADVISOR**.

**RETURNS**
**dict**. **allocation_advice** and **call_record**.

**FUNCTION SIGNATURE**
```
write_allocation_recommendation(shortage_profile, planner_request, allocation_facts, forecast_s
```

**PARAMETERS**
**shortage_profile** — **dict**. The product's own facts, passed by **compose_allocation_recommendation_node** from the workflow state.
**planner_request** — **str**. What the review was opened to look at, passed by the same node from the workflow state.
**allocation_facts** — **dict**. The allocation facts, passed by the same node from the workflow state.
**forecast_shortfall** — **int**. What **compute_forecast_shortfall** returned, worked out by the same node just before it calls this.
**review_plan** — **dict**. The planning agent's fields, passed by the same node from the workflow state.
**demand_analysis** — **dict**. The demand agent's fields, passed by the same node from the workflow state.
**request_review** — **dict**. The request agent's fields, passed by the same node from the workflow state.

**allocation_advice** — **dict**. The advisor's fields, passed by the same node from the workflow state.
**PURPOSE**
Write the recommendation the Supply Planning Manager decides from, and recommend one decision.
**REQUIREMENTS**

- The prompt states the product, the policy it was divided under, the request, what the planning, demand, request and allocation agents found, the totals, the cover, whether every protected customer is served in full, each release's figures with a release that is not open marked so, and the forecast shortfall with what it means; and it explains the three decisions in the words of **config.DECISION_DESCRIPTIONS** and when each is open, with this product's decisions open.
- The agent is told that it recommends and does not decide, never to change a customer's allocation, to quote every figure exactly, and to answer what the planner asked.
- The reply carries **RECOMMENDATION** and **RECOMMENDED DECISION**, which is returned as one of the decisions open, whatever its case or the decoration at either end; a **RECOMMENDED DECISION** over more than one line, or one that is not open, is refused; the **RECOMMENDATION** may run over several lines.
- The call is made as **config.AGENT_NAME_WRITER**.

**RETURNS**
**dict**. **allocation_recommendation**, holding the two fields, and **call_record**.

**FUNCTION SIGNATURE**
```
draft_allocation_notice(shortage_profile, policy_rules, allocation_outcome, customer_advice, cu
```
**PARAMETERS**
**shortage_profile** — **dict**. The product's own facts, passed by **draft_allocation_notice_node** from the workflow state.
**policy_rules** — **dict**. The policy read, from the allocation facts in the workflow state, passed by the same node.
**allocation_outcome** — **dict**. What **compute_allocation_outcome** returned at the gate, passed by the same node from the workflow state.
**customer_advice** — **str**. The passage **read_customer_advice** returned to the same node from the category's own policy.
**customer_accounts** — **dict[str, dict]**. The product's customers, passed by the same node from the workflow state, so the notice can be checked to name none of them.
**PURPOSE**
Write the allocation notice that goes to every customer that ordered.
**REQUIREMENTS**

- The prompt states the product, its pack and why it is short, the policy's four rule sentences with the document they came from, whether a reserve is held this week, and the advice to customers with the policy it came from — never a customer, a figure for one customer, a score or an email.
- The agent is told to write plainly and courteously, to say how allocations were worked out from the rules and what customers can do from the advice, never to name a customer, give a figure that belongs to one customer or compare customers, and to invent no contact details.
- The reply carries **HEADLINE**, **WHY IT IS SHORT**, **HOW ALLOCATIONS WERE WORKED OUT** and **WHAT CUSTOMERS CAN DO**; a headline over more than one line, and a notice that names any of the product's customers, whatever its case or its spacing, are refused. The call is made as **config.AGENT_NAME_NOTICE**.

**RETURNS**
**dict**. **allocation_notice** and **call_record**.

**FUNCTION SIGNATURE**
```
draft_sales_briefing(shortage_profile, policy_document, allocation_outcome, allocation_notice,
```
**PARAMETERS**
**shortage_profile** — **dict**. The product's own facts, passed by **draft_sales_briefing_node** from the workflow state.
**policy_document** — **str**. The policy's file name, from the allocation facts in the workflow state, passed by the same node.
**allocation_outcome** — **dict**. What **compute_allocation_outcome** returned at the gate, passed by the same node from the workflow state.
**allocation_notice** — **dict**. The notice's fields when one was written, and empty on an escalation, passed by the same node from the workflow state.
**briefing_guidance** — **str**. The passage **read_sales_briefing_guidance** returned to the same node from the category's own policy.
**customer_accounts** — **dict[str, dict]**. The product's customers, passed by the same node from the workflow state, so the briefing can be checked to name none of them.
**PURPOSE**
Write the briefing for the sales team.
**REQUIREMENTS**

- The prompt states the product, the decision with its description, the totals — the cases released and to how many customers, the reserve and the cases in stock — each channel's total from **channel_totals** or that there is none to give, the notice's headline or that no notice was written, and the briefing guidance with the policy it came from.
- The agent is told to quote only the totals given, never a customer's name or any other figure, and to invent no contact details.
- The reply carries **DECISION TAKEN**, **WHAT CUSTOMERS WERE TOLD** and **WHAT TO SAY**; a briefing that names any of the product's customers, whatever its case or its spacing, is refused, and so is a briefing written after a notice that does not quote the notice's **HEADLINE**, compared without its decoration, whatever its case or its spacing. The call is made as **config.AGENT_NAME_BRIEFING**.

**RETURNS**
**dict**. **sales_briefing** and **call_record**.

**MODULE: agentic/nodes.py**
The ten workflow steps and the router out of the gate. Each node takes the state and returns only the keys it wrote. A node that calls an agent records the call against the run, in the same transaction as its other writes, as soon as the agent function returns; a reply the agent function refuses stops the run before its call is recorded.

**MODULE-LEVEL RESOURCES**
Everything below is opened once, when the module is first imported, from the paths in config, and every node and the service use it.

| Name | Type | What it holds |
|---|---|---|
| inflated_order_model, demand_model, request_model | **sklearn.pipeline.Pipeline** | The saved models, loaded by **load_model** |
| inflated_order_feature_names, demand_feature_names | **list[str]** | Their saved schemas, loaded by **load_feature_schema** |
| shortage_products, open_orders, shipment_history, customer_master, open_customer_emails | **pandas.DataFrame** | The five live-queue files, read by **load_raw_dataset** |
| knowledge_collection | **chromadb.api.models.Collection.Collection** | The knowledge base, opened by **open_vector_index** |

**FUNCTION SIGNATURE**
`load_shortage_context_node(state)`

**PARAMETERS**
**state** — **AllocationReviewState**. The workflow's own state, a typed dictionary handed to the node by LangGraph and checkpointed after every node. This node reads **product_code**, **planner_request**, **planner_id** and **thread_id**.
**PURPOSE**
Gather the product's facts, run the three models and open the run.
**REQUIREMENTS**
- A product whose profile, open orders, customers, open emails, demand facts or request facts cannot be read opens no run and writes nothing.
- The run identifier is **config.RUN_ID_PREFIX** and the review identifier **config.REVIEW_ID_PREFIX**, each followed by **config.IDENTIFIER_HEX_LENGTH** random upper-case hexadecimal characters; the run is written to **agent_runs** with status **config.AGENT_RUN_STATUS_RUNNING**.
- Every value placed in the state is a plain value the checkpoint can store.

**RETURNS**
**dict**. **run_id**, **review_id**, **shortage_profile**, **open_orders**, **customer_accounts**, **demand_facts** and **request_facts**.

**FUNCTION SIGNATURE**
`plan_allocation_review_node(state)`

**PARAMETERS**
**state** — **AllocationReviewState**. The workflow's own state. This node reads **shortage_profile**, **open_orders**, **customer_accounts**, **planner_request** and **run_id**.
**PURPOSE**
Ask the planning agent what has to be established.
**REQUIREMENTS**
- The agent is called once, and the call is recorded against the run.

**RETURNS**
**dict**. **review_plan**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**
`analyse_demand_node(state)`

**PARAMETERS**
**state** — **AllocationReviewState**. The workflow's own state. This node reads **demand_facts**, **open_orders**, **customer_accounts**, **shortage_profile**, **review_plan** and **run_id**.
**PURPOSE**
Ask the demand agent to explain the scores and the forecasts.
**REQUIREMENTS**
- The agent is called once, and the call is recorded against the run.

**RETURNS**
**dict**. **demand_analysis**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**
`review_customer_requests_node(state)`

**PARAMETERS**

**state** — **AllocationReviewState**. The workflow's own state. This node reads **request_facts**, **shortage_profile**, **customer_accounts**, **review_plan** and **run_id**.

**PURPOSE**

Ask the request agent what customers are asking for.

**REQUIREMENTS**

- The agent is given the planner's checks from the guide of every request type the emails make, most urgent first; it is called once, and the call is recorded against the run.

**RETURNS**

**dict**. **request_review**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
compute_allocation_node(state)
```

**PARAMETERS**

**state** — **AllocationReviewState**. The workflow's own state. This node reads **shortage_profile**, **product_code**, **customer_accounts**, **open_orders**, **review_plan** and **run_id**.

**PURPOSE**

Divide the product's cases under its category's own policy, then ask the advisor to explain the division.

**REQUIREMENTS**

- **allocation_facts** holds the rules **read_allocation_policy** reads from the category's own policy, the division worked from the product's shipments and its summary, every case worked out with the exact cap, and the cap held as a **float**.
- The agent is called once, and the call is recorded against the run.

**RETURNS**

**dict**. **allocation_facts**, **allocation_advice**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
compose_allocation_recommendation_node(state)
```

**PARAMETERS**

**state** — **AllocationReviewState**. The workflow's own state. This node reads **shortage_profile**, **planner_request**, **demand_facts**, **request_facts**, **allocation_facts**, **review_plan**, **demand_analysis**, **request_review**, **allocation_advice**, **product_code**, **review_id**, **run_id** and **llm_call_count**.

**PURPOSE**

Work out the forecast shortfall, have the recommendation written, and record the review for the Supply Planning Manager.

**REQUIREMENTS**

- The writer is called once, and is given the forecast shortfall.
- In one transaction, the call is recorded, the review is written to **allocation_reviews** with every column known at the gate and status **config.REVIEW_STATUS_PENDING**, and the run moves to **config.AGENT_RUN_STATUS_AWAITING_DECISION** with its call count including this call.

**RETURNS**

**dict**. **llm_call_count** as **1**.

**FUNCTION SIGNATURE**

```
request_allocation_decision_node(state)
```

**PARAMETERS**

**state** — **AllocationReviewState**. The workflow's own state. This node reads **review_id**, **allocation_facts** and **shortage_profile**.

**PURPOSE**

Stop the run until a Supply Planning Manager decides, then work out what the decision confirms.

**REQUIREMENTS**

- The run is paused with the review identifier, and resumes with the Supply Planning Manager's decision record.
- After the decision, the state holds the outcome **compute_allocation_outcome** gives and an empty notice; the node writes nothing and makes no language model call, a paused node running again from its start when the run resumes.

**RETURNS**

**dict**. **allocation_decision**, **allocation_outcome** and **allocation_notice**.

**FUNCTION SIGNATURE**

```
route_allocation_decision(state)
```

**PARAMETERS**

**state** — **AllocationReviewState**. The workflow's own state. This router reads **allocation_decision**.

**PURPOSE**

Send the run down the path its decision calls for.

**REQUIREMENTS**

- The value returned is the decision itself, one of **config.DECISIONS**, and the router changes nothing.

**RETURNS**

**str**. The decision.

**FUNCTION SIGNATURE**

```
draft_allocation_notice_node(state)
```

**PARAMETERS**

**state** — **AllocationReviewState**. The workflow's own state. This node reads **shortage_profile**, **allocation_facts**, **allocation_outcome**, **customer_accounts** and **run_id**.
**PURPOSE**
Have the allocation notice written.
**REQUIREMENTS**

- The agent is given the advice to customers from the category's own policy; it is called once, and the call is recorded against the run.

**RETURNS**
**dict**. **allocation_notice**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**
`draft_sales_briefing_node(state)`

**PARAMETERS**
**state** — **AllocationReviewState**. The workflow's own state. This node reads **shortage_profile**, **allocation_facts**, **allocation_outcome**, **allocation_notice**, **customer_accounts** and **run_id**.
**PURPOSE**
Have the sales briefing written, after the notice on a release and on its own on an escalation.
**REQUIREMENTS**

- The agent is given the briefing guidance from the category's own policy; it is called once, and the call is recorded against the run.

**RETURNS**
**dict**. **sales_briefing**, and **llm_call_count** as **1**.

**FUNCTION SIGNATURE**
`record_allocation_outcome_node(state)`

**PARAMETERS**
**state** — **AllocationReviewState**. The workflow's own state. This node reads **allocation_decision**, **allocation_outcome**, **allocation_notice**, **sales_briefing**, **review_id**, **run_id** and **llm_call_count**.
**PURPOSE**
Close the review with its decision, what it confirms, holds and leaves in stock, and what customers and the sales team were told.
**REQUIREMENTS**

- In one transaction the decision is written to **allocation_decisions**, the review is closed and the run moves to **config.AGENT_RUN_STATUS_COMPLETE** with its call count.
- The review takes the status **config.REVIEW_STATUS_BY_DECISION** gives the decision, the outcome's cases confirmed, reserve, cases in stock and notice lines, the notice written as its labelled fields, each its label, a colon, a space and its text, joined by blank lines, or nothing where none was drafted, and the briefing written the same way.
- No language model is called here.

**RETURNS**
**dict**. Empty: the run is over.

**MODULE: agentic/graph.py**
The order the nodes run in, the gate, the branch and the checkpoint.

**FUNCTION SIGNATURE**
`build_workflow(checkpoint_path)`

**PARAMETERS**
**checkpoint_path** — **str**. The file paused runs are written to, passed by the service module when it loads, from **config.CHECKPOINT_DATABASE_FILE**.
**PURPOSE**
Assemble the ten nodes into a workflow that can be run, paused and resumed.
**REQUIREMENTS**

- The ten nodes are registered under exactly the names above, and the first six run in the order shown, followed by the gate.
- After the gate there is one conditional branch, **route_allocation_decision**, and each of the three values in **config.DECISIONS** leads to its node: both releases to **draft_allocation_notice**, **ESCALATE** to **draft_sales_briefing**. The notice leads to the briefing, the briefing to **record_allocation_outcome**, which is joined to the workflow's **END**, and every node is reachable from the start.
- The compiled workflow is given a checkpointer that writes to **checkpoint_path**, whose connection the web framework may use from any thread, so a run paused at the gate can be resumed in another process, after a restart; building the workflow starts nothing and calls no model.

**RETURNS**
**langgraph.graph.state.CompiledStateGraph**. The compiled workflow, ready to be run and resumed.

**========== THE REST SERVICE ==========**
Every fact the dashboard shows comes from this service; the dashboard holds no business rule and reads no file.

**MODULE: api/schemas.py**

Every request and reply shape, declared once. A request failing its conditions is refused with status **422** by its schema, before any work and before any product or review is looked up, and a reply carries exactly the fields listed. Every facts dictionary holds only **str**, **int**, **float**, **bool**, **None** and lists or dictionaries of them, never a NumPy value, a fraction or a NaN. Reproduce field names exactly.

**MODULE-LEVEL TYPES**

- **CategoryFilter** — **typing.Literal**. The values the queue may be filtered by, from **config.QUEUE_CATEGORY_OPTIONS**: **config.QUEUE_CATEGORY_ALL** for the whole queue, or one of the five categories.
- **ReviewStatusValue** — **typing.Literal**. The values a review's status may take, from **config.REVIEW_STATUS_OPTIONS**.
- **DecisionValue** — **typing.Literal**. The decisions a Supply Planning Manager may send, from **config.DECISIONS**.
- **PlannerRequestText**, **DecisionReasonText** and **QuestionText** — **typing.Annotated** text types, stripped of surrounding whitespace and at least **config.PLANNER_REQUEST_MINIMUM_CHARACTERS**, **config.DECISION_REASON_MINIMUM_CHARACTERS** and **config.CHAT_QUESTION_MINIMUM_CHARACTERS** characters long.

**REQUEST SHAPE**

`SignInRequest`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| username | **str** | The account signing in |
| password | **str** | The password as typed |

**VALIDATION**

- Both fields are required.

**RESPONSE SHAPE**

`SignedInUserResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| username | **str** | The account that signed in |
| display_name | **str** | The person's name as the dashboard shows it |
| role | **str** | The role that account holds |
| session_token | **str** | The token every later request carries in **config.SESSION_TOKEN_HEADER** |
| pages | **list[str]** | The pages this role is offered, from **config.ROLE_PAGE_ACCESS** |
| areas | **list[str]** | The areas this role reaches, from **config.ROLE_AREA_ACCESS** |

**VALIDATION**

None beyond the field types above.

**RESPONSE SHAPE**

`SignedOutResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| username | **str** | The account whose session ended |
| signed_out | **bool** | That the session is over |

**VALIDATION**

None beyond the field types above.

**RESPONSE SHAPE**

`QueueRow`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| queue_position | **int** | The product's place in the queue, from **1** |
| product_code, product_name, product_category | **str** | The product's own facts |
| cases_available_this_week, cases_available_next_week | **int** | Its cases this week and next |
| cases_ordered, cases_eligible | **int** | What its customers ordered and may have, as **summarise_allocation** gives them |
| cover_percent | **float \| None** | Its cover; **None** where no customer may have any case |
| protected_served_in_full | **bool** | Whether every protected customer gets its eligible cases under **RELEASE** |
| decisions_open | **list[str]** | The decisions its allocation leaves open |
| forecast_shortfall | **int** | Its forecast shortfall |

#### VALIDATION

None beyond the field types above.

#### RESPONSE SHAPE

`ShortageDetailResponse`

#### FIELDS

| Field | Type | What it carries |
|---|---|---|
| queue_position | **int** | The product's place in the queue |
| profile | **dict** | The product's own facts, as **fetch_shortage_profile** returns them |
| policy_rules | **dict** | Its category's rules, as **read_allocation_policy** returns them, the cap as a **float** |
| allocation_lines | **list[dict]** | Every customer's line, as **allocate_cases** returns them |
| summary | **dict** | As **summarise_allocation** returns it |
| forecast_shortfall | **int** | Its forecast shortfall |
| order_lines | **list[dict]** | Every open order, holding **order_line_id**, **order_reference**, **order_route**, **account_code**, **customer_name**, **ordered_quantity** in cases, **inflated_order_score** and **inflated_order_band** |
| email_requests | **list[dict]** | Every open email read, as **build_request_facts** returns them |
| customer_requests | **dict** | Each customer's most urgent request |

#### VALIDATION

None beyond the field types above.

#### REQUEST SHAPE

`ReviewRequest`

#### FIELDS

| Field | Type | What it carries |
|---|---|---|
| product_code | **str** | The product to review |
| planner_request | **PlannerRequestText** | What the person opening the review wants looked at |

#### VALIDATION

- **planner_request**, stripped, is at least **config.PLANNER_REQUEST_MINIMUM_CHARACTERS** characters long.

#### REQUEST SHAPE

`DecisionRequest`

#### FIELDS

| Field | Type | What it carries |
|---|---|---|
| decision | **DecisionValue** | One of **config.DECISIONS** |
| decision_reason | **DecisionReasonText** | The Supply Planning Manager's reason, in their own words |

#### VALIDATION

- **decision** is one of **config.DECISIONS**, and **decision_reason**, stripped, is at least **config.DECISION_REASON_MINIMUM_CHARACTERS** characters long.

#### RESPONSE SHAPE

`ReviewRecordResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| every column of **allocation_reviews** but **created_at** and **status_changed_at** | as in the table | The review as stored |
| thread_id, planner_id | **str** | From its run |
| llm_call_count | **int** | The calls its run has recorded |
| decision, decided_by, decided_by_role, decision_reason | **str** \| **None** | From its decision, **None** until decided |

**VALIDATION**

None beyond the field types above.

**RESPONSE SHAPE**

`CallBudgetResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| review_id | **str** | The review |
| run_id | **str** | Its run |
| call_count | **int** | The calls the run has recorded |
| expected_call_count | **int** | **config.LLM_CALLS_BEFORE_DECISION** until decided, then **config.LLM_CALLS_BY_DECISION** for the decision taken |
| total_latency_milliseconds | **int** | The recorded calls' latencies added together |
| calls | **list[dict]** | Every recorded call, in the order made, as **fetch_llm_calls** returns them |

**VALIDATION**

None beyond the field types above.

**REQUEST SHAPE**

`ChatRequest`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| question | **QuestionText** | The question asked |

**VALIDATION**

- **question**, stripped, is at least **config.CHAT_QUESTION_MINIMUM_CHARACTERS** characters long.

**RESPONSE SHAPE**

`ChatResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| question_type | **str** | One of **config.CHAT_TYPE_OPTIONS** |
| account_code | **str** \| **None** | The account the question named, canonical where it could be read, or **None** |
| product_code | **str** \| **None** | The product code the question named, or **None** |
| answer | **str** | The answer given |
| source_documents | **list[str]** | The documents the answer was given from, empty for a refusal or an allocation status answer |
| was_refused | **bool** | Whether the question was refused |

**VALIDATION**

None beyond the field types above.

**RESPONSE SHAPE**

`ChatTurnResponse`

**FIELDS**

| Field | Type | What it carries |
|---|---|---|
| every column of **chat_turns** but **asked_at** | as in the table | One stored turn |

**VALIDATION**

None beyond the field types above.

**MODULE: api/routes.py**
**PURPOSE**
Serve the eleven endpoints in one application, all but signing in and out behind the sign-in and access guard, with one failure handler.
**MODULE-LEVEL OBJECTS**

- **review_workflow** — **langgraph.graph.state.CompiledStateGraph**. The workflow, built once by **build_workflow** from **config.CHECKPOINT_DATABASE_FILE**.
- **policy_rules_by_category** — **dict[str, dict]**. Every category in **config.CATEGORY_POLICY_DOCUMENTS** against its rules, read once by **read_allocation_policy**.
- **shortage_details** — **dict[str, dict]**. Every live product, keyed by its code, against its **profile**, **product_orders**, **customer_accounts**, **policy_rules** with the cap as a **float**, **allocation_lines**, **summary** and **forecast_shortfall**.
- **queue_entries** — **list[dict]**. One entry per live product, holding a queue row's fields but its position.
- **shortage_queue** — **list[dict]**. The entries in the order **rank_shortage_queue** gives them, each with its **queue_position** from **1**.
- **queue_positions** — **dict[str, int]**. Each product's queue position, keyed by its code.
- **review_start_locks** — **dict[str, threading.Lock]**. One lock per product: two simultaneous starts of one product open one run.
- **decision_lock** — **threading.Lock**. Held while a decision is checked and recorded: two simultaneous decisions record one.
- **public_router** — **fastapi.APIRouter**. Signing in and signing out, reachable without a token.
- **guarded_router** — **fastapi.APIRouter**. Every other endpoint, behind the guard.
- **SessionTokenHeader** — a **typing.Annotated** alias for **str | None**. The optional header **config.SESSION_TOKEN_HEADER**; a request without it is answered **401**, not **422**.
- **SignedInUser** — a **typing.Annotated** alias for **dict**. The endpoint parameter type that receives the account **authorize_request** returned.
- **service_application** — **fastapi.FastAPI**. The application, titled **config.API_TITLE**, holding both routers, the guarded one behind **authorize_request**, and **report_failed_request** as its failure handler.

**REQUIREMENTS**

- The models, the live queue and the knowledge base are taken from the workflow's nodes module, not loaded again. Every product's facts above are worked out once, at start-up, with the same tools the workflow uses, so no review can stop half-way on a policy or a shipment history it cannot read.
- No endpoint carries a default parameter value except the optional session token header. Every endpoint answers **200** when it succeeds, except **POST /reviews**, which answers **201**.
- Only **POST /reviews**, **POST /reviews/{review_id}/decision** and **POST /chat/ask** may cost language model calls.

**sign_in_user**
**ENDPOINT**
**POST /auth/signin**
**FUNCTION SIGNATURE**
`sign_in_user(sign_in_request)`
**PARAMETERS**
**sign_in_request** — **SignInRequest**. The username and password posted, read from the request body by FastAPI.
**PURPOSE**
Exchange a username and password for a session token.
**REQUIREMENTS**

- An unknown username and a wrong password are answered identically, with **401** and **config.SIGN_IN_FAILED_MESSAGE**; the password is checked against the stored form with the account's own salt, compared in constant time.
- A correct password issues a fresh token made from **config.SESSION_TOKEN_BYTES** random bytes and stores it against the account, replacing any token it held.

**RETURNS**
**dict**. A **SignedInUserResponse**.

**sign_out_user**
**ENDPOINT**
**POST /auth/signout**
**FUNCTION SIGNATURE**
`sign_out_user(session_token)`
**PARAMETERS**
**session_token** — **str | None**. The token to end, declared as **SessionTokenHeader** and taken by FastAPI from the header **config.SESSION_TOKEN_HEADER**, and **None** when the header is absent.
**PURPOSE**
End a session.
**REQUIREMENTS**

- The account's token is cleared and cannot be used again; a request with no token, or a token no account holds, is answered **401** with **config.SIGN_IN_REQUIRED_MESSAGE**.

**RETURNS**
**dict**. A **SignedOutResponse**.

**list_shortage_queue**
**ENDPOINT**
**GET /queue**
**FUNCTION SIGNATURE**
`list_shortage_queue(product_category)`

**PARAMETERS**
**product_category** — **str**. One of **config.QUEUE_CATEGORY_OPTIONS**, declared as **CategoryFilter** and taken by FastAPI from the query string.
**PURPOSE**
Give the desk its ranked list of products on allocation.
**REQUIREMENTS**

- **config.QUEUE_CATEGORY_ALL** gives every live product once; a category gives exactly the rows of that category, in the same order and with the same positions; any other value is answered **422**.

**RETURNS**
**list[dict]**. One **QueueRow** per product.

**get_shortage_detail**
**ENDPOINT**
**GET /queue/{product_code}**
**FUNCTION SIGNATURE**
`get_shortage_detail(product_code)`

**PARAMETERS**
**product_code** — **str**. The product, taken by FastAPI from the path.
**PURPOSE**
Show one product with every fact the review works from.
**REQUIREMENTS**

- A product not in the queue is answered **404** with **config.PRODUCT_NOT_IN_QUEUE_MESSAGE** filled with the code.
- The facts come from the same tools, policy and division the workflow uses, and agree with its queue row; every order is scored and every open email read, and no language model call is made.

**RETURNS**
**dict**. A **ShortageDetailResponse**.

**start_allocation_review**
**ENDPOINT**
**POST /reviews**
**FUNCTION SIGNATURE**
`start_allocation_review(review_request, signed_in_user, request)`

**PARAMETERS**
**review_request** — **ReviewRequest**. The product and what is to be looked at, read from the request body by FastAPI.
**signed_in_user** — **dict**. The account of whoever is opening the review, declared as **SignedInUser** and supplied by FastAPI from **authorize_request**.
**request** — **fastapi.Request**. The request itself, supplied by FastAPI, through which **report_failed_request** learns the thread of a run that fails.
**PURPOSE**
Run the five reviewing agents on one product and leave it waiting for a Supply Planning Manager.
**REQUIREMENTS**

- A product not in the queue is answered **404** with **config.PRODUCT_NOT_IN_QUEUE_MESSAGE**, and a product that already has a review, whatever its status, **409** with **config.PRODUCT_ALREADY_REVIEWED_MESSAGE**, each filled with the code; neither opens a run.
- The run's thread is the product code, a hyphen and **config.IDENTIFIER_HEX_LENGTH** random upper-case hexadecimal characters, known to **report_failed_request** through **request** even when the workflow fails; the caller's username is the run's **planner_id**, and the run stops at the gate after **config.LLM_CALLS_BEFORE_DECISION** calls.

**RETURNS**
**dict**. A **ReviewRecordResponse** for the review written, with status **config.REVIEW_STATUS_PENDING**.

**list_allocation_reviews**
**ENDPOINT**
**GET /reviews**
**FUNCTION SIGNATURE**
`list_allocation_reviews(status_value)`

**PARAMETERS**
**status_value** — **list[str]**. One or more of **config.REVIEW_STATUS_OPTIONS**, declared as a list of **ReviewStatusValue** and taken by FastAPI from the query string, where it may be repeated.
**PURPOSE**
List reviews by status, for the decision queue, the audit trail and the allocation detail page.
**REQUIREMENTS**

- Every review whose status is one of the values given is returned, newest first, and no other.

**RETURNS**
**list[dict]**. One **ReviewRecordResponse** per review.

**submit_allocation_decision**
**ENDPOINT**
**POST /reviews/{review_id}/decision**
**FUNCTION SIGNATURE**
`submit_allocation_decision(review_id, decision_request, signed_in_user)`

**PARAMETERS**
**review_id** — **str**. The review being decided, taken by FastAPI from the path.
**decision_request** — **DecisionRequest**. The decision and the reason, read from the request body by FastAPI.
**signed_in_user** — **dict**. The Supply Planning Manager deciding, declared as **SignedInUser** and supplied by FastAPI from **authorize_request**.
**PURPOSE**
Record a Supply Planning Manager's decision and finish the run down the path it calls for.
**REQUIREMENTS**

- In this order of precedence: an unknown review is answered **404** with **config.REVIEW_NOT_FOUND_MESSAGE**, a decision by the person who opened it **403** with **config.OWN_REVIEW_REFUSED_MESSAGE**, a review already decided **409** with **config.ALREADY_DECIDED_MESSAGE**, and a decision the allocation does not leave open **409** with **config.DECISION_NOT_OPEN_MESSAGE**, naming its reasons; every refusal is settled under **decision_lock**, before the run resumes, so none costs a call.
- The run resumes with **decision**, **decided_by** as the Supply Planning Manager's display name, **decided_by_role** and **decision_reason**.
- A decision that failed while the notice or the briefing was being drafted can be finished only by the same decision, person and reason; any other open decision, person or reason is answered **409** with **config.DECISION_IN_PROGRESS_MESSAGE**, filled with the decision already sent and the display name of the Supply Planning Manager who sent it.

**RETURNS**
**dict**. A **ReviewRecordResponse** for the review, as decided.

**get_call_budget**
**ENDPOINT**
**GET /reviews/{review_id}/calls**
**FUNCTION SIGNATURE**
`get_call_budget(review_id)`

**PARAMETERS**
**review_id** — **str**. The review, taken by FastAPI from the path.
**PURPOSE**
Show the run's language model calls against the expected count.
**REQUIREMENTS**

- An unknown review is answered **404** with **config.REVIEW_NOT_FOUND_MESSAGE**.

**RETURNS**
**dict**. A **CallBudgetResponse**.

**ask_allocation_assistant**
**ENDPOINT**
**POST /chat/ask**
**FUNCTION SIGNATURE**
`ask_allocation_assistant(chat_request, signed_in_user)`

**PARAMETERS**
**chat_request** — **ChatRequest**. The question, read from the request body by FastAPI.
**signed_in_user** — **dict**. The account asking, declared as **SignedInUser** and supplied by FastAPI from **authorize_request**.
**PURPOSE**
Answer one question from the documents, or about one account or product from its records, and remember the exchange.
**REQUIREMENTS**

- No delivery postcode matched by **config.POSTCODE_PATTERN** reaches the memory, the prompt or the stored turn: each is replaced by **config.CHAT_POSTCODE_WITHHELD_TEXT**.
- Each question costs exactly one language model call, made through **answer_from_documents** with the asker's own recalled conversation; a **config.CHAT_TYPE_ALLOCATION_STATUS** answer is the one **answer_allocation_status** gives the person asking, with the postcode the question gave when it gave exactly one, postcodes compared without spaces and whatever their case.
- The turn is stored in **chat_turns** with the asker, their role, the question as withheld, its type, the account and product named, the answer, the source documents joined by a comma and a space, the closest distance and whether it was refused.

**RETURNS**
**dict**. A **ChatResponse**.

**list_own_chat_turns**
**ENDPOINT**

**GET /chat/turns**
**FUNCTION SIGNATURE**
`list_own_chat_turns(signed_in_user, turn_limit)`

**PARAMETERS**
**signed_in_user** — **dict**. The account asking, declared as **SignedInUser** and supplied by FastAPI from **authorize_request**.
**turn_limit** — **int**. How many turns to return, taken by FastAPI from the query string, between **1** and **config.CHAT_AUDIT_MAX_ROWS**.
**PURPOSE**
Show a person their own latest chat turns.
**REQUIREMENTS**
- Only the asker's own turns are returned, newest first, at most **turn_limit** of them.

**RETURNS**
**list[dict]**. One **ChatTurnResponse** per turn.

**list_recent_chat_turns**
**ENDPOINT**
**GET /audit/chat-turns**
**FUNCTION SIGNATURE**
`list_recent_chat_turns(turn_limit)`

**PARAMETERS**
**turn_limit** — **int**. How many turns to return, taken by FastAPI from the query string, between **1** and **config.CHAT_AUDIT_MAX_ROWS**.
**PURPOSE**
List everyone's latest chat turns.
**REQUIREMENTS**
- The latest turns of everyone are returned, newest first, at most **turn_limit** of them.

**RETURNS**
**list[dict]**. One **ChatTurnResponse** per turn.

**report_failed_request**
**FUNCTION SIGNATURE**
`report_failed_request(request, failure)`

**PARAMETERS**
**request** — **fastapi.Request**. The request that failed, supplied by FastAPI.
**failure** — **Exception**. What went wrong: a **ValueError** from the project's own code, a **google.genai.errors.APIError** from the language model client, or an **httpx.HTTPError** from its HTTP connection.
**PURPOSE**
Turn every failure into one plain reply, and close the run a failed request opened.
**REQUIREMENTS**
- It is registered for **ValueError**, **google.genai.errors.APIError** and **httpx.HTTPError**.
- When the failed request had opened a run, that run moves to **config.AGENT_RUN_STATUS_FAILED** with the calls it recorded.
- The reply is **502** with a **detail** naming the method, the path and the failure.

**RETURNS**
**fastapi.responses.JSONResponse**. The failure reply.

## ========== ACCESS CONTROL ==========
Everyone signs in, and the service, not the dashboard, decides what each role may reach, as **THE THREE ROLES** sets out.

**MODULE: access/password_security.py**
Turns a password and its salt into the form the database stores.

**FUNCTION SIGNATURE**
`hash_password(plain_password, password_salt)`

**PARAMETERS**
**plain_password** — **str**. The password as typed, passed by **sign_in_user** from the sign-in body, or by **run_database_setup.py** from **config.SEED_USER_ACCOUNTS**.
**password_salt** — **str**. The account's own salt, read by **sign_in_user** from its **user_accounts** row, or made by **run_database_setup.py** from **config.PASSWORD_SALT_BYTES** random bytes, written as hexadecimal, when the account is seeded.
**PURPOSE**
Turn a password into the form the database stores.
**REQUIREMENTS**
- The stored form is a key derived with **config.PASSWORD_HASH_ALGORITHM** over **config.PASSWORD_HASH_ROUNDS** rounds, salted with **password_salt**, and returned as hexadecimal text.
- The same password and salt always give the same result, and a different salt gives a different result.

**RETURNS**
**str**. The stored form.

**MODULE: access/permissions.py**
The sign-in and access guard every protected endpoint sits behind.

**FUNCTION SIGNATURE**
`authorize_request(request, session_token)`

**PARAMETERS**
**request** — **fastapi.Request**. The incoming request, supplied by FastAPI and read for the route it matched and its method.
**session_token** — **str | None**. The token sent under **config.SESSION_TOKEN_HEADER**, taken by FastAPI from the header with a default of **None** when the header is absent.
**PURPOSE**
Decide, for every guarded request, who is asking and whether their role reaches that area.
**REQUIREMENTS**

- A request with no token, or a token no account holds, is refused **401** with **config.SIGN_IN_REQUIRED_MESSAGE**.
- The area is the one **config.AREA_BY_ROUTE** gives the request's method and the path template of the route it matched, never the address as written.
- A role that does not reach that area in **config.ROLE_AREA_ACCESS** is refused **403** with **config.AREA_REFUSED_MESSAGE**, naming the role and the area, before the endpoint runs and whether or not what it asks for exists.

**RETURNS**
**dict**. The signed-in person's account record.

**THE ALLOCATION CHAT**
One question, one language model call, answered only from the documents the chat may read. The assistant classifies the question, answers the kinds it may answer and refuses the rest. A question about one account's or one product's allocation is answered by the service from the records, by the asker's role, as **THE IDENTITY LINE** sets out.

| Kind | What it covers | Answered from |
|---|---|---|
| **config.CHAT_TYPE_POLICY_RULES** | What a category's policy says: the weeks of shipments behind usual demand, the cap on an order, the protected customers, the reserve, the advice to customers and what the sales team is told | The five policies, each read whole, and the passages found |
| **config.CHAT_TYPE_ALLOCATION_RULES** | How allocation works: account codes and quantities in each route's notation, an order sent twice, usual demand and the silent week, what a customer may have, the division and the largest remainder, the decisions and who takes them, the shortage queue | The documents the chat may read |
| **config.CHAT_TYPE_REQUEST_HANDLING** | What the request guides say about a customer's email: how a request shows up, what the planner checks, the evidence to ask for, what to tell the customer and when to escalate | The documents the chat may read |
| **config.CHAT_TYPE_ALLOCATION_STATUS** | How many cases one account or one product is getting, whether its allocation is under review, released or escalated, or anything else about one account's or one product's allocation | The records, by the service |
| **config.CHAT_TYPE_OUT_OF_SCOPE** | Anything the documents do not answer | Nothing: it is refused |

**MODULE: chat/conversation_memory.py**
Recalls the last few exchanges of a person's own conversation.

**FUNCTION SIGNATURE**
`recall_conversation(session, username, question_text)`

**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **ask_allocation_assistant** from **session_factory**.
**username** — **str**. Whose conversation to recall, taken by **ask_allocation_assistant** from the signed-in account.
**question_text** — **str**. The question just asked, its postcodes withheld, passed by **ask_allocation_assistant**.
**PURPOSE**
Recall the person's recent exchanges as history and as search text.
**REQUIREMENTS**

- At most **config.CHAT_HISTORY_TURN_LIMIT** of that person's own most recent turns are recalled, and nobody else's.
- **history_text** shows them oldest first, each exchange with its question and its answer, an allocation status answer replaced by **config.CHAT_STATUS_ANSWER_IN_HISTORY**; with no earlier turns it says this is the first question.

- **search_text** is the question asked just before, when there is one, followed by the new question.

**RETURNS**
**dict**. **history_text** and **search_text**.

**MODULE: chat/allocation_assistant.py**
Settles what kind of question was asked and answers it from the documents the chat may read, in one language model call.

**FUNCTION SIGNATURE**
```
answer_from_documents(collection, question_text, conversation)
```
**PARAMETERS**
**collection** — **chromadb.api.models.Collection.Collection**. The knowledge base, opened once by **open_vector_index** at start-up in the workflow's nodes module, passed by **ask_allocation_assistant**.
**question_text** — **str**. The question as written, its postcodes withheld, passed by **ask_allocation_assistant**.
**conversation** — **dict**. **history_text** and **search_text**, as **recall_conversation** returned them to **ask_allocation_assistant**.
**PURPOSE**
Answer a question from the documents the chat may read in one language model call, refusing when they do not ground an answer.
**REQUIREMENTS**

- The prompt gives the five policies, each read whole, the passages **search_documents** finds for the question and for **search_text**, each once, closest first, and the conversation; it tells the model that a question leaving out its subject continues the last exchange, that a postcode has been withheld and is never written, and that a policy's rule is given with the policy named.
- The prompt names the kinds in **config.CHAT_TYPE_OPTIONS** and tells the model to answer only from what it is given, to quote a category's weeks of history, cap, protected customers or reserve only from that category's own policy, and that it knows nothing about any account's or product's allocation; the call is made as **config.AGENT_NAME_ALLOCATION_ASSISTANT**.
- The reply carries **QUESTION TYPE**, **ACCOUNT**, as the question writes it, **PRODUCT CODE** and **ANSWER**, with **config.CHAT_NO_VALUE_REPLY** where one does not apply, each recognised whatever its case and the decoration at either end. A type, account or product code that is not a single line, a type not among the five, and an answerable type whose **ANSWER** is **config.CHAT_NO_VALUE_REPLY**, are refused with a **ValueError**, not answered as **was_refused**.
- An answer is given only when the type is one of **config.CHAT_ANSWERED_TYPES** and the nearest passage is within **config.CHAT_MAX_RELEVANCE_DISTANCE**; otherwise **was_refused** is true, the type becomes **config.CHAT_TYPE_OUT_OF_SCOPE** and the answer **config.CHAT_REFUSAL_MESSAGE**, except that a **config.CHAT_TYPE_ALLOCATION_STATUS** question keeps its type and the model's own **ANSWER** and is not marked refused.
- **account_code** is the account in canonical form where one of the notations of **config.ACCOUNT_CODE_NOTATIONS** reads it, whatever its case, otherwise the account as written less the decoration at either end, and **None** for **config.CHAT_NO_VALUE_REPLY**. **product_code** is the code in upper case where it matches **config.PRODUCT_CODE_PATTERN**, and **None** otherwise.
- **source_documents** names once each the documents of the passages found, closest first, after the five policies for a **config.CHAT_TYPE_POLICY_RULES** answer, and is empty unless an answer was given; **closest_distance** is the nearest passage's distance.

**RETURNS**
**dict**. **question_type**, **account_code**, **product_code**, **answer**, **source_documents**, **closest_distance** and **was_refused**.

**MODULE: chat/allocation_status_answer.py**
Answers a question about one account's or one product's allocation from the records, in the way the asker's role allows.

**FUNCTION SIGNATURE**
```
answer_allocation_status(session, signed_in_user, customer_master, account_code, postcode, prod
```
**PARAMETERS**
**session** — **sqlalchemy.orm.Session**. An open database session, opened by **ask_allocation_assistant** from **session_factory**.
**signed_in_user** — **dict**. The asker's account record, as **authorize_request** returned it, passed by **ask_allocation_assistant**; its **role** is read.
**customer_master** — **pandas.DataFrame**. The live customer master, loaded once at start-up in the workflow's nodes module, passed by **ask_allocation_assistant**.
**account_code** — **str** | **None**. The account **answer_from_documents** read from the model's **ACCOUNT** field, or **None**, passed by **ask_allocation_assistant**.
**postcode** — **str** | **None**. The one delivery postcode the question gave, taken out of it by **ask_allocation_assistant**, or **None** where it gave none or more than one.
**product_code** — **str** | **None**. The product code **answer_from_documents** read from the model's **PRODUCT CODE** field, or **None**, passed by **ask_allocation_assistant**.
**PURPOSE**
Answer a question about one allocation from the records, across the identity line.
**REQUIREMENTS**

- A **config.ROLE_ADVISER** hears the account's line for each reviewed product it ordered, its cases ordered being all those of that product, only where the postcode given is the one **customer_master** records for that account, compared without spaces and whatever its case, under a heading naming the customer as recorded there; otherwise, and where no account is named, the exact words **THE IDENTITY LINE** gives.
- Any other role hears every review of the product or account named, as **THE IDENTITY LINE** sets out, the subject being **config.CHAT_SUBJECT_PRODUCT_TEMPLATE**, **config.CHAT_SUBJECT_ACCOUNT_TEMPLATE**, or both joined by **and**, product first; where an account is named, each review shows that account's line alone and a review without it is left out; each status is written by **config.REVIEW_STATUS_LABELS** and the decisions open are joined by a comma and a space; while a review waits, a customer's cases are those of each release still open, its name, a space and its cases grouped by thousands, joined by a semicolon and a space.
- Reviews come newest first, narrowed to **product_code** where one is given; every other value is filled in as stored, and the lines are joined by line breaks.
- No language model call is made.

**RETURNS**
**str**. The answer.

**============== THE DASHBOARD ==============**
Five pages over the service, named by **config.PAGE_QUEUE**, **config.PAGE_DETAIL**, **config.PAGE_DECISION**, **config.PAGE_AUDIT** and **config.PAGE_CHAT**. Every allocation fact on a page comes from the service. A page checks only the service's input limits, a decision on one's own review and whether a product is already reviewed. Each page is a function that draws itself and returns nothing.
Where a page badges an inflated-order band, a decision, a status or a question type, the badge takes its colour from **config.INFLATED_ORDER_BAND_BADGE_COLOURS**, **config.DECISION_BADGE_COLOURS**, **config.STATUS_BADGE_COLOURS** and **config.CHAT_TYPE_BADGE_COLOURS**. The division is drawn with **config.DIVISION_FILL_COLOURS**, **config.DIVISION_LINE_COLOURS**, **config.DIVISION_TEXT_COLOUR** and **config.DIVISION_EDGE_COLOUR**, and each customer's cases as **config.ALLOCATION_CHART_SERIES** in **config.ALLOCATION_CHART_COLOURS**. Statuses are shown with **config.REVIEW_STATUS_LABELS**. The theme and the port are pre-loaded in **.streamlit/config.toml**.

**MODULE: dashboard/api_client.py**
Every request the dashboard sends passes through this module; no page holds an address, a timeout or a session token. The one exception is **run_dashboard.py**, which checks that something is listening at **config.API_BASE_URL** before any page is drawn.

**FUNCTION SIGNATURE**
```
call_service(method, path, query, payload, waiting_message)
```
**PARAMETERS**
**method** — **str**. The HTTP method, written by the page making the call, or by **run_dashboard.py** to sign in and to sign out, for example **GET** or **POST**.
**path** — **str**. The path beneath **config.API_BASE_URL**, written by the same caller, for example **/queue** or **/auth/signin**.
**query** — **dict | None**. Query parameters built by the calling page from what the reader chose, or **None**, the default, where the endpoint takes none.
**payload** — **dict | None**. The request body built by the same caller from what the reader typed, or **None**, the default, where the endpoint takes none.
**waiting_message** — **str | None**. What the page tells the reader while a call that makes language model calls runs, written by the calling page, or **None**, the default, for a call that returns at once.
**PURPOSE**
Make one call to the service and return its reply.
**REQUIREMENTS**
- The address is built from **config.API_BASE_URL**, the call is given **config.API_REQUEST_TIMEOUT_SECONDS**, and the token travels under **config.SESSION_TOKEN_HEADER** whenever there is a session; while the request runs, a spinner shows **waiting_message** with the time elapsed.
- A **401** while signed in ends the dashboard's session and returns the reader to the sign-in form with **config.SESSION_ENDED_MESSAGE**.
- Any other reply with status **400** or more is shown as the service's own **detail**, or as the status and the text when the reply is not JSON, and the page stops there.

**RETURNS**
**dict | list**. The service's reply read from JSON.

**MODULE: dashboard/shortage_queue.py**
The shortage queue page.

**FUNCTION SIGNATURE**
```
render_shortage_queue_page()
```
**PARAMETERS**
None. The page reads from the service through **call_service** and from the browser session.
**PURPOSE**

Show the ranked queue and let the reader pick a product.

**REQUIREMENTS**

- Counts along the top, from the whole queue, give the products on allocation, those with **RELEASE** open, those with **ESCALATE** alone open and the cases available this week.
- The reader filters by one of **config.QUEUE_CATEGORY_OPTIONS**, and the service does the filtering and the ordering. The table shows each product's position, code, name, category, cases this week, cases ordered, cases its customers may have, cover, whether every protected customer is served in full, decisions open and forecast shortfall.
- Selecting a row remembers that product for the detail page.

**RETURNS**

**None**. The page draws itself.

**MODULE: dashboard/allocation_detail.py**

The allocation detail page, the division drawn release by release, and the form that opens a review.

**FUNCTION SIGNATURE**

`render_allocation_detail_page()`

**PARAMETERS**

None. The page reads from the service through **call_service** and from the browser session.

**PURPOSE**

Show one product's allocation and start its review.

**REQUIREMENTS**

- The product is chosen in queue order, opening on the one the queue page remembered, and everything shown comes from its detail and from the reviews of every status in **config.REVIEW_STATUS_OPTIONS** that name it.
- The headline shows the category, the decisions open, the queue position and any review status, the pack and its case and pallet sizes, the reason for the shortage and the policy, with the cases available, ordered and eligible, the cover or **config.COVER_NOT_WORKED_OUT_TEXT**, and the forecast shortfall.
- The division is drawn for the release the reader chooses, **RELEASE** at first, with **config.DECISION_NOT_OPEN_MESSAGE** where it is not open: the cases available split between the protected customers, the customers sharing by usual demand, the reserve and the stock left, and each customer's cases against what it may have.
- The page also shows the four rules as read, under **config.POLICY_RULE_LABELS**, beside the policy's own sentences; every customer's line with its usual week to **config.USUAL_DEMAND_DECIMALS** places, its cases under each release and its most urgent request; every open order with its route, cases, and inflated-order score to **config.INFLATED_ORDER_SCORE_DECIMALS** places and band, counted by band; and every open email read.
- The review form takes a request of at least **config.PLANNER_REQUEST_MINIMUM_CHARACTERS**; starting it calls the service once with a waiting message and shows the review waiting for a Supply Planning Manager, with its recommended decision. A product already reviewed shows its review's identifier, who opened it and the recommendation instead.

**RETURNS**

**None**. The page draws itself.

**MODULE: dashboard/decision_queue.py**

The decision queue page and the decision form.

**FUNCTION SIGNATURE**

`render_decision_queue_page()`

**PARAMETERS**

None. The page reads from the service through **call_service** and from the browser session.

**PURPOSE**

Show the reviews waiting at the gate and let a Supply Planning Manager decide one.

**REQUIREMENTS**

- Only reviews at **config.REVIEW_STATUS_PENDING** are listed, and the chosen one is shown in full: its recommended decision and policy, the cases available, what its customers may have, the cover and the forecast shortfall, the recommendation, every customer's line with its cases under each release, each decision not open with its reasons, who opened it, what they asked and the plan.
- Where the signed-in person opened the review, the page says that another Supply Planning Manager must decide it, in place of the form.
- The form offers exactly the decisions open and requires a reason of at least **config.DECISION_REASON_MINIMUM_CHARACTERS**; recording it calls the service once with a waiting message, and the page reports the status reached, the cases released, the cases held in reserve and the cases left in stock.

**RETURNS**

**None**. The page draws itself.

**MODULE: dashboard/audit_trail.py**

The audit trail page: reviews, the record behind each one, the notice and the briefing, the model calls and the chat log.

**FUNCTION SIGNATURE**

`render_audit_trail_page()`

**PARAMETERS**
None. The page reads from the service through **call_service** and from the browser session.
**PURPOSE**
Show the reviews, the record behind each and the chat log.
**REQUIREMENTS**

- Reviews are listed by the statuses the reader chooses from **config.REVIEW_STATUS_OPTIONS**, **config.DECIDED_REVIEW_STATUSES** chosen at first, each with its product, category, status, decision and who took it, cases released and model calls.
- Selecting one shows who opened it and its recommended decision, the decision's reason, the cases held in reserve and left in stock, the recommendation, the allocation notice with its lines and the sales briefing where drafted, and its model calls against the calls expected, with the total latency and every call.
- A second tab shows the most recent **config.CHAT_AUDIT_MAX_ROWS** chat turns from everyone, newest first, with who asked, their role, each question, its type, the account and product it named, whether it was refused and its sources.

**RETURNS**
**None**. The page draws itself.

**MODULE: dashboard/allocation_chat.py**
The allocation chat page.

**FUNCTION SIGNATURE**
`render_allocation_chat_page()`

**PARAMETERS**
None. The page reads from the service through **call_service** and from the browser session.
**PURPOSE**
Ask the allocation assistant a question and show the conversation.
**REQUIREMENTS**

- The conversation is the signed-in person's own turns, oldest at the top, at most **config.CHAT_PAGE_TURN_LIMIT** of them, each answer with its type and with the documents it drew on.
- The question box says what may be asked: a policy, the allocation rules, a request guide, or an allocation by its account code and delivery postcode.
- A question shorter than **config.CHAT_QUESTION_MINIMUM_CHARACTERS** is not sent; asking calls the service once, with a waiting message.

**RETURNS**
**None**. The page draws itself.

## ========== THE SIX ENTRY POINTS ==========

The first four entry points build the system and run once, in order; the last two start the service and the dashboard. Each prints its progress.

**MODULE: run_database_setup.py**
Creates the database, its tables and the accounts people sign in with. It must be safe to run again on a machine that already has a database.

**FUNCTION SIGNATURE**
`main()`

**PARAMETERS**
None.
**PURPOSE**
Create the database, its tables and the seed accounts.
**REQUIREMENTS**

- The folder **config.DATABASE_DIRECTORY** and every table exist when the run finishes, and a second run neither fails nor duplicates anything; an account already present is left exactly as it is.
- Each account in **config.SEED_USER_ACCOUNTS** can sign in afterwards with the username, password and role given there, stored with its own random salt of **config.PASSWORD_SALT_BYTES** bytes, written as hexadecimal, and the hashed form of its password.
- The console names **config.APPLICATION_DATABASE_FILE**, the tables, each account as it is created, and how many accounts were created and how many were already present.

**RETURNS**
**None**.
**MAIN BLOCK**

- Running the file performs the setup once; importing the module performs none of it.

**MODULE: run_data_pipeline.py**

Cleans all five raw extracts and writes the quality report. The module defines one module-level constant, **DATASET_JOBS**, a **list[tuple]** of the five cleaning runs in order, each pairing a raw extract's path and its cleaned file's path in config with the function that cleans it, its answer column and the function that profiles its details.

**FUNCTION SIGNATURE**
`main()`

**PARAMETERS**
None.
**PURPOSE**
Clean the five extracts and write the quality report.
**REQUIREMENTS**

- All five jobs run in order, and each extract is profiled as it arrived and as cleaned.
- Each extract's profile is its general and detail profiles merged into one flat dictionary, filed under the raw file's name without **.csv**, and the report is written to **config.DATA_QUALITY_REPORT_FILE**.
- One console line per extract says how many rows were received, kept and removed, and a closing line names the report.

**RETURNS**
**None**.
**MAIN BLOCK**

- Running the file cleans all five extracts and writes the report; importing the module cleans nothing.

**MODULE: run_model_training.py**
Trains and saves all three models and reports whether each reaches the minimum config sets for it.

**FUNCTION SIGNATURE**
`main()`

**PARAMETERS**
None.
**PURPOSE**
Produce the three saved models, their schemas and their scores.
**REQUIREMENTS**

- The inflated-order and demand models are fitted on their cleaned training extracts and scored on their cleaned held-out extracts, on inputs from **build_feature_frame**; the inflated-order answer is encoded by **config.ORDER_TARGET_ENCODING**.
- Each schema saved is exactly the columns its model was fitted on, in order.
- The cleaned emails are split once, **config.EMAIL_HELD_OUT_SHARE** held out, stratified by request type, with **config.RANDOM_SEED**, and the request model is fitted on the training part only.
- The three models and two schemas are saved to the paths in config, and **config.MODEL_METRICS_FILE** records under **inflated_order**, **demand** and **request** the **algorithm**, the class name of the estimator, or for the request model of every step of its pipeline joined by a space, a plus sign and a space, the **training_rows** and **held_out_rows**, the **scores**, the **minimum_required**, whether it **meets_minimum** and the **scikit_learn_version**.
- The console gives each model's held-out score, training score and gap, and closes by saying either that every model reaches its minimum or which do not; a model below its minimum is reported, not raised.

**RETURNS**
**None**.
**MAIN BLOCK**

- Running the file trains, saves and scores all three models; importing the module trains nothing.

**MODULE: run_knowledge_indexing.py**
Builds the knowledge base and reads every category's rules back from its own policy.

**FUNCTION SIGNATURE**
`main()`

**PARAMETERS**
None.
**PURPOSE**
Build the searchable store and read every policy back from it.
**REQUIREMENTS**

- The store at **config.VECTOR_STORE_DIRECTORY** holds every passage of the sixteen documents, and is rebuilt from scratch on every run; the console says how many documents were read, how many sections were found, how many passages were stored and where the store is.
- Every category's rules, read back from the reopened store, are printed with the policy they came from: the weeks of history, the cap, the protected channels or that nobody is protected, and the reserve or that none is held.
- A policy that cannot be read stops the run with its refusal rather than reporting success.

**RETURNS**
**None**.
**MAIN BLOCK**

- Running the file builds the store and reads it back; importing the module builds nothing.

**MODULE: run_api_service.py**
Starts the REST service after checking that everything it needs is on disk. The module defines one module-level constant, **REQUIRED_BEFORE_STARTING**, a **list[tuple[str, str]]** pairing each path the service needs with what produces it: the three models and two schemas, the five live-queue files, the knowledge base's **config.VECTOR_STORE_FILE_NAME** and the database.

**FUNCTION SIGNATURE**
`main()`

**PARAMETERS**
None.
**PURPOSE**
Start the service, or name what is missing.
**REQUIREMENTS**

- Every path in **REQUIRED_BEFORE_STARTING** is checked before anything is imported from the service; where something is missing, the console says the service cannot start and names each missing path and what produces it.
- Otherwise the service starts on **config.API_HOST** and **config.API_PORT**, **8081**, and the console says where the interactive documentation is.

**RETURNS**
**int**. Nought when the service ran, **1** when it could not start.
**MAIN BLOCK**

- Running the file starts the service and makes **main**'s result the process's exit status; importing the module starts nothing.

**MODULE: run_dashboard.py**
The dashboard: signing in, the navigation, and the five pages. The module defines one module-level constant, **PAGE_RENDERERS**, a **dict[str, tuple]** pairing each page name in config with the function that draws it and the icon it is shown with.

**FUNCTION SIGNATURE**
`main()`

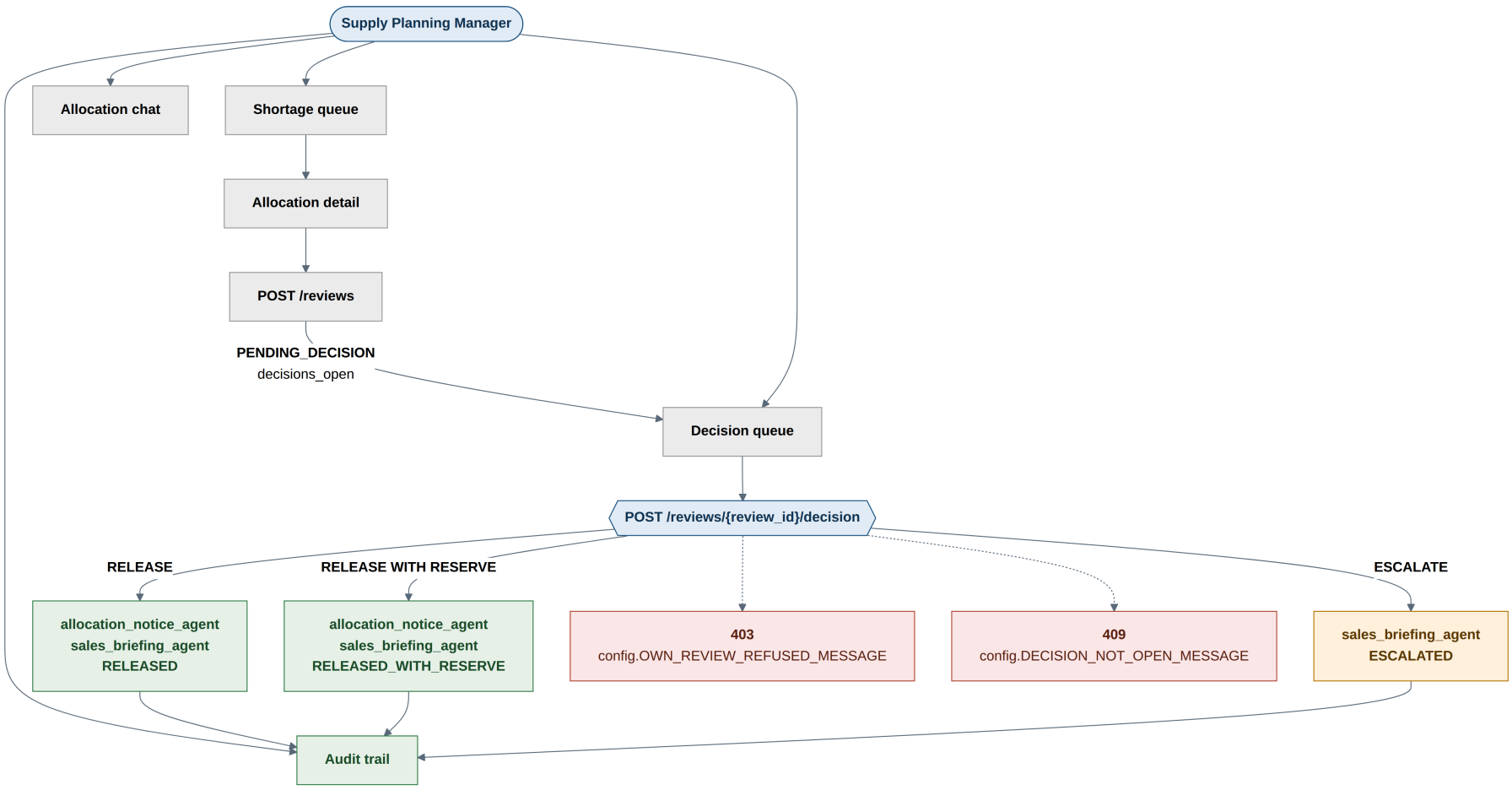
**PARAMETERS**
None.
**PURPOSE**
Draw the dashboard with only the pages the signed-in role is offered.
**REQUIREMENTS**

- The page is titled **config.DASHBOARD_TITLE**, and a service that is not answering at **config.API_BASE_URL** is reported as such, naming the address and how to start it.
- Before anyone signs in, the page shows only the sign-in form, under **config.DASHBOARD_TITLE** and **config.DASHBOARD_SUBTITLE**, and a refused sign-in shows the service's own message; after signing in, the person's name and role are shown, with a way to sign out that ends the session at the service.
- The pages offered are exactly the ones the service returned for that role, in that order, each drawn by the function **PAGE_RENDERERS** pairs with it.

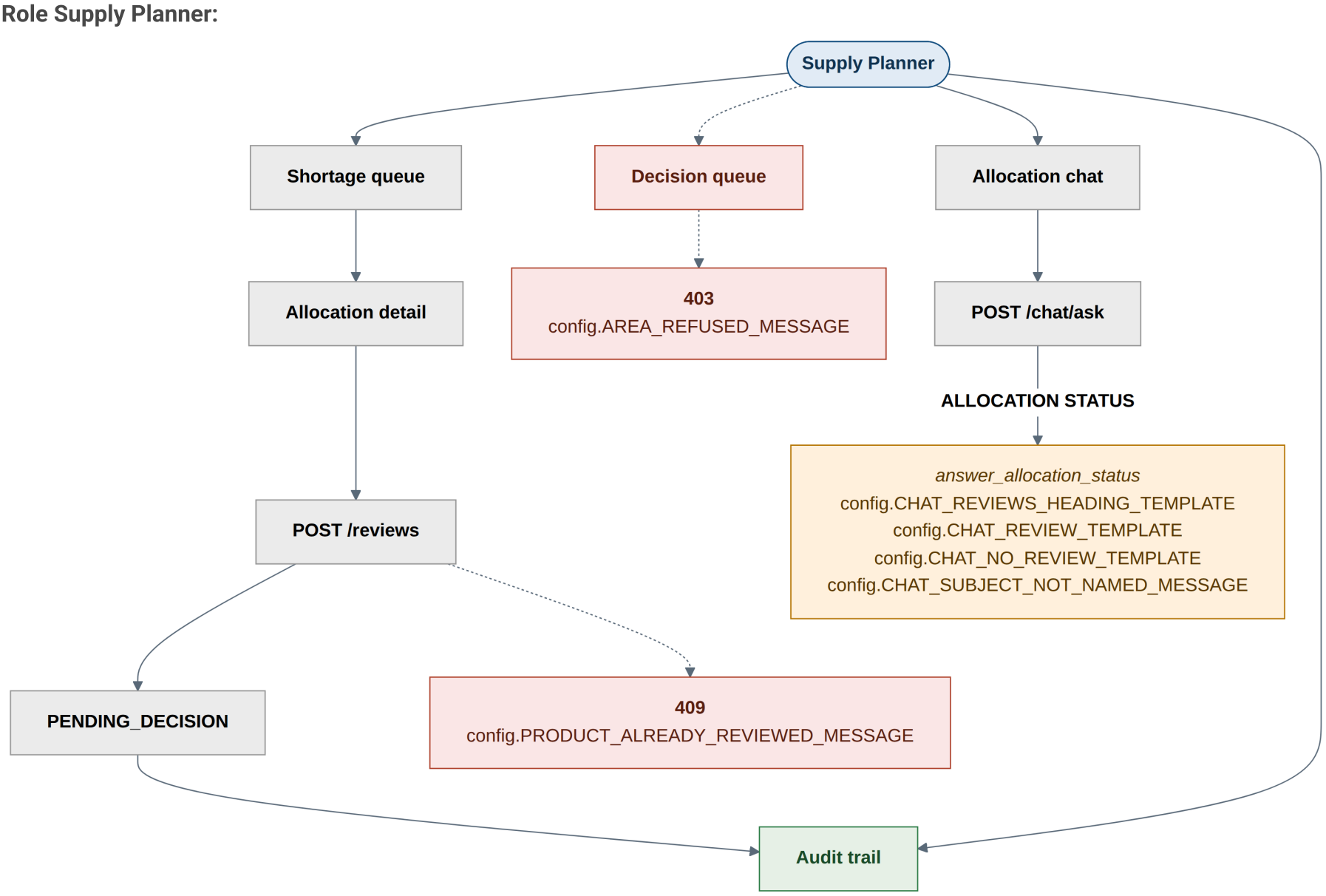
**RETURNS**
**None**. The dashboard draws itself.
**MAIN BLOCK**

- Running the file through Streamlit draws the dashboard on port **8080**; importing the module draws nothing.

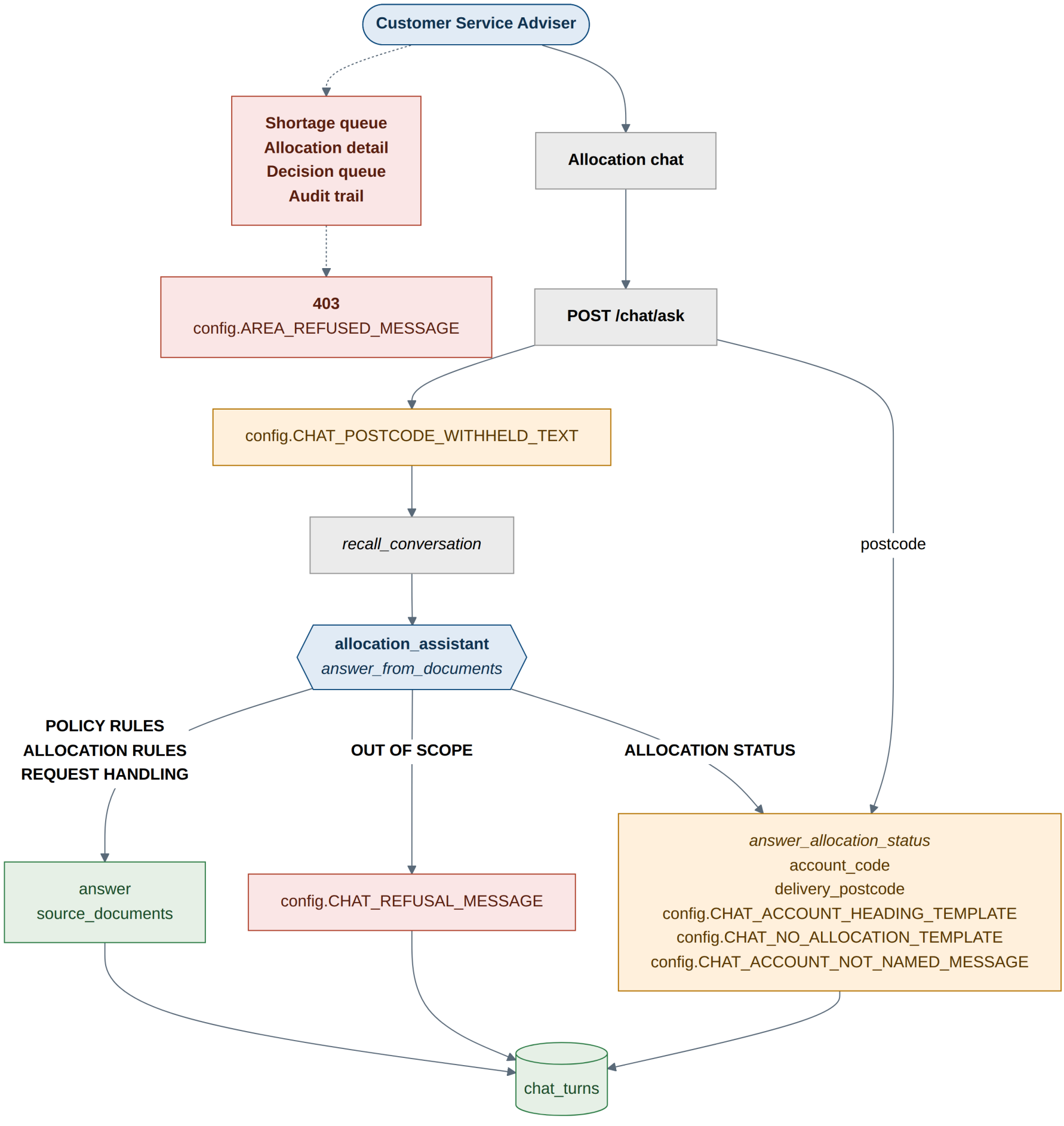
### Role Supply Planning Manager:




## Role Supply Planner:




## Role Consumer Service Adviser:




**Agent to Agent Communication Flow:**

load_shortage_context

*fetch_shortage_profile*
*fetch_open_orders*
*fetch_customer_accounts*
*fetch_open_emails*

*build_inflated_order_facts*
*forecast_customer_demand*
*build_request_facts*

Allocation chat

**shortage_intake_planner**
*plan_allocation_review*

review_plan

**demand_analyst**
*analyse_demand*

**allocation_assistant**
*answer_from_documents*

chat_turns

*read_planner_checks*

planner_checks

**customer_request_analyst**
*review_customer_requests*

demand_analysis

request_review

compute_allocation

*read_allocation_policy*
*fetch_shipment_history*
*compute_usual_demand*
*allocate_cases*
*summarise_allocation*

allocation_facts

**allocation_advisor**
*advise_on_allocation*

allocation_facts
allocation_advice

compose_allocation_recommendation

*compute_forecast_shortfall*

forecast_shortfall

**allocation_writer**
*write_allocation_recommendation*

**PENDING_DECISION**

**request_allocation_decision**
**Supply Planning Manager**
*compute_allocation_outcome*

*read_customer_advice*

customer_advice

**RELEASE**
**RELEASE WITH RESERVE**

**ESCALATE**

*read_sales_briefing_guidance*

**allocation_notice_agent**
*draft_allocation_notice*

briefing_guidance

allocation_notice

**sales_briefing_agent**
*draft_sales_briefing*

sales_briefing

**record_allocation_outcome**

database/fairshare.db

### OUTPUT

Running the six entry points in order produces the following inside **Project/**.

### FILES CREATED

| Path | Created by | Contents |
|---|---|---|
| **database/fairshare.db** | **run_database_setup.py** | The six tables the application writes to, and the fourteen seed accounts |
| **processed/order_history_train_clean.csv** | **run_data_pipeline.py** | Cleaned inflated-order training data, every account canonical and every quantity in whole cases |
| **processed/order_history_eval_clean.csv** | **run_data_pipeline.py** | Cleaned inflated-order held-out data |
| **processed/weekly_demand_train_clean.csv** | **run_data_pipeline.py** | Cleaned demand training data |
| **processed/weekly_demand_eval_clean.csv** | **run_data_pipeline.py** | Cleaned demand held-out data |
| **processed/customer_emails_clean.csv** | **run_data_pipeline.py** | Cleaned customer emails |
| **output/data_quality_report.json** | **run_data_pipeline.py** | How dirty each raw extract was, and what cleaning did to it |
| **output/models/inflated_order_model.pkl** | **run_model_training.py** | The saved inflated-order model |
| **output/models/demand_model.pkl** | **run_model_training.py** | The saved demand model |
| **output/models/request_model.pkl** | **run_model_training.py** | The saved request model, text handling included |
| **output/models/inflated_order_features.json** | **run_model_training.py** | The columns the inflated-order model expects, in order |
| **output/models/demand_features.json** | **run_model_training.py** | The columns the demand model expects, in order |
| **output/model_metrics.json** | **run_model_training.py** | Every score of every saved model |
| **database/chroma_store/** | **run_knowledge_indexing.py** | The searchable knowledge base |
| **database/checkpoint.db** | **run_api_service.py** | Every run's workflow state, checkpointed after each node |

### INFORMATION EACH COMMAND MUST REPORT

The marking suite reads no console output.

| Command | What it must make clear |
|---|---|
| **run_database_setup.py** | Where the database was created, which tables exist, each account it created, and how many accounts were created and how many were already present |
| **run_data_pipeline.py** | For each of the five extracts, how many rows arrived and how many survived cleaning, and where the quality report was written |
| **run_model_training.py** | For each of the three models, the score on data it did not learn from, the score on data it did, the gap between them, and whether it clears the minimum config sets for it |
| **run_knowledge_indexing.py** | How many documents were read, how many sections were found and how many passages were stored, and every category's four rules read back from its own policy |
| **run_api_service.py** | That the service is listening on port **8081**, and where its interactive documentation can be opened |
| **run_dashboard.py** | That the dashboard is listening on port **8080**, and the address at which it can be opened |

### EXPECTED OUTPUT

One correct solution's output. Scores and the passage count depend on the models and the chunking; the row, document and section counts are fixed. Only the hand hygiene policy's rules are shown; the other four are shown in outline.

**ENTRY POINT 1 — run_database_setup.py**

```
database: database/fairshare.db
  tables: agent_runs, allocation_decisions, allocation_reviews, chat_turns, llm_calls, user_acc
  account created  priya    Priya Raman    Supply Planning Manager
```

```
  account created  tom      Tom Hadley     Supply Planning Manager
  account created  grace    Grace Odum     Supply Planning Manager
  account created  ben      Ben Carver     Supply Planner
  ...
  account created  ruth     Ruth Ellis     Supply Planner
  account created  dev      Dev Patel      Customer Service Adviser
  ...
  account created  jack     Jack Turner    Customer Service Adviser
  accounts created: 14, already present: 0
```

Run a second time it creates nothing and reports **accounts created: 0, already present: 14**.

**ENTRY POINT 2 — run_data_pipeline.py**

```
order_history_train              34338 received ->  31663 kept  (2675 removed)
order_history_eval                8870 received ->   8218 kept  (652 removed)
weekly_demand_train              32005 received ->  30855 kept  (1150 removed)
weekly_demand_eval               10579 received ->  10214 kept  (365 removed)
customer_emails                  36712 received ->  35223 kept  (1489 removed)

quality report written to output/data_quality_report.json
```

**ENTRY POINT 3 — run_model_training.py**

```
training three models
  inflated order  held-out ROC AUC N.NNNN  (train N.NNNN, gap +N.NNNN)
  demand          held-out R2 N.NNNN  MAE N.NN cases  (train N.NNNN, gap +N.NNNN)
  request         held-out macro F1 N.NNNN  (train N.NNNN, gap +N.NNNN)

metrics written to output/model_metrics.json
every model reaches the minimum the configuration sets for it
```

A model below its minimum is named instead, and the run still finishes.

**ENTRY POINT 4 — run_knowledge_indexing.py**

```
building the allocation knowledge index
    documents read:  16
    sections found:  145
    chunks stored:   NNN
    store location:  database/chroma_store

rules read from each product category's own allocation policy
  Hand hygiene           Allocation-Policy-Hand-Hygiene.pdf
      usual demand over 8 weeks, an order capped at 1.0 times it, Healthcare protected, a reser
  Surface disinfection   Allocation-Policy-Surface-Disinfection.pdf
      usual demand over N weeks, an order capped at N.N times it, <protected> protected, <reser
  Laundry                Allocation-Policy-Laundry.pdf
      ...
  Dishwash               Allocation-Policy-Dishwash.pdf
      ...
  Toilet care            Allocation-Policy-Toilet-Care.pdf
      ...
```

Where no channel is protected, **** reads **nobody**, and two protected channels are joined by **and**; where no reserve is held, **** reads **no reserve**, and otherwise **a reserve of up to N percent**.

**ENTRY POINT 5 — run_api_service.py**

```
Starting FairShare AI API on port 8081. Interactive documentation: http://localhost:8081/docs
INFO:     Started server process [NNNN]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)
```

With the saved inflated-order model missing, it names what is missing and stops:

```
The service cannot start.
  output/models/inflated_order_model.pkl is missing; it comes from run_model_training.py
```

**ENTRY POINT 6 — run_dashboard.py**

```
  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8080
```

**DASHBOARD OUTPUT:**

Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Shortage queue

Products on allocation
40

With RELEASE open
30

With ESCALATE alone open
10

Cases available this week
54,978

Category

All | Hand hygiene | Surface disinfection | Laundry | Dishwash | Toilet care

| # | Product | Name | Category | This week | Ordered | May have | Cover % | Protected in full | Decisions open | Shortfall next week |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | DW-0900 | Wrenfield Dishwash Liquid 900 ml | Dishwash | 2,105 | 6,713 | 5,521 | 38.13 | Yes | ESCALATE | 3,223 |
| 2 | SM-0750 | Wrenfield Antibacterial Multi-Surface Sp | Surface disinfection | 2,212 | 5,518 | 3,942 | 56.11 | Yes | RELEASE, RELEASE WITH RESERVE, ESCALATE | 3,096 |
| 3 | LS-1000 | Wrenfield Stain Remover 1 litre | Laundry | 1,637 | 5,854 | 4,192 | 39.05 | Yes | ESCALATE | 2,700 |
| 4 | SF-1000 | Wrenfield Floor Disinfectant 1 litre | Surface disinfection | 320 | 3,791 | 2,610 | 12.26 | No | ESCALATE | 2,602 |
| 5 | HS-0050 | Wrenfield Hand Sanitiser 50 ml | Hand hygiene | 224 | 4,339 | 2,369 | 9.46 | No | ESCALATE | 2,579 |
| 6 | LW-0750 | Wrenfield Delicates Wash 750 ml | Laundry | 1,519 | 3,002 | 2,549 | 59.59 | Yes | RELEASE, ESCALATE | 1,962 |
| 7 | HW-0250 | Wrenfield Hand Wash 250 ml | Hand hygiene | 2,339 | 4,656 | 3,047 | 76.76 | Yes | RELEASE, RELEASE WITH RESERVE, ESCALATE | 1,837 |
| 8 | TG-0750 | Wrenfield Thick Toilet Gel 750 ml | Toilet care | 1,817 | 3,901 | 3,108 | 58.46 | Yes | RELEASE, ESCALATE | 1,758 |
| 9 | LL-1000 | Wrenfield Laundry Liquid 1 litre | Laundry | 1,699 | 6,176 | 3,984 | 42.65 | Yes | ESCALATE | 1,751 |
| 10 | SK-0750 | Wrenfield Kitchen Disinfectant Spray 75 | Surface disinfection | 2,656 | 8,338 | 3,287 | 80.80 | Yes | RELEASE, RELEASE WITH RESERVE, ESCALATE | 1,630 |
| 11 | SM-5000 | Wrenfield Multi-Surface Cleaner 5 litre | Surface disinfection | 1,444 | 4,049 | 2,057 | 70.20 | Yes | RELEASE, RELEASE WITH RESERVE, ESCALATE | 1,537 |
| 12 | HS-0250 | Wrenfield Hand Sanitiser 250 ml | Hand hygiene | 1,596 | 3,683 | 2,440 | 65.41 | Yes | RELEASE, RELEASE WITH RESERVE, ESCALATE | 1,473 |
| 13 | HF-0400 | Wrenfield Foaming Hand Wash 400 ml | Hand hygiene | 1,547 | 5,076 | 2,538 | 60.95 | Yes | RELEASE, RELEASE WITH RESERVE, ESCALATE | 1,433 |
| 14 | SV-0750 | Wrenfield Virucidal Disinfectant Spray 7 | Surface disinfection | 1,403 | 2,412 | 1,691 | 82.97 | Yes | RELEASE, RELEASE WITH RESERVE, ESCALATE | 1,313 |
| 15 | HW-0300 | Wrenfield Moisturising Hand Wash 300 r | Hand hygiene | 1,172 | 2,521 | 1,847 | 63.45 | Yes | RELEASE, RELEASE WITH RESERVE, ESCALATE | 1,276 |

Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Allocation detail

Product

DW-0900 Wrenfield Dishwash Liquid 900 ml

DW-0900 Wrenfield Dishwash Liquid 900 ml
SM-0750 Wrenfield Antibacterial Multi-Surface Spray 750 ml
LS-1000 Wrenfield Stain Remover 1 litre
SF-1000 Wrenfield Floor Disinfectant 1 litre
HS-0050 Wrenfield Hand Sanitiser 50 ml
LW-0750 Wrenfield Delicates Wash 750 ml
HW-0250 Wrenfield Hand Wash 250 ml
TG-0750 Wrenfield Thick Toilet Gel 750 ml

losure shortage. Divided under Allocation-Policy-Dishwash.pdf.

Customers may have
5,521

Cover
38.13%

Shortfall next week
3,223

RELEASE

RELEASE is not open: the cases cover less than 50 percent of what customers may have




Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Allocation detail

Product

SK-0750 Wrenfield Kitchen Disinfectant Spray 750 ml

Surface disinfection | RELEASE open | RELEASE WITH RESERVE open | ESCALATE open | Queue position 10 of 40

6x750ml, 6 units to a case and 100 cases to a pallet. Short because: Demand surge after news coverage. Divided under Allocation-Policy-Surface-Disinfection.pdf.

Cases this week
2,656

Ordered
8,338

Customers may have
3,287

Cover
80.80%

Shortfall next week
1,630

Division | Customers and rules | Orders and emails | Review

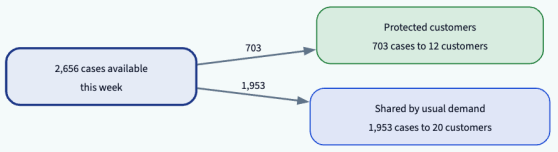
Release

RELEASE | RELEASE WITH RESERVE

Division | Customers and rules | Orders and emails | Review

Release

RELEASE | RELEASE WITH RESERVE

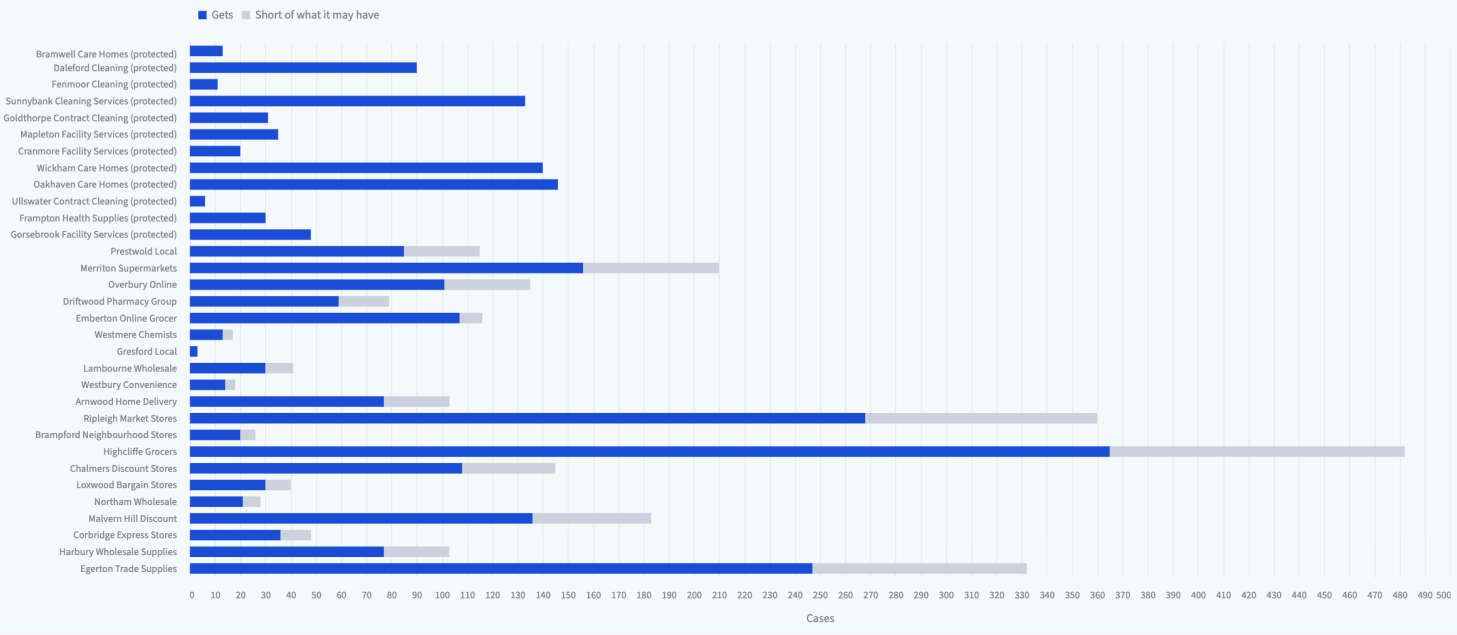



Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out



Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

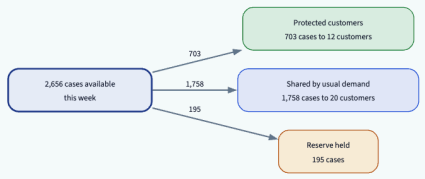
Priya Raman
Supply Planning Manager

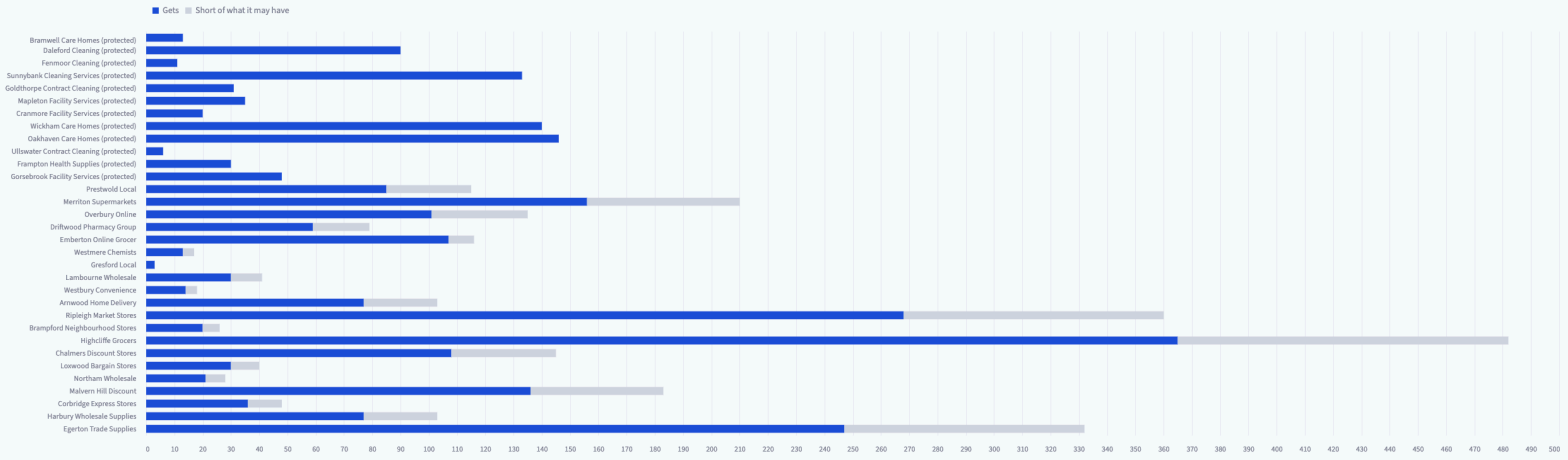
Sign out

Division Customers and rules Orders and emails Review

Release

RELEASE RELEASE WITH RESERVE




Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Allocation detail

Product

SK-0750 Wrenfield Kitchen Disinfectant Spray 750 ml

Surface disinfection RELEASE open RELEASE WITH RESERVE open ESCALATE open Queue position 10 of 40

6x750ml, 6 units to a case and 100 cases to a pallet. Short because: Demand surge after news coverage. Divided under Allocation-Policy-Surface-Disinfection.pdf.

| Cases this week | Ordered | Customers may have | Cover | Shortfall next week |
|---|---|---|---|---|
| 2,656 | 8,338 | 3,287 | 80.80% | 1,630 |

Division Customers and rules Orders and emails Review

**Rules from Allocation-Policy-Surface-Disinfection.pdf**

| Rule | Read as | Policy |
|---|---|---|
| Usual demand | 8 weeks | Usual demand is the average weekly cases shipped over the last 8 weeks. |
| Cap on an order | 1.0 times usual demand | An order counts for no more than 1.0 times the customer's usual demand. |
| Protected customers | Healthcare and Facilities | Customers in the Healthcare and Facilities channels are protected and are served first. |
| Reserve | Up to 10 percent | Up to 10 percent of the cases left after protected customers may be held back as a reserve. |

| Customer | Account | Channel | Protected | Ordered | Usual week | May have | RELEASE | RELEASE WITH RESERVE | Most urgent request |
|---|---|---|---|---|---|---|---|---|---|
| Bramwell Care Homes | CUS-0002 | Healthcare | Yes | 69 | 13.0000 | 13 | 13 | 13 | |
| Prestwold Local | CUS-0003 | Convenience | No | 479 | 115.2500 | 115 | 85 | 77 | |
| Merriton Supermarkets | CUS-0009 | Supermarket | No | 220 | 210.3750 | 210 | 156 | 141 | |
| Overbury Online | CUS-0011 | Online | No | 344 | 135.7500 | 135 | 101 | 91 | Quantity increase |
| Driftwood Pharmacy Group | CUS-0016 | Pharmacy | No | 134 | 79.2500 | 79 | 59 | 53 | |
| Emberton Online Grocer | CUS-0022 | Online | No | 116 | 144.0000 | 116 | 107 | 96 | Quantity increase |
| Daleford Cleaning | CUS-0024 | Facilities | Yes | 425 | 90.2500 | 90 | 90 | 90 | |
| Fenmoor Cleaning | CUS-0026 | Facilities | Yes | 36 | 11.1250 | 11 | 11 | 11 | |
| Sunnybank Cleaning Services | CUS-0032 | Facilities | Yes | 340 | 133.1250 | 133 | 133 | 133 | Quantity increase |
| Westmere Chemists | CUS-0048 | Pharmacy | No | 23 | 17.1250 | 17 | 13 | 11 | |

Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Allocation detail

Product

SK-0750 Wrenfield Kitchen Disinfectant Spray 750 ml

Surface disinfection RELEASE open RELEASE WITH RESERVE open ESCALATE open Queue position 10 of 40

6x750ml, 6 units to a case and 100 cases to a pallet. Short because: Demand surge after news coverage. Divided under Allocation-Policy-Surface-Disinfection.pdf.

| Cases this week | Ordered | Customers may have | Cover | Shortfall next week |
|---|---|---|---|---|
| 2,656 | 8,338 | 3,287 | 80.80% | 1,630 |

Division Customers and rules Orders and emails Review

HIGH 6 MEDIUM 21 LOW 14

| Order line | Purchase order | Route | Customer | Cases | Inflated-order score | Band |
|---|---|---|---|---|---|---|
| OL-8000422 | PO-791439 | EDI | Malvern Hill Discount | 435 | 0.7153 | HIGH |
| OL-8000330 | PO-787121 | EDI | Merriton Supermarkets | 220 | 0.2351 | LOW |
| OL-8000925 | PO-796812 | Email | Fenmoor Cleaning | 9 | 0.2663 | LOW |
| OL-8000649 | PO-796853 | Portal | Frampton Health Supplies | 67 | 0.2280 | LOW |
| OL-8001280 | PO-787362 | Email | Mapleton Facility Services | 123 | 0.7301 | HIGH |
| OL-8000212 | PO-795390 | Email | Overbury Online | 344 | 0.4895 | MEDIUM |
| OL-8000413 | PO-793671 | Portal | Arnwood Home Delivery | 103 | 0.2128 | LOW |
| OL-8001156 | PO-785498 | Email | Fenmoor Cleaning | 18 | 0.4959 | MEDIUM |
| OL-8001082 | PO-785033 | Portal | Gresford Local | 6 | 0.3765 | MEDIUM |
| OL-8000501 | PO-782282 | Email | Cranmore Facility Services | 20 | 0.1311 | LOW |

Quantity increase **Overbury Online**, buyer, Friday 10:18: Following up

Dear Wrenfield team, We underordered Wrenfield Kitchen Disinfectant Spray 750 ml this week and need another 120 cases on the same delivery. Can someone look at this for us, please? Everything else on the order stays as it is. I hope you are well. Regards, Buyer, Overbury Online

Quantity increase **Emberton Online Grocer**, procurement officer, Wednesday 10:42: Top-up order

Good afternoon, Please add another 150 cases of SK-0750 to PO-537844. Everything else on the order stays as it is. Following our call this morning, here is the detail. Best wishes, Procurement officer, Emberton Online Grocer

Quantity increase **Sunnybank Cleaning Services**, procurement officer, Wednesday 13:04: Stock update

Hi team, We need to increase our current order of Wrenfield Kitchen Disinfectant Spray 750 ml from 390 to 475 cases. Best wishes, Procurement officer, Sunnybank Cleaning Services

Quantity increase **Arnwood Home Delivery**, buyer, Wednesday 14:19: Following up

Hello, Please confirm the revised order total when you can. We would like 155 extra cases of Wrenfield Kitchen Disinfectant Spray 750 ml on top of what we ordered this week. Regards, Buyer, Arnwood Home Delivery

Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

Quantity increase **Emberton Online Grocer**, procurement officer, Wednesday 10:42: Top-up order

Good afternoon, Please add another 150 cases of SK-0750 to PO-537844. Everything else on the order stays as it is. Following our call this morning, here is the detail. Best wishes, Procurement officer, Emberton Online Grocer

Quantity increase **Sunnybank Cleaning Services**, procurement officer, Wednesday 13:04: Stock update

Hi team, We need to increase our current order of Wrenfield Kitchen Disinfectant Spray 750 ml from 390 to 475 cases. Best wishes, Procurement officer, Sunnybank Cleaning Services

Quantity increase **Arnwood Home Delivery**, buyer, Wednesday 14:19: Following up

Hello, Please confirm the revised order total when you can. We would like 155 extra cases of Wrenfield Kitchen Disinfectant Spray 750 ml on top of what we ordered this week. Regards, Buyer, Arnwood Home Delivery

Quantity increase **Arnwood Home Delivery**, buyer, Friday 15:49: Re: your email

Hi, Please add another 140 cases of Wrenfield Kitchen Disinfectant Spray 750 ml to PO-432812. Kind regards, Buyer, Arnwood Home Delivery

Substitution accepted **Brampford Neighbourhood Stores**, operations manager, Monday 12:52: Happy to take another size

Hello supply team, An alternative pack size is fine for us, so please send Wrenfield Virucidal Disinfectant Spray 750 ml rather than Wrenfield Kitchen Disinfectant Spray 750 ml. Thanks, Operations manager, Brampford Neighbourhood Stores

Delivery complaint **Brampford Neighbourhood Stores**, buyer, Wednesday 17:53: Stock update

Hello supply team, We have bought from Wrenfield for a long time. We would appreciate a quick reply. We did not get what we ordered this week. Thank you, Buyer, Brampford Neighbourhood Stores

Substitution accepted **Loxwood Bargain Stores**, store manager, Tuesday 09:57: Happy to take another size

Hello supply team, We are happy to take the 500 ml surface disinfectant spray instead of the 750 ml kitchen disinfectant spray if that helps. The order reference is PO-514185. Regards, Store manager, Loxwood Bargain Stores

Delivery complaint **Northam Wholesale**, operations manager, Wednesday 14:31: Delivery problem

Hello, We received Wrenfield Bathroom Disinfectant Spray 750 ml on PO-572502 when we had ordered Wrenfield Kitchen Disinfectant Spray 750 ml. Thanks, Operations manager, Northam Wholesale

Allocation query **Corbridge Express Stores**, buyer, Thursday 10:46: How is our share decided?

Hi team, This concerns our account with you. Which customers are being served first while the 750 ml kitchen disinfectant spray is short? We are not complaining about the delivery itself. Thank you, Buyer, Corbridge Express Stores

Quantity increase **Egerton Trade Supplies**, operations manager, Tuesday 09:28: Top-up order

Hello, Our account code is CUS-0165. We have been looking at our numbers for kitchen disinfectant spray. We would appreciate a quick reply. Thank you for the update earlier. Thank you, Operations manager, Egerton Trade Supplies

Allocation query **Gorsebrook Facility Services**, operations manager, Friday 11:43: Our allocation

Hello supply team, Thanks for all your help during the shortage. How long will the 750 ml kitchen disinfectant spray stay on allocation, and how is our share set? Thank you, Operations manager, Gorsebrook Facility Services

Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Allocation detail

Product

SK-0750 Wrenfield Kitchen Disinfectant Spray 750 ml

Surface disinfection RELEASE open RELEASE WITH RESERVE open ESCALATE open Queue position 10 of 40

6x750ml, 6 units to a case and 100 cases to a pallet. Short because: Demand surge after news coverage. Divided under Allocation-Policy-Surface-Disinfection.pdf.

| Cases this week | Ordered | Customers may have | Cover | Shortfall next week |
|---|---|---|---|---|
| 2,656 | 8,338 | 3,287 | 80.80% | 1,630 |

Division Customers and rules Orders and emails Review

What do you want looked at?

Need more info.

Press ⌘+Enter to submit for

Start the review

Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Allocation detail

Product

SK-0750 Wrenfield Kitchen Disinfectant Spray 750 ml

REV-E46A14A49BE4 is waiting for a Supply Planning Manager, recommending RELEASE.

Surface disinfection RELEASE open RELEASE WITH RESERVE open ESCALATE open Queue position 10 of 40 Pending decision

6x750ml, 6 units to a case and 100 cases to a pallet. Short because: Demand surge after news coverage. Divided under Allocation-Policy-Surface-Disinfection.pdf.

| Cases this week | Ordered | Customers may have | Cover | Shortfall next week |
|---|---|---|---|---|
| 2,656 | 8,338 | 3,287 | 80.80% | 1,630 |

Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Decision queue

3 waiting for a decision

SK-0750 | Wrenfield Kitchen Disinfectant Spray 750 ml, Surface disinfection

SK-0750 | Wrenfield Kitchen Disinfectant Spray 750 ml, Surface disinfection
SM-5000 | Wrenfield Multi-Surface Cleaner 5 litre, Surface disinfection
LS-1000 | Wrenfield Stain Remover 1 litre, Laundry

Wrenfield Kitchen Disinfectant Spray 750 ml (SK-0750) is currently on allocation due to a demand surge following recent news coverage, with open orders totaling 8,338 cases against 2,656 available this week. The review verified that all 12 protected healthcare and facilities accounts are fully served with their 703 requested cases, and an overall cover of 80.80 percent has been established. Several major non-protected accounts show inflated orders driven by shortage anxiety, while various customer requests for quantity increases, substitutions, and delivery queries have been reviewed strictly against the allocation policy rules. Because every protected customer is served in full and the available cases cover more than 50 percent of what customers may have without holding any reserve stock back, the standard release option is fully supported. Therefore, RELEASE fits and why.

| Customer | Channel | Protected | Ordered | May have | RELEASE | RELEASE WITH RESERVE |
|---|---|---|---|---|---|---|
| Bramwell Care Homes | Healthcare | Yes | 69 | 13 | 13 | 13 |
| Prestwold Local | Convenience | No | 479 | 115 | 85 | 77 |
| Merriton Supermarkets | Supermarket | No | 220 | 210 | 156 | 141 |
| Overbury Online | Online | No | 344 | 135 | 101 | 91 |
| Driftwood Pharmacy Group | Pharmacy | No | 134 | 79 | 59 | 53 |
| Emberton Online Grocer | Online | No | 116 | 116 | 107 | 96 |
| Daleford Cleaning | Facilities | Yes | 425 | 90 | 90 | 90 |
| Fenmoor Cleaning | Facilities | Yes | 36 | 11 | 11 | 11 |
| Sunnybank Cleaning Services | Facilities | Yes | 340 | 133 | 133 | 133 |
| Westmere Chemists | Pharmacy | No | 23 | 17 | 13 | 11 |

Request and plan, opened by priya

Shortage queue
Allocation detail
Decision queue
Audit trail
Allocation chat

FairShare AI

Priya Raman
Supply Planning Manager

Sign out

# Decision queue

3 waiting for a decision

SK-0750 | Wrenfield Kitchen Disinfectant Spray 750 ml, Surface disinfection

Recommended: RELEASE Allocation-Policy-Surface-Disinfection.pdf

You opened this review, so another Supply Planning Manager must decide it.

| Cases available | Customers may have | Cover | Shortfall next week |
|---|---|---|---|
| 2,656 | 3,287 | 80.80% | 1,630 |

Wrenfield Kitchen Disinfectant Spray 750 ml (SK-0750) is currently on allocation due to a demand surge following recent news coverage, with open orders totaling 8,338 cases against 2,656 available this week. The review verified that all 12 protected healthcare and facilities accounts are fully served with their 703 requested cases, and an overall cover of 80.80 percent has been established. Several major non-protected accounts show inflated orders driven by shortage anxiety, while various customer requests for quantity increases, substitutions, and delivery queries have been reviewed strictly against the allocation policy rules. Because every protected customer is served in full and the available cases cover more than 50 percent of what customers may have without holding any reserve stock back, the standard release option is fully supported. Therefore, RELEASE fits and why.

| Customer | Channel | Protected | Ordered | May have | RELEASE | RELEASE WITH RESERVE |
|---|---|---|---|---|---|---|
| Bramwell Care Homes | Healthcare | Yes | 69 | 13 | 13 | 13 |
| Prestwold Local | Convenience | No | 479 | 115 | 85 | 77 |
| Merriton Supermarkets | Supermarket | No | 220 | 210 | 156 | 141 |
| Overbury Online | Online | No | 344 | 135 | 101 | 91 |
| Driftwood Pharmacy Group | Pharmacy | No | 134 | 79 | 59 | 53 |
| Emberton Online Grocer | Online | No | 116 | 116 | 107 | 96 |
| Daleford Cleaning | Facilities | Yes | 425 | 90 | 90 | 90 |
| Fenmoor Cleaning | Facilities | Yes | 36 | 11 | 11 | 11 |
| Sunnybank Cleaning Services | Facilities | Yes | 340 | 133 | 133 | 133 |
| Westmere Chemists | Pharmacy | No | 23 | 17 | 13 | 11 |

Request and plan, opened by priya

**Request**

Need more info.

**Plan**

PRIORITY: Fair allocation of available stock during a demand surge

CHECKS: 1. Establish the total volume of open orders against the 2,656 cases available this week. 2. Review the sequence of orders as received to apply the order intake correctly. 3. Examine customer requests for any inflated orders or specific operational context. 4. Assess the mix of customer channels represented in the open orders.

RATIONALE: A structured review of order sequences and volumes ensures the available stock is managed transparently before distribution. Examining customer context and channel mix maintains fairness and integrity during an unexpected demand surge.

Shortage queue
Allocation detail
Decision queue
**Audit trail**
Allocation chat

**FairShare AI**

**Priya Raman**
Supply Planning Manager

Sign out

# Audit trail

Allocation reviews | Chat log

Status

Pending decision | Released | Released with reserve | Escalated

| Product | Category | Status | Decision | Cases released | Model calls |
|---|---|---|---|---|---|
| SK-0750 | Surface disinfection | Pending decision | | | 5 |
| DW-0450 | Dishwash | Released | RELEASE by Grace Odum | 1,000 | 7 |
| DG-1000 | Dishwash | Released | RELEASE by Grace Odum | 156 | 7 |
| SM-5000 | Surface disinfection | Pending decision | | | 5 |
| LS-1000 | Laundry | Pending decision | | | 5 |

Shortage queue
Allocation detail
Decision queue
Audit trail
**Allocation chat**

**FairShare AI**

**Priya Raman**
Supply Planning Manager

Sign out

# Allocation chat

Are care homes served first for hand sanitiser?

POLICY RULES

Under the Wrenfield Hygiene Allocation Policy: Hand Hygiene, customers in the Healthcare channel are protected and are served first. Where this section protects customers, each protected customer is served before every other customer, with all that it may have, as far as the cases allow. A customer's channel is read from the customer master, never from the way an order was written.

› Sources

How many weeks of shipments make up usual demand for laundry?

POLICY RULES

Under the Wrenfield Hygiene Allocation Policy: Laundry, usual demand is the average weekly cases shipped over the last 13 weeks. The weeks are counted back from the current week, where week 1 is last week.

› Sources

How many weeks of shipments make up usual demand under the laundry policy?

POLICY RULES

Under the Wrenfield Hygiene Allocation Policy: Laundry, usual demand is the average weekly cases shipped over the last 13 weeks. The weeks are counted back from the current week, where week 1 is last week.

› Sources

And what is its cap?

POLICY RULES

Under the Wrenfield Hygiene Allocation Policy: Laundry, an order counts for no more than 1.2 times the customer's usual demand. What a customer may have is the lower of the cases it ordered and this multiple of its usual demand, rounded down to whole cases. The multiple is applied exactly, to the fraction, before anything is rounded. Cases a customer ordered above what it may have are not allocated while the product is on allocation, and are not carried forward to a later week.

› Sources

Does it protect any customers?

POLICY RULES

Under the Wrenfield Hygiene Allocation Policy: Laundry, no customer is protected. Where this section protects customers, each protected customer is served before every other customer, with all that it may have, as far as the cases allow.

› Sources

What about the hand hygiene policy?

POLICY RULES

Under the Wrenfield Hygiene Allocation Policy: Hand Hygiene, usual demand is the average weekly cases shipped over the last 8 weeks. An order counts for no more than 1.0 times the customer's usual demand, and customers in the Healthcare channel are protected and served first. Up to 10 percent of the cases left after protected customers may be held back as a reserve when a Supply Planning Manager decides RELEASE WITH RESERVE.

› Sources

And what is the cap?

POLICY RULES

Under the Wrenfield Hygiene Allocation Policy: Hand Hygiene, an order counts for no more than 1.0 times the customer's usual demand. What a customer may have is the lower of the cases it ordered and this multiple of its usual demand, rounded down to whole cases. The multiple is applied exactly, to the fraction, before anything is rounded. Cases a customer ordered above what it may have are not allocated while the product is on allocation, and are not carried forward to a later week.

› Sources

Ask about a policy, the allocation rules, a request guide, or an allocation by its account code and delivery postcode

Shortage queue
Allocation detail
**Decision queue**
Audit trail
Allocation chat

**FairShare AI**

**Tom Hadley**
Supply Planning Manager

Sign out

# Decision queue

3 waiting for a decision

SK-0750 | Wrenfield Kitchen Disinfectant Spray 750 ml, Surface disinfection

Recommended: RELEASE  Allocation-Policy-Surface-Disinfection.pdf

| Cases available | Customers may have | Cover | Shortfall next week |
|---|---|---|---|
| 2,656 | 3,287 | 80.80% | 1,630 |

Wrenfield Kitchen Disinfectant Spray 750 ml (SK-0750) is currently on allocation due to a demand surge following recent news coverage, with open orders totaling 8,338 cases against 2,656 available this week. The review verified that all 12 protected healthcare and facilities accounts are fully served with their 703 requested cases, and an overall cover of 80.80 percent has been established. Several major non-protected accounts show inflated orders driven by shortage anxiety, while various customer requests for quantity increases, substitutions, and delivery queries have been reviewed strictly against the allocation policy rules. Because every protected customer is served in full and the available cases cover more than 50 percent of what customers may have without holding any reserve stock back, the standard release option is fully supported. Therefore, RELEASE fits and why.

| Customer | Channel | Protected | Ordered | May have | RELEASE | RELEASE WITH RESERVE |
|---|---|---|---|---|---|---|
| Bramwell Care Homes | Healthcare | Yes | 69 | 13 | 13 | 13 |
| Prestwold Local | Convenience | No | 479 | 115 | 85 | 77 |
| Merriton Supermarkets | Supermarket | No | 220 | 210 | 156 | 141 |
| Overbury Online | Online | No | 344 | 135 | 101 | 91 |
| Driftwood Pharmacy Group | Pharmacy | No | 134 | 79 | 59 | 53 |
| Emberton Online Grocer | Online | No | 116 | 116 | 107 | 96 |
| Daleford Cleaning | Facilities | Yes | 425 | 90 | 90 | 90 |
| Fenmoor Cleaning | Facilities | Yes | 36 | 11 | 11 | 11 |
| Sunnybank Cleaning Services | Facilities | Yes | 340 | 133 | 133 | 133 |
| Westmere Chemists | Pharmacy | No | 23 | 17 | 13 | 11 |

› Request and plan, opened by priya

**Your decision**

- RELEASE
- RELEASE WITH RESERVE
- ESCALATE

Reason

For example: every protected customer is served in full, so the cases can go out.

Record the decision

Shortage queue
Allocation detail
**Decision queue**
Audit trail
Allocation chat

**FairShare AI**

**Tom Hadley**
Supply Planning Manager

Sign out

# Decision queue

SK-0750 released with reserve: 2,461 cases released, 195 held in reserve and 0 left in stock.

2 waiting for a decision

SM-5000 | Wrenfield Multi-Surface Cleaner 5 litre, Surface disinfection

Recommended: RELEASE WITH RESERVE  Allocation-Policy-Surface-Disinfection.pdf

| Cases available | Customers may have | Cover | Shortfall next week |
|---|---|---|---|
| 1,444 | 2,057 | 70.20% | 1,537 |

We are currently managing a significant supply shortfall for SM-5000 due to planned maintenance overruns, with total orders of 4,049 cases against 1,444 cases available. Our review confirms that all protected healthcare and facilities customers have been served in full, and non-protected accounts are receiving 70.20 percent of their eligible demand. We have identified several instances of speculative stockpiling among wholesale accounts, which we have mitigated by strictly adhering to the allocation policy limits. While we have received multiple requests for expedites and quantity increases, these are being managed in accordance with the established category policy to ensure fair distribution. Given that we have successfully protected essential service continuity and the current fulfillment rate exceeds the 50 percent threshold, we have the capacity to either release all stock or hold a 10 percent reserve for potential volatility. RELEASE WITH RESERVE is the most appropriate decision as it secures our ability to support urgent clinical needs while maintaining the integrity of the allocation plan.

| Customer | Channel | Protected | Ordered | May have | RELEASE | RELEASE WITH RESERVE |
|---|---|---|---|---|---|---|
| Ollerton Food Halls | Supermarket | No | 31 | 24 | 6 | 5 |
| Castleford Wholesale Supplies | Wholesale | No | 23 | 23 | 6 | 5 |
| Netherfield Clinical Supplies | Healthcare | Yes | 534 | 201 | 201 | 201 |
| Rosedale Hospital Supplies | Healthcare | Yes | 119 | 119 | 119 | 119 |
| Tenbury Trade Supplies | Wholesale | No | 37 | 37 | 9 | 8 |
| Orchardleigh Facilities | Facilities | Yes | 384 | 91 | 91 | 91 |
| Kirkwood Contract Cleaning | Facilities | Yes | 306 | 103 | 103 | 103 |
| Millbrook Health Supplies | Healthcare | Yes | 8 | 8 | 8 | 8 |
| Mapleton Facility Services | Facilities | Yes | 40 | 12 | 12 | 12 |
| Dryden Facility Services | Facilities | Yes | 28 | 26 | 26 | 26 |

› Request and plan, opened by priya

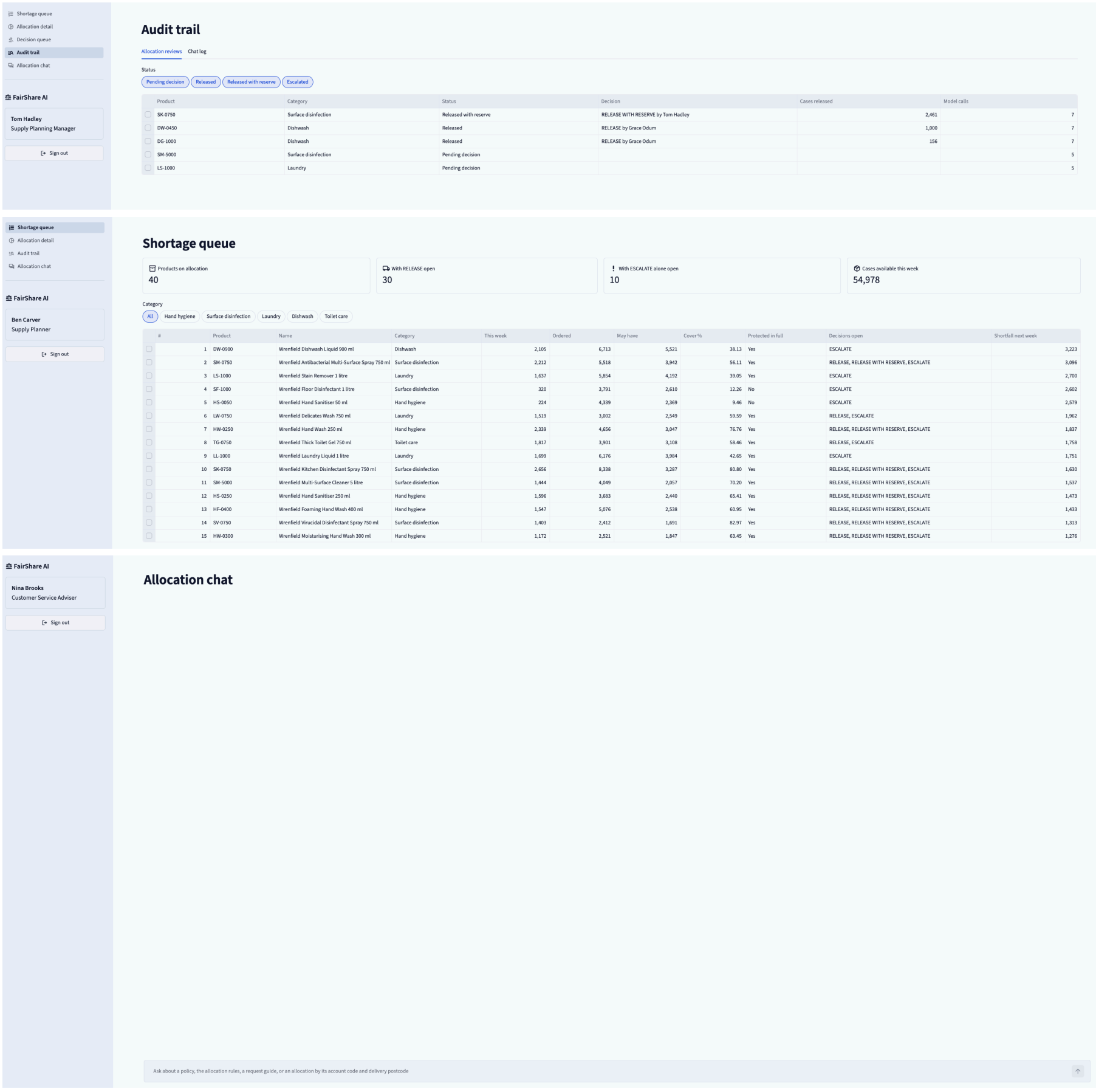
**Your decision**

- RELEASE
- RELEASE WITH RESERVE
- ESCALATE

Reason

For example: every protected customer is served in full, so the cases can go out.

Record the decision




**IMPLEMENTATION STEPS**

Every command is run from the Project folder.

**STEP 0. INSTALL AND CONFIGURE**

**cd Project**

**pip install -r requirements.txt**

**STEP 1. CREATE THE DATABASE**

**python3 run_database_setup.py**

**STEP 2. CLEAN THE RAW DATA**

**python3 run_data_pipeline.py**

**STEP 3. TRAIN THE MODELS**

**python3 run_model_training.py**

**STEP 4. BUILD THE KNOWLEDGE BASE**

**python3 run_knowledge_indexing.py**

**STEP 5. START THE REST SERVICE**

Leave this running and use a second terminal for anything else.

**python3 run_api_service.py**

**STEP 6. START THE DASHBOARD**

In a third terminal, with the service still running:

**cd Project**

**python3 -m streamlit run run_dashboard.py**